

FRC DOCS / WPILIB

Basic Programming

基础编程



11 篇中英对照文档 • 按 WPILib Docs 目录顺序

FRC 技术文档翻译合集

Contents / 目录

01	Git Version Control Introduction / Git 版本控制简介
02	The Java Units Library / Java Units Library
03	Joysticks / Joystick
04	Coordinate System / 坐标系
05	Setting Robot Preferences / 设置 Robot Preferences
06	Robot Project Deploy Directory / Robot Project Deploy Directory
07	Using Test Mode / 使用 Test Mode
08	Reading Stacktraces / 阅读 Stack Trace
09	Treating Functions as Data / 将函数作为数据处理
10	Get Alliance Color / 获取 Alliance Color
11	Java Garbage Collection / Java 垃圾回收

Git Version Control Introduction / Git 版本控制简介

IMPORTANT: A more in-depth guide on Git is available on the Git website.

重要： Git 网站提供了更深入的 Git 指南。

Git is a Distributed Version Control System (VCS) created by Linus Torvalds, also known for creating and maintaining the Linux kernel. Version Control is a system for tracking changes of code for developers. The advantages of Git Version Control are:

Git 是由 Linus Torvalds 创建的分布式版本控制系统（Distributed Version Control System，VCS）；他也因创建和维护 Linux Kernel 而闻名。版本控制是一种供开发者跟踪代码变更的系统。Git 版本控制的优点包括：

- Separation of testing environments into *branches*
- 将测试环境划分到不同的 *branch* 中
- Ability to navigate to a particular *commit* without removing history
- 能够切换到特定 *commit*，且不会删除历史记录
- Ability to manage *commits* in various ways, including combining them
- 能够以多种方式管理 *commit*，包括合并 commit
- Various other features, see here
- 其他多种功能，参见此处

Prerequisites / 前置条件

IMPORTANT: This tutorial uses the Windows operating system.

重要： 本教程使用 Windows Operating System。

You have to download and install Git from the following links:

必须从以下链接下载并安装 Git：

- Windows
- Windows
- macOS
- macOS
- Linux

- Linux

NOTE: You may need to add Git to your path.

说明： 可能需要将 Git 添加到系统的 path 中。

Git Vocabulary / Git 术语

Git revolves around several core data structures and commands:

Git 围绕若干核心数据结构和命令展开：

- **Repository:** the data structure of your code, including a `.git` folder in the root directory
- **Repository:** 代码的数据结构，其 Root Directory 中包含 `.git` 文件夹
- **Commit:** a particular saved state of the repository, which includes all files and additions
- **Commit:** Repository 的某个已保存状态，其中包括所有文件和新增内容
- **Branch:** a means of grouping a set of commits. Each branch has a unique history. This is primarily used for separating development and stable branches.
- **Branch:** 对一组 commit 进行分组的方式。每个 branch 都有独立的历史记录，主要用于分离开发分支与稳定分支。
- **Push:** update the remote repository with your local changes
- **Push:** 使用本地变更更新 Remote Repository
- **Pull:** update your local repository with the remote changes
- **Pull:** 使用远程变更更新 Local Repository
- **Clone:** retrieve a local copy of a repository to modify
- **Clone:** 获取 Repository 的本地副本以进行修改
- **Fork:** duplicate a pre-existing repository to modify, and to compare against the original
- **Fork:** 复制已有 Repository，以便修改并与原 Repository 比较
- **Merge:** combine various changes from different branches/commits/forks into a single history
- **Merge:** 将来自不同 branch、commit 或 fork 的多项变更合并为一条历史记录

Repository / Repository

A Git repository is a data structure containing the structure, history, and files of a project.

Git Repository 是一种包含项目结构、历史记录和文件的数据结构。

Git repositories usually consist of:

Git Repository 通常包括：

- A `.git` folder. This folder contains the various information about the repository.
- `.git` 文件夹。该文件夹包含 Repository 的各类信息。
- A `.gitignore` file. This file contains the files or directories that you do *not* want included when you commit.
- `.gitignore` 文件。该文件列出你在 commit 时不希望包含的文件或目录。
- Files and folders. This is the main content of the repository.
- 文件和文件夹。它们是 Repository 的主要内容。

Creating the repository / 创建 Repository

You can store the repository locally, or through a remote -- a remote being the cloud, or possibly another storage medium or server that hosts your repository. GitHub is a popular free hosting service. Numerous developers use it, and that's what this tutorial will use.

可以将 Repository 存储在本地，也可以通过 Remote 存储；Remote 可以是 Cloud，也可以是承载 Repository 的其他存储介质或服务器。GitHub 是一种常用的免费托管服务，许多开发者都在使用，本教程也将使用它。

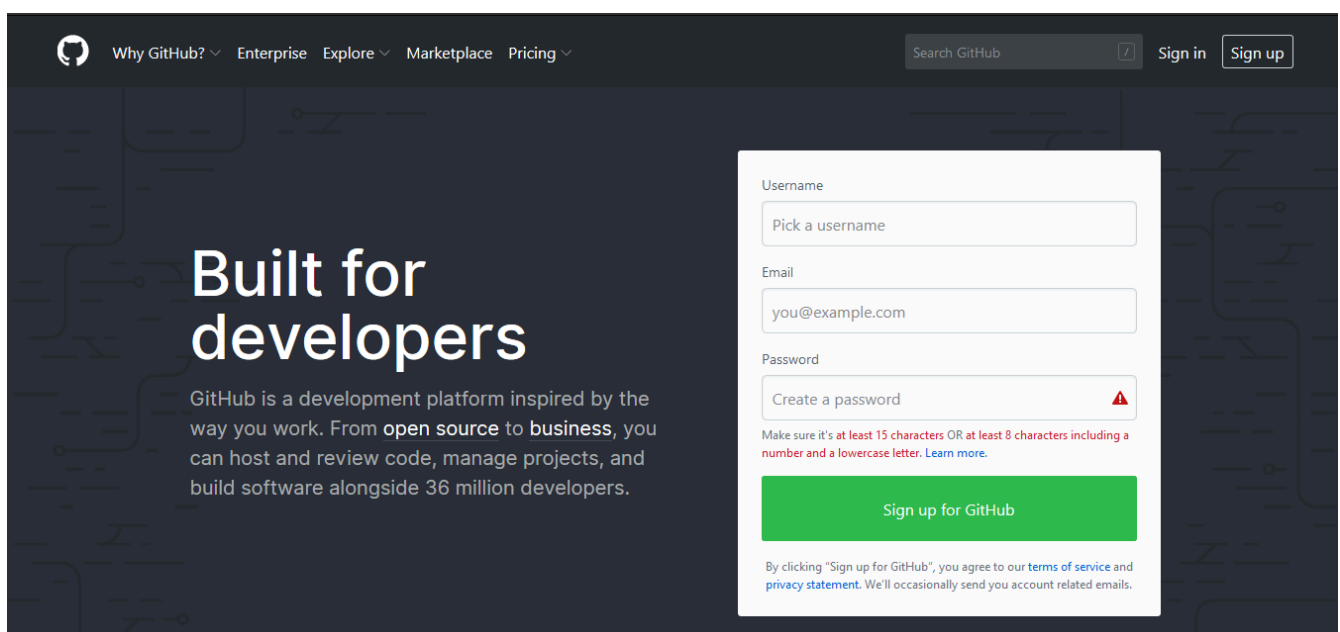
NOTE: There are various providers that can host repositories. Gitlab and Bitbucket are a few alternatives to Github.

说明： 有多种 Provider 可以托管 Repository。Gitlab 和 Bitbucket 是 Github 的部分替代方案。

Creating a GitHub Account / 创建 GitHub Account

Go ahead and create a GitHub account by visiting the website and following the on-screen prompts.

访问 GitHub 网站，按照屏幕提示创建 GitHub Account。



Why GitHub? ▾ Enterprise Explore ▾ Marketplace Pricing ▾ Search GitHub [7] Sign in Sign up

Built for developers

GitHub is a development platform inspired by the way you work. From **open source** to **business**, you can host and review code, manage projects, and build software alongside 36 million developers.

Username
Pick a username

Email
you@example.com

Password
Create a password 
Make sure it's **at least 15 characters** OR **at least 8 characters including a number and a lowercase letter**. [Learn more](#).

[Sign up for GitHub](#)

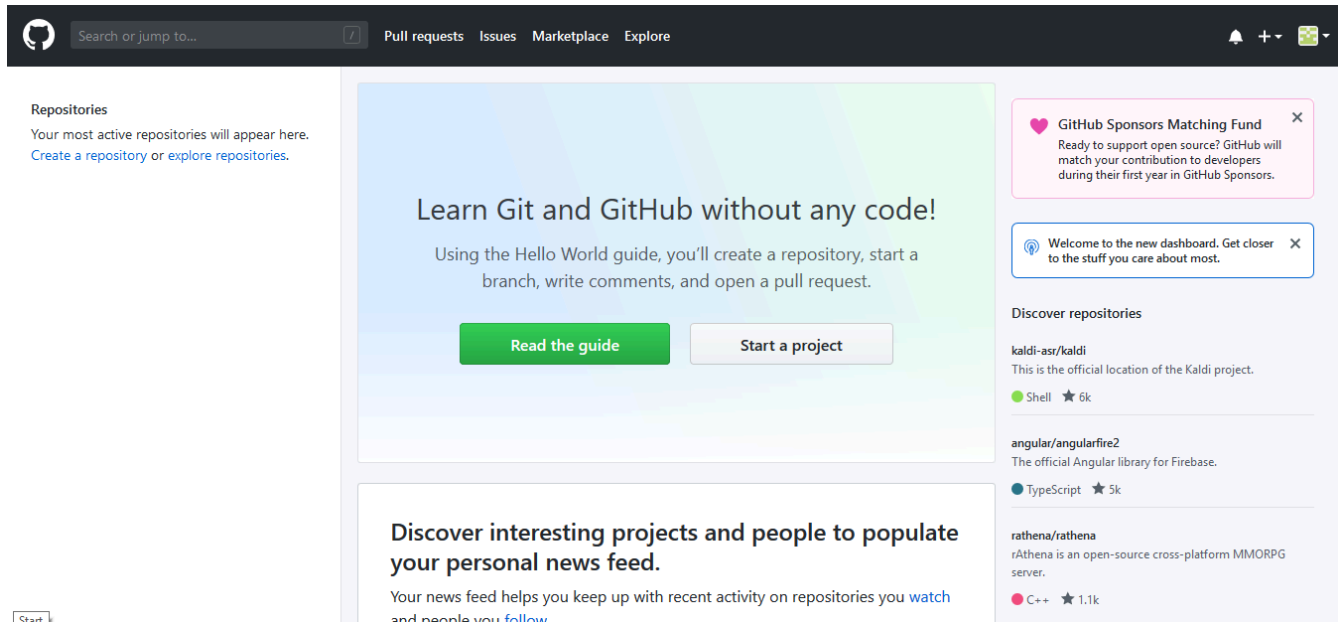
By clicking "Sign up for GitHub", you agree to our [terms of service](#) and [privacy statement](#). We'll occasionally send you account related emails.

How to create a new GitHub account. / 如何创建新的 GitHub Account。

Local Creation / 本地创建

After creating and verifying your account, you'll want to visit the homepage. It'll look similar to the shown image.

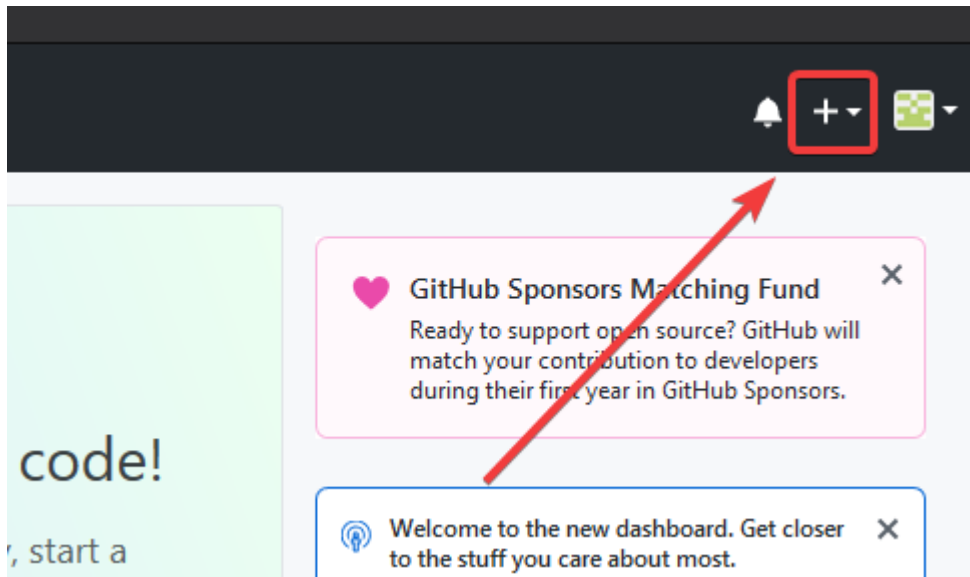
创建并验证 Account 后，访问 Homepage。它看起来应与图中类似。



Showing the newly created account homepage. / 显示新建 Account 的 Homepage。

Click the plus icon in the top right.

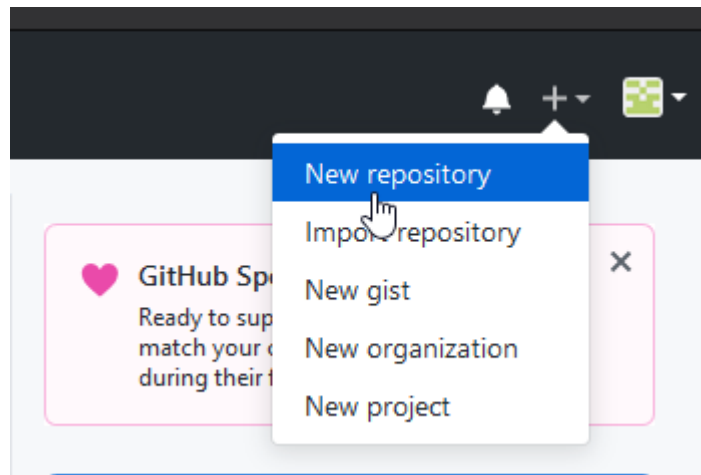
单击右上角的加号图标。



Location of the plus button. / 加号按钮的位置。

Then click "New Repository".

然后单击 "New Repository"。



Creating a new menu after clicking the plus button. / 单击加号按钮后打开新建菜单。

Fill out the appropriate information, and then click *"Create repository"*.

填写相应信息，然后单击 *"Create repository"*。

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner

 ExampleUser9007 ▼

Repository name *

ExampleRepo ✓

Great repository names are short and memorable. Need inspiration? How about **reimagined-palm-tree**?

Description (optional)



Public

Anyone can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: None ▼

Add a license: None ▼

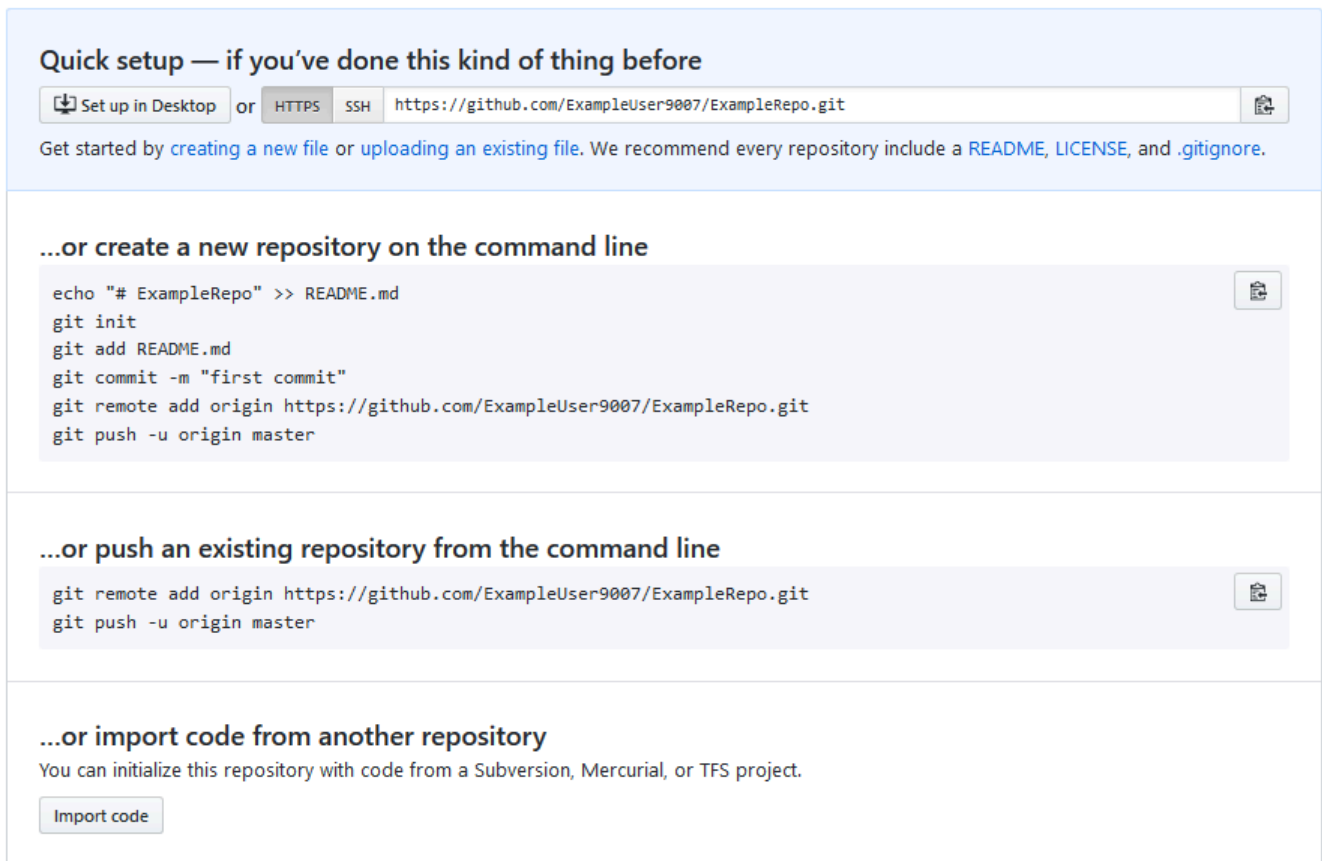


Create repository

Showing the "create repository" button. / 显示 "create repository" 按钮。

You should see a screen similar to this.

你应看到与此类似的界面。



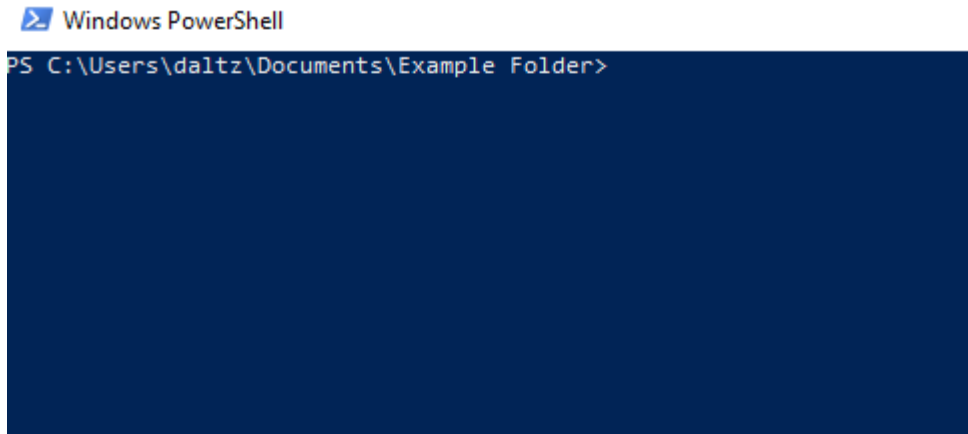
The quick setup screen after creating a repository. / 创建 Repository 后的 Quick Setup 界面。

NOTE: The keyboard shortcut `Ctrl+~` can be used to open a terminal in Visual Studio Code for Windows.

说明： 在 Windows 版 Visual Studio Code 中，可以使用快捷键 `Ctrl+~` 打开 Terminal。

Now you'll want to open a PowerShell window and navigate to your project directory. An excellent tutorial on PowerShell can be found [here](#). Please consult your search engine on how to open a terminal on alternative operating systems.

现在打开 PowerShell 窗口并转到 Project Directory。此处提供了一篇优秀的 PowerShell 教程。对于其他 Operating System，请使用 Search Engine 查询如何打开 Terminal。



An empty powershell window. / 空白 PowerShell 窗口。

If a directory is empty, a file needs to be created in order for git to have something to track. In the below Empty Directory example, we created a file called `README.md` with the contents of `# Example Repo`. For FRC® Robot projects, the below Existing Project commands should be run in the root of a project created by the VS Code WPILib Project Creator. More details on the various commands can be found in the subsequent sections.

如果 Directory 为空，则需要创建一个文件，以便 Git 有内容可跟踪。在下方 Empty Directory 示例中，创建了名为 `README.md` 的文件，其内容为 `# Example Repo`。对于 FRC® Robot Project，应在由 VS Code WPILib Project Creator 创建的 Project Root 中运行下方 Existing Project 命令。后续章节提供了各个命令的更多详细信息。

NOTE: Replace the filepath "`C:\Users\ExampleUser9007\Documents\Example Folder`" with the one you want to create the repo in, and replace the remote URL `https://github.com/ExampleUser9007/ExampleRepo.git` with the URL for the repo you created in the previous steps.

说明： 将 File Path "`C:\Users\ExampleUser9007\Documents\Example Folder`" 替换为要在其中创建 Repository 的路径，并将 Remote URL `https://github.com/ExampleUser9007/ExampleRepo.git` 替换为前面步骤中所创建 Repository 的 URL。

Empty Directory / 空目录

```
> cd "C:\Users\ExampleUser9007\Documents\Example Folder"
> git init
Initialized empty Git repository in C:/Users/ExampleUser9007/Documents/Example Folder/.git/
> echo "# ExampleRepo" >> README.md
> git add README.md
> git commit -m "First commit"
[main (root-commit) fafafa] First commit
 1 file changed, 1 insertions(+), 0 deletions(-)
 create mode 100644 README.md
> git remote add origin https://github.com/ExampleUser9007/ExampleRepo.git
> git push -u origin main
```

Existing Project / 现有项目

```
> cd "C:\Users\ExampleUser9007\Documents\Example Folder"
> git init
Initialized empty Git repository in C:/Users/ExampleUser9007/Documents/Example Folder/.git/
> git add .
> git commit -m "First commit"
[main (root-commit) fafafa] First commit
 1 file changed, 1 insertions(+), 0 deletions(-)
 create mode 100644 README.md
> git remote add origin https://github.com/ExampleUser9007/ExampleRepo.git
> git push -u origin main
```

Commits / Commit

Repositories are primarily composed of commits. Commits are saved states or *versions* of code.

Repository 主要由 commit 构成。Commit 是代码已保存的状态或版本。

In the previous example, we created a file called README.md. Open that file in your favorite text editor and edit a few lines. After tinkering with the file for a bit, simply save and close. Navigate to PowerShell and type the following commands.

在前面的示例中，创建了名为 README.md 的文件。使用常用的 Text Editor 打开该文件并编辑几行。修改完成后，保存并关闭文件。转到 PowerShell 并输入以下命令。

```
> git add README.md
> git commit -m "Adds a description to the repository"
[main bcbcbc] Adds a description to the repository
 1 file changed, 2 insertions(+), 0 deletions(-)
> git push
```

NOTE: Writing good commit messages is a key part of a maintainable project. A guide on writing commit messages can be found [here](#).

说明： 编写良好的 Commit Message 是保持项目可维护性的关键部分。此处提供了一份 Commit Message 编写指南。

Git Pull / Git Pull

NOTE: `git fetch` can be used when the user does not wish to automatically merge into the current working branch.

说明： 如果不希望自动 Merge 到当前 Working Branch，可以使用 `git fetch`。

This command retrieves the history or commits from the remote repository. When the remote contains work you do not have, it will attempt to automatically merge. See Merging.

该命令从 Remote Repository 获取历史记录或 commit。当 Remote 中包含本地没有的工作时，它会尝试自动 Merge。请参阅“Merging / 合并”。

Run: `git pull`

运行: `git pull`

Git Add / Git Add

This command "stages" the specified file(s) so that they will be included in the next commit.

该命令会将指定文件加入暂存区（stage），使其包含在下一次 commit 中。

For a single file, run `git add FILENAME.txt` where `FILENAME.txt` is the name and extension of the file to add. To add every file/folder that isn't excluded via *gitignore*, run `git add .`. When run in the root of the repository this command will stage every untracked, unexcluded file.

对于单个文件，运行 `git add FILENAME.txt`，其中 `FILENAME.txt` 是要添加文件的名称和 Extension。要添加所有未被 *gitignore* 排除的文件或文件夹，请运行 `git add .`。在 Repository Root 中运行时，该命令会暂存所有未跟踪且未排除的文件。

Git Commit / Git Commit

This command creates the commit and stores it locally. This saves the state and adds it to the repository's history. The commit will consist of whatever changes ("diffs") were made to the staged files since the last commit. It is required to specify a "commit message" explaining why you changed this set of files or what the change accomplishes.

该命令创建 commit 并将其存储在本地。它会保存当前状态，并将其添加到 Repository 的历史记录中。

Commit 包含自上次 commit 以来对已暂存文件所做的所有变更（“diff”）。必须指定“Commit Message”，说明为何更改这组文件，或本次更改实现了什么。

Run: `git commit -m "type message here"`

运行: `git commit -m "type message here"`

Git Push / Git Push

Upload (Push) your local changes to the remote (Cloud).

将本地变更上传（Push）到 Remote（Cloud）。

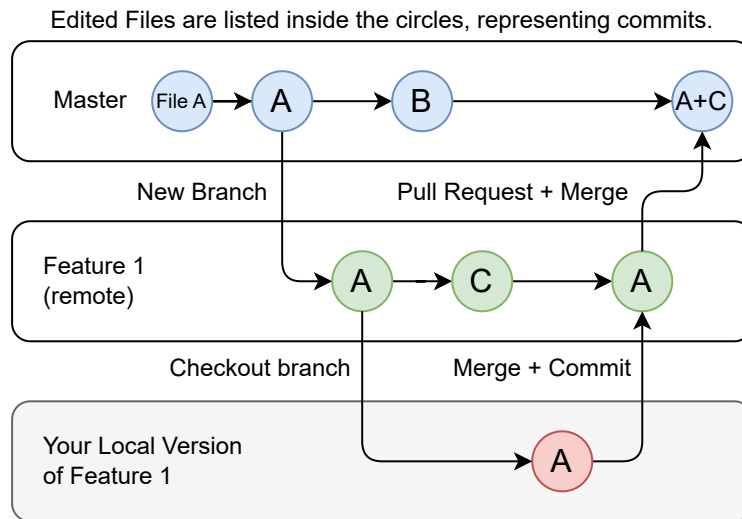
Run: `git push`

运行: `git push`

Branches / Branch

Branches in Git are similar to parallel worlds. They start off the same, and then they can "branch" out into different varying paths. Consider the Git control flow to look similar to this.

Git 中的 branch 类似平行世界。它们起初相同，之后可以“分支”到不同路径。可以将 Git Control Flow 看作类似下图。



A branch workflow state diagram. / Branch 工作流状态图。

In the above example, main was branched (or duplicated) into the branch Feature 1 and someone checked out the branch, creating a local copy. Then, someone committed (or uploaded) their changes, merging them into the branch Feature 1. You are "merging" the changes from one branch into another.

在上方示例中，main 被分支（或复制）为 Feature 1 branch，有人 checkout 该 branch，从而创建本地副本。随后，有人 commit（或上传）其变更，并将其合并到 Feature 1 branch。这里是在将一个 branch 的变更“Merge”到另一个 branch。

Creating a Branch / 创建 Branch

Run: `git branch branch-name` where branch-name is the name of the branch to create. The new branch history will be created from the current active branch.

运行 `git branch branch-name`，其中 branch-name 是要创建的 branch 名称。新 branch 的历史记录将从当前 Active Branch 创建。

Entering a Branch / 进入 Branch

Once a branch is created, you have to then enter the branch.

创建 branch 后，需要进入该 branch。

Run: `git checkout branch-name` where branch-name is the branch that was previously created.

运行 `git checkout branch-name`，其中 branch-name 是之前创建的 branch。

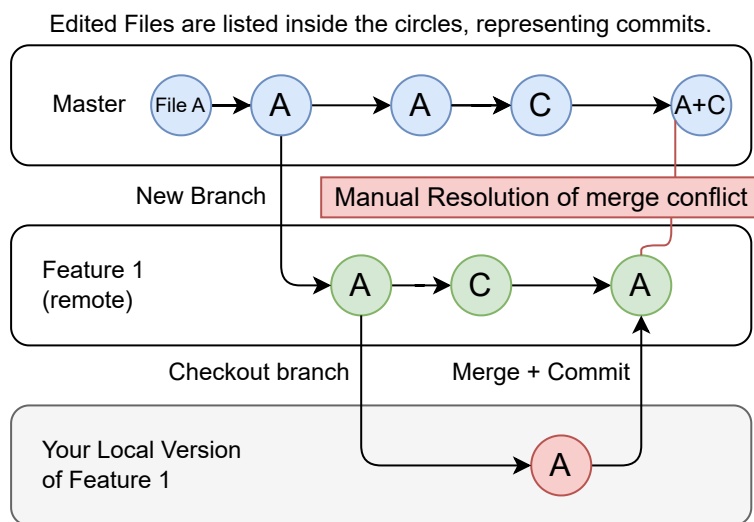
Merging / 合并

In scenarios where you want to copy one branches history into another, you can merge them. A merge is done by calling `git merge branch-name` with `branch-name` being the name of the branch to merge from. It is automatically merged into the current active branch.

如果希望将一个 branch 的历史记录复制到另一个 branch，可以将它们 Merge。通过调用 `git merge branch-name` 执行 Merge，其中 `branch-name` 是作为来源的 branch 名称。它会自动 Merge 到当前 Active Branch。

It's common for a remote repository to contain work (history) that you do not have. Whenever you run `git pull`, it will attempt to automatically merge those commits into your local copy. That merge may look like the below.

Remote Repository 中经常会包含本地没有的工作（历史记录）。每当运行 `git pull` 时，它都会尝试将这些 commit 自动 Merge 到本地副本中。该 Merge 可能如下图所示。



A merge workflow state diagram. / Merge workflow状态图。

However, in the above example, what if File A was modified by both branch Feature1 and Feature2? This is called a **merge conflict**. A merge conflict can be resolved by editing the conflicting file. In the example, we would need to edit File A to keep the history or changes that we want. After that has been done, simply re-add, re-commit, and then push your changes.

但是在上方示例中，如果 File A 同时被 Feature1 和 Feature2 branch 修改，会发生什么？这称为 **Merge Conflict**。可以通过编辑发生冲突的文件来解决 Merge Conflict。在该示例中，需要编辑 File A，以保留所需的历史记录或变更。完成后，只需重新 add、重新 commit，然后 push 变更。

Resets / Reset

Sometimes history needs to be reset, or a commit needs to be undone. This can be done multiple ways.

有时需要 Reset 历史记录，或撤销某个 commit。可以通过多种方式完成。

Reverting the Commit / Revert Commit

NOTE: You cannot revert a merge, as git does not know which branch or origin it should choose.

说明：无法直接 Revert Merge，因为 Git 不知道应选择哪个 branch 或 origin。

To revert history leading up to a commit run `git revert commit-id`. The commit IDs can be shown using the `git log` command.

要 Revert 直至某个 commit 的历史记录，请运行 `git revert commit-id`。可以使用 `git log` 命令显示 Commit ID。

Resetting the Head / Reset Head

WARNING: Forcibly resetting the head is a dangerous command. It permanently erases all history past the target. You have been warned!

警告：强制 Reset Head 是危险命令。它会永久删除目标之后的所有历史记录。请务必谨慎！

Run: `git reset --hard commit-id`.

运行: `git reset --hard commit-id`。

Forks / Fork

Forks can be treated similarly to branches. You can merge the upstream (original repository) into the origin (forked repository).

可以用类似 branch 的方式看待 fork。可以将 upstream (Original Repository) Merge 到 origin (Forked Repository)。

Cloning an Existing Repo / Clone 现有 Repository

In the situation that a repository is already created and stored on a remote, you can clone it using

如果 Repository 已创建并存储在 Remote 上，可以使用以下命令 Clone：

```
git clone https://github.com/myrepo.git
```

where `myrepo.git` is replaced with your git repo. If you follow this, you can skip to Commits.

将 `myrepo.git` 替换为你的 Git Repository。如果采用此方式，可以直接跳到 “Commits / Commit”。

Updating a Fork / 更新 Fork

1. Add the upstream: `git remote add upstream`
`https://github.com/ORIGINAL_OWNER/ORIGINAL_REPOSITORY.git`
2. Confirm it was added via: `git remote -v`
3. Pull changes from upstream: `git fetch upstream`
4. Merge the changes into head: `git merge upstream/upstream-branch-name`
5. 添加 upstream: `git remote add upstream`
`https://github.com/ORIGINAL_OWNER/ORIGINAL_REPOSITORY.git`
6. 使用以下命令确认已添加: `git remote -v`
7. 从 upstream 获取变更: `git fetch upstream`
8. 将变更 Merge 到 Head: `git merge upstream/upstream-branch-name`

Gitignore / Gitignore

IMPORTANT: It is extremely important that teams **do not** modify the `.gitignore` file that is included with their robot project. This can lead to offline deployment not working.

重要： 队伍**不得**修改 Robot Project 附带的 `.gitignore` 文件，这一点极其重要。修改该文件可能导致 Offline Deployment 无法工作。

A `.gitignore` file is commonly used as a list of files to not automatically commit with `git add`. Any files or directory listed in this file will **not** be committed. They will also not show up with `git status`.

`.gitignore` 文件通常用于列出不应通过 `git add` 自动加入 commit 的文件。此文件中列出的任何文件或目录都**不会被** commit，也不会出现在 `git status` 中。

Additional Information can be found [here](#).

此处提供了更多信息。

Hiding a Folder / 隐藏文件夹

Simply add a new line containing the folder to hide, with a forward slash at the end.

只需新增一行，写入要隐藏的文件夹，并在末尾添加正斜杠。

EX: `directory-to-exclude/`

示例: `directory-to-exclude/`

Hiding a File / 隐藏文件

Add a new line with the name of the file to hide, including any prepending directory relative to the root of the repository.

新增一行，写入要隐藏的文件名，并包含相对于 Repository Root 的所有前置目录。

EX: `directory/file-to-hide.txt`

示例: `directory/file-to-hide.txt`

EX: `file-to-hide2.txt`

示例: `file-to-hide2.txt`

Additional Information / 其他信息

A much more in-depth tutorial can be found at the official git website.

可以在 Git 官方网站找到更深入的教程。

A guide for correcting common mistakes can be found at the git flight rules repository.

可以在 Git flight rules Repository 中找到常见错误的纠正指南。

The Java Units Library / Java Units Library

NOTE: New for 2025: The units library has been refactored to have unit-specific measurement classes instead of a single generic `Measure` class. The new measurement classes have clearer names (`Distance` instead of `Measure<Distance>`, Or `LinearAcceleration` instead of `Measure<Velocity<Velocity<Distance>>>`), and implement math operations to return the most specific result types possible instead of a wildcard `Measure<?>`.

说明： 2025 年新增：Units Library 已进行重构，改为使用各 Unit 专属的 Measurement Class，而不是单一的 Generic Measure Class。新的 Measurement Class 名称更加清晰（例如使用 `Distance` 而不是 `Measure<Distance>`，使用 `LinearAcceleration` 而不是 `Measure<Velocity<Velocity<Distance>>>`），其 Mathematical Operation 会返回尽可能具体的 Result Type，而不是 Wildcard `Measure<?>`。

The units library is a tool that helps programmers avoid mistakes related to units of measurement. It does this by keeping track of the units of measurement, and by ensuring that all operations are performed with the correct units. This can help to prevent errors that can lead to incorrect results, such as adding a distance in inches to a distance in meters.

Units Library 是一种帮助 Programmer 避免 Measurement Unit 相关错误的工具。它会跟踪 Measurement Unit，并确保所有 Operation 都使用正确的 Unit 执行。这有助于防止产生错误结果，例如将以 inch 表示的 Distance 与以 meter 表示的 Distance 相加。

An added benefit is readability and maintainability, which also reduces bugs. By making the units of measurement explicit in your code, it becomes easier to read and understand what your code is doing. This can also help to make your code more maintainable, as it is easier to identify and fix errors related to units of measurement.

它还能提高 Readability 和 Maintainability，从而减少 Bug。在代码中显式标明 Measurement Unit，可以更容易地阅读和理解代码正在执行的操作。这也能提升代码的 Maintainability，因为更容易识别并修复与 Measurement Unit 有关的错误。

The units library has a number of features:

Units Library 具有以下功能：

- A set of predefined units, such as meters, degrees, and seconds.
- 一组预定义 Unit，例如 meter、degree 和 second。
- The ability to convert between different units.
- 能够在不同 Unit 之间进行转换。
- Support for performing arithmetic and comparisons on quantities with units.
- 支持对带 Unit 的 Quantity 执行 Arithmetic Operation 和 Comparison。
- Support for displaying quantities with units in a human-readable format.

- 支持以 Human-readable Format 显示带 Unit 的 Quantity。

Terminology / 术语

Dimension

Dimension (量纲)

Dimensions represent the nature of a physical quantity, such as length, time, or mass. They are independent of any specific unit system. For example, the dimension of meters is length, regardless of whether the length is expressed in meters, millimeters, or inches.

Dimension 表示 Physical Quantity 的性质，例如 Length、Time 或 Mass。它不依赖任何特定 Unit System。例如，meter 的 Dimension 是 Length；无论 Length 使用 meter、millimeter 还是 inch 表示，其 Dimension 都相同。

Unit

Unit (单位)

Units are specific realizations of dimensions. They are the way of expressing physical quantities. Each dimension has a base unit, such as the meter for length, the second for time, the kilogram for mass. Derived units are formed by combining base units, such as meters per second for velocity.

Unit 是 Dimension 的具体实现，也是表达 Physical Quantity 的方式。每个 Dimension 都有 Base Unit，例如 Length 使用 meter，Time 使用 second，Mass 使用 kilogram。Derived Unit 由 Base Unit 组合而成，例如 Velocity 使用 meter per second。

Measure

Measure (测量值)

Measures are the specific magnitude of physical quantities, expressed in a particular unit. For example, 5 meters is a measure of distance.

Measure 是以特定 Unit 表示的 Physical Quantity 的具体 Magnitude。例如，5 meters 是一个 Distance Measure。

These concepts are used within the Units Library. For example, the **measure** *10 seconds* has a magnitude of 10, the **dimension** is time, and the **unit** is seconds.

Units Library 会使用这些概念。例如，**Measure** *10 seconds* 的 Magnitude 为 10，**Dimension** 为 Time，**Unit** 为 second。

Using the Units Library / 使用 Units Library

The Java units library is available in the `edu.wpi.first.units` package. The most relevant classes are:

Java Units Library 位于 `edu.wpi.first.units` Package 中。最相关的 Class 包括：

- The various classes for predefined dimensions, such as `DistanceUnit` and `TimeUnit`
- 用于预定义 Dimension 的各种 Class，例如 `DistanceUnit` 和 `TimeUnit`
- `Units`, which contains a set of predefined units. Take a look at the `Units` javadoc to browse the available units and their types.
- `Units`，其中包含一组预定义 Unit。可以查看 `Units Javadoc`，浏览可用 Unit 及其 Type。
- `Measure`, which is used to tag a value with a unit, and the dimension-specific implementations like `Distance` and `Time`
- `Measure`，用于为 Value 标记 Unit；以及 `Distance` 和 `Time` 等特定 Dimension 的 Implementation

NOTE: It is recommended to static import `edu.wpi.first.units.Units.*` to get full access to all the predefined units.

说明： 建议 Static Import `edu.wpi.first.units.Units.*`，以完整访问所有预定义 Unit。

Creating Measures / 创建 Measure

Every dimension has a measurement class with the corresponding name - for example, a `Distance` measures distance, `Time` measures time, and `LinearVelocity` measures linear velocity. To instantiate one of these measurements, call the `Unit.of` method on the appropriate unit object. For example, to create a `Distance` object representing a distance of 6 inches, you would write:

每个 Dimension 都有名称对应的 Measurement Class——例如，`Distance` 测量 Distance，`Time` 测量 Time，`LinearVelocity` 测量 Linear Velocity。要实例化这些 Measurement 之一，请在相应 Unit Object 上调用 `Unit.of` Method。例如，要创建表示 6 inches Distance 的 `Distance` Object，可以写为：

```
Distance wheelDiameter = Inches.of(6);
```

Other measures can also be created using their `Unit.of` method:

其他 Measure 也可以使用各自的 `Unit.of` Method 创建：

```
Mass kArmMass = Kilograms.of(1.423);
Distance kArmLength = Inches.of(32.25);
Angle kMinArmAngle = Degrees.of(5);
Angle kArmMaxTravel = Rotations.of(0.45);
LinearVelocity kMaxSpeed = MetersPerSecond.of(2.5);
```

WARNING: Composite units with `PerUnit` and `MultUnit` have special requirements, and the `of` method is not recommended to be used with them.

警告： 使用 `PerUnit` 和 `MultUnit` 的 Composite Unit 有特殊要求，不建议对它们使用 `of` Method。

Using Composite Unit Types / 使用 Composite Unit Type

Due to requirements of inheritance in Java's type system, `PerUnit` and `MultUnit` cannot return a normal `Per` or `Mult` object from their `of` factory methods. Instead, they need to return a bounded wildcard `Measure<? extends PerUnit<...>>` or `Measure<? extends MultUnit<...>>` to allow subclasses like `LinearVelocity` to return a compatible type. New `ofNative` methods are provided to be able to work with known `Per` and `Mult` objects.

由于 Java Type System 的 Inheritance 要求, `PerUnit` 和 `MultUnit` 无法从其 `of` Factory Method 返回普通的 `Per` 或 `Mult` Object。它们必须返回 Bounded Wildcard `Measure<? extends PerUnit<...>>` 或 `Measure<? extends MultUnit<...>>`, 以允许 `LinearVelocity` 等 Subclass 返回 Compatible Type。新增的 `ofNative` Method 可用于处理已知的 `Per` 和 `Mult` Object。

```
// Using ofNative:
Per<VoltageUnit, DistanceUnit> kP = VoltsPerMeter.ofNative(1);
kP.in(VoltsPerMeter); // 1.0

Measure<VoltageUnit> output = kP.timesDivisor(Meters.of(1));
output.in(Volts); // 1.0

// Without ofNative
Measure<? extends PerUnit<VoltageUnit, DistanceUnit>> kP = VoltsPerMeter.of(1);
kP.in(VoltsPerMeter); // Compilation error!

Measure<?> output = kP.times(Meters.of(1)); // The compiler can't know what unit this is
output.in(Volts); // Compilation error!
```

Performing Calculations / 执行计算

The `Measure` class also supports arithmetic operations, such as addition, subtraction, multiplication, and division. These are done by calling methods on the objects. These operations always ensure that the units are compatible before performing the calculation, and they return a new `Measure` object. For example, you can add two `Distance` objects together, even if they have different units:

`Measure` Class 还支持 Addition、Subtraction、Multiplication 和 Division 等 Arithmetic Operation。这些 Operation 通过调用 Object 上的 Method 执行。它们始终会在计算前确保 Unit Compatible, 并返回新的 `Measure` Object。例如, 即使两个 `Distance` Object 使用不同 Unit, 也可以将它们相加:

```
Distance distance1 = Inches.of(10);
Distance distance2 = Meters.of(0.254);
Distance totalDistance = distance1.plus(distance2);
```

In this code, the units library will automatically convert the measures to the same unit before adding the two distances. The resulting `totalDistance` object will be a new `Distance` object that has a value of 0.508 meters, or 20 inches.

在此代码中, Units Library 会先自动将 `Measure` 转换为相同 Unit, 然后再将两个 `Distance` 相加。所得 `totalDistance` Object 是新的 `Distance` Object, 其 Value 为 0.508 meters, 即 20 inches。

NOTE: Mathematical operations are type safe. It is impossible to add a distance to a time, or subtract an angle from a voltage. However, multiplication and division operations make a best-effort attempt to return results in the most appropriate unit type; dividing a distance by time results in a `LinearVelocity` measurement, and multiplying it by time returns a `Distance`.

说明： Mathematical Operation 具有 Type Safety。无法将 Distance 与 Time 相加，也无法从 Voltage 中减去 Angle。不过，Multiplication 和 Division Operation 会尽可能返回最合适的 Unit Type；Distance 除以 Time 会得到 LinearVelocity Measurement，再将其乘以 Time 会返回 Distance。

This example combines the wheel diameter and gear ratio to calculate the distance per rotation of the wheel:

此示例结合 Wheel Diameter 和 Gear Ratio，计算 Wheel 每转一圈的 Distance：

```
Distance wheelDiameter = Inches.of(3);
double gearRatio = 10.48;
Distance distancePerRotation = wheelDiameter.times(Math.PI).divide(gearRatio);
```

WARNING: By default, arithmetic operations create new `Measure` instances for their results. See [Java Garbage Collection](#) for discussion on creating a large number of short-lived objects. See also, the [Mutability and Object Creation](#) section below for a possible workaround.

警告： 默认情况下，Arithmetic Operation 会为结果创建新的 `Measure` Instance。有关创建大量 Short-lived Object 的讨论，请参阅 [Java Garbage Collection / Java 垃圾回收](#)。另请参阅下方“Mutability and Object Creation / 可变性与 Object 创建”一节，了解可能的替代方案。

Converting Units / 转换 Unit

Unit conversions can be done by calling `Measure.in(Unit)`. The Java type system will prevent units from being converted between incompatible types, such as distances to angles. The returned values will be bare `double` values without unit information - it is up to you, the programmer, to interpret them correctly! It is strongly recommended to only use unit conversions when interacting with APIs that do not support the units library.

可以通过调用 `Measure.in(Unit)` 执行 Unit Conversion。Java Type System 会阻止在 Incompatible Type 之间转换 Unit，例如将 `Distance` 转换为 `Angle`。返回值是没有 Unit Information 的普通 `double` Value——需要由 Programmer 正确解释。强烈建议只在与不支持 Units Library 的 API 交互时使用 Unit Conversion。

```

LinearVelocity kMaxVelocity = FeetPerSecond.of(12.5);
LinearAcceleration kMaxAcceleration = FeetPerSecond.per(Second).of(22.9);
kMaxVelocity.in(MetersPerSecond); // => OK! Returns 3.81
kMaxVelocity.in(RadiansPerSecond); // => Compile error! LinearVelocity cannot be converted to
AngularVelocity

// The WPILib math libraries use SI metric units, so we have to convert to meters:
TrapezoidProfile.Constraints kDriveConstraints = new TrapezoidProfile.Constraints(
    maxVelocity.in(MetersPerSecond),
    maxAcceleration.in(MetersPerSecondPerSecond)
);

```

Usage Example / 使用示例

Pulling all of the concepts together, we can create an example that calculates the end effector position of an arm mechanism:

综合以上所有概念，可以创建一个计算 Arm Mechanism End Effector Position 的示例：

```

Distance armLength = Feet.of(3).plus(Inches.of(4.25));
Distance endEffectorX = armLength.times(Math.cos(getArmAngle().in(Radians)));
Distance endEffectorY = armLength.times(Math.sin(getArmAngle().in(Radians)));

```

Human-readable Formatting / 人类可读的格式化

The `Measure` class has methods that can be used to get a human-readable representation of the measure. This feature is useful to display a measure on a dashboard or in logs.

`Measure` Class 提供了用于获取 Measure Human-readable Representation 的 Method。该功能适合在 Dashboard 或 Log 中显示 Measure。

- `toString()` and `toShortString()` return a string representation of the measure in a shorthand form. The symbol of the backing unit is used, rather than the full name, and the magnitude is represented in scientific notation. For example, 1.234e+04 V/m
- `toString()` 和 `toShortString()` 返回 Measure 的 Shorthand String Representation。它使用底层 Unit 的 Symbol 而不是 Full Name，并以 Scientific Notation 表示 Magnitude。例如：1.234e+04 V/m
- `toLongString()` returns a string representation of the measure in a longhand form. The name of the backing unit is used, rather than its symbol, and the magnitude is represented in a full string, not scientific notation. For example, 1234 Volt per Meter
- `toLongString()` 返回 Measure 的 Longhand String Representation。它使用底层 Unit 的 Name 而不是 Symbol，并以完整 String 而不是 Scientific Notation 表示 Magnitude。例如：1234 Volt per Meter

Mutability and Object Creation / 可变性与 Object 创建

To reduce the number of object instances you create, and reduce memory usage, a special `MutableMeasure` class is available, with unit-specific subtypes like `MutDistance` and `MutTime`. You may want to consider using mutable objects if you are using the units library repeatedly, such as in the robot's periodic loop. See Java Garbage Collection for more discussion on creating a large number of short-lived objects.

Mutable measures can be created in a similar way to regular, immutable measures using the `Unit.mutable` method (instead of `Unit.of`).

为了减少所创建 Object Instance 的数量并降低 Memory Usage，可以使用特殊的 `MutableMeasure` Class，以及 `MutDistance` 和 `MutTime` 等 Unit-specific Subtype。如果会重复使用 Units Library，例如在 Robot Periodic Loop 中使用，可以考虑使用 Mutable Object。有关创建大量 Short-lived Object 的更多讨论，请参阅 Java Garbage Collection / Java 垃圾回收。Mutable Measure 的创建方式与普通 Immutable Measure 类似，但使用 `Unit.mutable` Method，而不是 `Unit.of`。

`MutableMeasure` allows the internal state of the object to be updated, such as with the results of arithmetic operations, to avoid allocating new objects. Special care needs to be taken when mutating a measure because it will change the value every place that instance is referenced. If the object will be exposed as part of a public method, have that method return a regular `Measure` in its signature to prevent the caller from modifying your internal state.

`MutableMeasure` 允许更新 Object Internal State，例如使用 Arithmetic Operation 的结果更新，从而避免分配新 Object。修改 Measure 时必须格外小心，因为这会更改引用该 Instance 的所有位置中的 Value。如果 Object 将作为 Public Method 的一部分暴露，请让该 Method 在 Signature 中返回普通 `Measure`，以防止 Caller 修改 Internal State。

Extra methods are available on `MutableMeasure` for updating the internal value. Note that these methods all begin with the `mut_` prefix - this is to make it obvious that these methods will be mutating the object and are potentially unsafe! For the full list of methods and API documentation, see the `MutableMeasure` API documentation.

`MutableMeasure` 提供了用于更新 Internal Value 的额外 Method。请注意，这些 Method 都以 `mut_` Prefix 开头——这是为了明确表明它们会修改 Object，并且可能不安全。有关完整 Method List 和 API Documentation，请参阅 `MutableMeasure` API Documentation。

Method / 方法	Description / 说明
<code>mut_plus(double, Unit)</code>	Increments the internal value by an amount in another unit. The internal unit will stay the same
<code>mut_plus(double, Unit)</code>	将 Internal Value 增加以另一 Unit 表示的 Amount。Internal Unit 保持不变
<code>mut_plus(Measure)</code>	Increments the internal value by another measurement. The internal unit will stay the same
<code>mut_plus(Measure)</code>	将 Internal Value 增加另一个 Measurement。Internal Unit 保持不变
<code>mut_minus(double, Unit)</code>	Decrements the internal value by an amount in another unit. The internal unit will stay the same
<code>mut_minus(double, Unit)</code>	将 Internal Value 减少以另一 Unit 表示的 Amount。Internal Unit 保持不变
<code>mut_minus(Measure)</code>	Decrements the internal value by another measurement. The internal unit will stay the same
<code>mut_minus(Measure)</code>	将 Internal Value 减少另一个 Measurement。Internal Unit 保持不变

Method / 方法	Description / 说明
<code>mut_times(double)</code>	Multiplies the internal value by a scalar
<code>mut_times(double)</code>	将 Internal Value 乘以 Scalar
<code>mut_divide(double)</code>	Divides the internal value by a scalar
<code>mut_divide(double)</code>	将 Internal Value 除以 Scalar
<code>mut_replace(double, Unit)</code>	Overrides the internal state and sets it to equal the given value and unit
<code>mut_replace(double, Unit)</code>	覆盖 Internal State, 并将其设置为等于给定 Value 和 Unit
<code>mut_replace(Measure)</code>	Overrides the internal state to make it identical to the given measurement
<code>mut_replace(Measure)</code>	覆盖 Internal State, 使其与给定 Measurement 相同
<code>mut_setMagnitude(double)</code>	Overrides the internal value, keeping the internal unit. Be careful when using this!
<code>mut_setMagnitude(double)</code>	覆盖 Internal Value, 同时保留 Internal Unit。使用时请小心!

```

MutDistance measure = Feet.mutable(0);
measure.mut_plus(10, Inches);    // 0.8333 feet
measure.mut_plus(Inches.of(10)); // 1.6667 feet
measure.mut_minus(5, Inches);    // 1.25 feet
measure.mut_minus(Inches.of(5)); // 0.8333 feet
measure.mut_times(6);            // 0.8333 * 6 = 5 feet
measure.mut_divide(5);           // 5 / 5 = 1 foot
measure.mut_replace(6.2, Meters) // 6.2 meters - note the unit changed!
measure.mut_replace(Millimeters.of(14.2)) // 14.2mm - the unit changed again!
measure.mut_setMagnitude(72)     // 72mm

```

Revisiting the arm example from above, we can use `mut_replace` - and, optionally, `mut_times` - to calculate the end effector position.

重新查看上方 Arm 示例, 可以使用 `mut_replace`, 并可选择使用 `mut_times`, 来计算 End Effector Position。

```

import edu.wpi.first.units.Measure;
import edu.wpi.first.units.MutableMeasure;
import static edu.wpi.first.units.Units.*;

public class Arm {
    // Note the two ephemeral object allocations for the Feet.of and Inches.of calls.
    // Because this is a constant value computed just once, they will easily be garbage collected without
    // any problems with memory use or loop timing jitter.
    private static final Distance kArmLength = Feet.of(3).plus(Inches.of(4.25));

    // Angle and X/Y locations will likely be called in the main robot loop, let's store them in a
    MutableMeasure
    // to avoid allocating lots of short-lived objects
    private final MutAngle m_angle = Degrees.mutable(0);
    private final MutDistance m_endEffectorX = Feet.mutable(0);
    private final MutDistance m_endEffectorY = Feet.mutable(0);
    private final Encoder m_encoder = new Encoder(...);

    public Distance getEndEffectorX() {
        return m_endEffectorX.mut_replace(
            Math.cos(getAngle().in(Radians)) * kArmLength.in(Feet), // the new magnitude to store
            Feet // the units of the new magnitude
        );
    }

    public Distance getEndEffectorY() {
        // An alternative approach so we don't have to unpack and repack the units
        m_endEffectorY.mut_replace(kArmLength);
        m_endEffectorY.mut_times(Math.sin(getAngle().in(Radians)));
        return m_endEffectorY;
    }

    public Angle getAngle() {
        double rawAngle = m_encoder.getPosition();
        m_angle.mut_replace(rawAngle, Degrees); // NOTE: the encoder must be configured with distancePerPulse
        in terms of degrees!
        return m_angle;
    }
}

```

WARNING: `MutableMeasure` objects can - by definition - change their values at any time! It is unsafe to keep a stateful reference to them - prefer to extract a value using the `Measure.in` method, or create a copy with `Measure.copy` that can be safely stored. For the same reason, library authors must also be careful about methods accepting `Measure`.

警告： 根据定义，`MutableMeasure` Object 的 Value 可以随时发生变化。保留指向它们的 Stateful Reference 并不安全；应优先使用 `Measure.in` Method 提取 Value，或使用 `Measure.copy` 创建可安全存储的副本。出于相同原因，Library Author 也必须谨慎处理接收 `Measure` 的 Method。

Can you spot the bug in this code?

你能找出这段代码中的 Bug 吗？

```
private Distance m_lastDistance;
public Distance calculateDelta(Distance currentDistance) {
    if (m_lastDistance == null) {
        m_lastDistance = currentDistance;
        return currentDistance;
    } else {
        Distance delta = currentDistance.minus(m_lastDistance);
        m_lastDistance = currentDistance;
        return delta;
    }
}
```

If we run the `calculateDelta` method a few times, we can see a pattern:

如果多次运行 `calculateDelta` Method，可以看到一种 Pattern：

```
MutDistance distance = Inches.mutable(0);
distance.mut_plus(10, Inches);
calculateDelta(distance); // expect 10 inches and get 10 - good!
distance.mut_plus(2, Inches);
calculateDelta(distance); // expect 2 inches, but get 0 instead!
distance.mut_plus(8, Inches);
calculateDelta(distance); // expect 8 inches, but get 0 instead!
```

This is because the `m_lastDistance` field is a reference to the *same* `MutDistance` object as the input! Effectively, the delta is calculated as $(\text{currentDistance} - \text{currentDistance})$ on every call after the first, which naturally always returns zero. One solution would be to track `m_lastDistance` as a *copy* of the input measure to take a snapshot; however, this approach does incur one extra object allocation for the copy. If you need to be careful about object allocations, `m_lastDistance` could also be stored as a `MutDistance`.

这是因为 `m_lastDistance` Field 与 Input 引用了同一个 `MutDistance` Object。实际上，在第一次调用后的每次调用中，Delta 都按 $(\text{currentDistance} - \text{currentDistance})$ 计算，因此结果自然始终为零。一种解决方法是将 `m_lastDistance` 作为 Input Measure 的副本进行跟踪，从而拍摄 Snapshot；不过，这种方法确实会为副本增加一次 Object Allocation。如果需要谨慎控制 Object Allocation，也可以将 `m_lastDistance` 存储为 `MutDistance`。

Immutable Copies / Immutable 副本

```
private Distance m_lastDistance;
public Distance calculateDelta(Distance currentDistance) {
    if (m_lastDistance == null) {
        m_lastDistance = currentDistance.copy();
        return currentDistance;
    } else {
        var delta = currentDistance.minus(m_lastDistance);
        m_lastDistance = currentDistance.copy();
        return delta;
    }
}
```

Zero-allocation Mutables / 零分配 Mutable

```
private final MutDistance m_lastDistance = Meters.mutable(0);
private final MutDistance m_delta = Meters.mutable(0);
public Distance calculateDelta(Distance currentDistance) {
    // m_delta = currentDistance - m_lastDistance
    m_delta.mut_replace(currentDistance);
    m_delta.mut_minus(m_lastDistance);
    m_lastDistance.mut_replace(currentDistance);
    return m_delta;
}
```

Defining New Units / 定义新 Unit

There are four ways to define a new unit that isn't already present in the library:

可以通过四种方式定义 Library 中尚不存在的新 Unit:

- Using the `Unit.per` or `Unit.mult` methods to create a composite of two other units;
- 使用 `Unit.per` 或 `Unit.mult` Method 创建另外两个 Unit 的 Composite;
- Using the `Milli`, `Micro`, and `Kilo` helper methods;
- 使用 `Milli`、`Micro` 和 `Kilo` Helper Method;
- Using the `derive` method and customizing how the new unit relates to the base unit; and
- 使用 `derive` Method, 并自定义新 Unit 与 Base Unit 的关系; 以及
- Subclassing `Unit` to define a new dimension.
- 通过 Subclass `Unit` 定义新 Dimension。

New units can be defined as combinations of existing units using the `Unit.mult` and `Unit.per` methods.

可以使用 `Unit.mult` 和 `Unit.per` Method, 将新 Unit 定义为现有 Unit 的组合。

```
PerUnit<VoltageUnit, DistanceUnit> VoltsPerInch = Volts.per(Inch);
VelocityUnit<MassUnit> KgPerSecond = Kilograms.per(Second); // Could also be declared as PerUnit<MassUnit,
TimeUnit>
DistanceUnit FootMinutesPerSecond = FeetPerSecond.mult(Minutes);
```

Using `mult` and `per` will store the resulting unit. Every call will return the same object to avoid unnecessary allocations and garbage collector pressure.

使用 `mult` 和 `per` 会存储生成的 Unit。每次调用都会返回同一个 Object, 以避免不必要的 Allocation 和 Garbage Collector Pressure。

```
@Override
public void robotPeriodic() {
    // Feet.per(Millisecond) creates a new unit on the first loop,
    // which will be reused on every successive loop
    SmartDashboard.putNumber("Speed", m_drivebase.getSpeed().in(Feet.per(Millisecond)));
}
```

NOTE: Calling `Unit.per(Time)` will return a velocity unit, which is different from and incompatible with a Per unit!

说明： 调用 `Unit.per(Time)` 会返回 `Velocity Unit`，它与 `Per Unit` 不同且不兼容。

New dimensions can also be created by subclassing `Unit` and implementing the two constructors. Dimension-specific measurement types are recommended, but take considerable effort to implement all the unit-specific math operations.

也可以通过 Subclass `Unit` 并实现两个 Constructor 来创建新 Dimension。建议使用特定 Dimension 的 Measurement Type，但实现所有 Unit-specific Mathematical Operation 需要付出大量工作。

```

public class ElectricChargeUnit extends Unit {
    public ElectricCharge(double baseUnitEquivalent, String name, String symbol) {
        super(ElectricCharge.class, baseUnitEquivalent, name, symbol);
    }
    // required for derivation with Milli, Kilo, etc.
    public ElectricCharge(UnaryFunction toBaseConverter, UnaryFunction fromBaseConverter, String name,
String symbol) {
        super(ElectricCharge.class, toBaseConverter, fromBaseConverter, name, symbol);
    }

    @Override
    public ElectricChargeUnit getBaseUnit() {
        // The base method must be overridden in order to return the correct type
        return (ElectricChargeUnit) super.getBaseUnit();
    }

    @Override
    public Measure<ElectricChargeUnit> of(double magnitude) {
        return ImmutableMeasure.ofRelativeUnits(magnitude, this);
    }

    @Override
    public Measure<ElectricChargeUnit> ofBaseUnits(double baseUnitMagnitude) {
        return ImmutableMeasure.ofBaseUnits(baseUnitMagnitude, this);
    }

    @Override
    public Measure<ElectricChargeUnit> zero() {
        return (Measure<ElectricChargeUnit>) super.zero();
    }

    @Override
    public Measure<ElectricChargeUnit> one() {
        return (Measure<ElectricChargeUnit>) super.one();
    }

    @Override
    public MutableMeasure<ElectricChargeUnit> mutable(double magnitude) {
        return new GenericMutableMeasureImpl(magnitude, toBaseUnits(magnitude), this);
    }

    @Override
    public VelocityUnit<ElectricChargeUnit> per(TimeUnit period) {
        // Note: technically, this would return a CurrentUnit, since electric charge per time is current
        (measured in Amperes)
        return VelocityUnit.combine(this, period);
    }

    public double convertFrom(double magnitude, ElectricChargeUnit otherUnit) {
        return fromBaseUnits(otherUnit.toBaseUnits(magnitude));
    }
}

public static final ElectricCharge Coulomb = new ElectricCharge(1, "Coulomb", "C");
public static final ElectricCharge ElectronCharge = new ElectricCharge(1.60217646e-19, "Electron Charge",
    "e");
public static final ElectricCharge AmpHour = new ElectricCharge(3600, "Amp Hour", "Ah");
public static final ElectricCharge MilliampHour = Milli(AmpHour);

```

Joysticks / Joystick

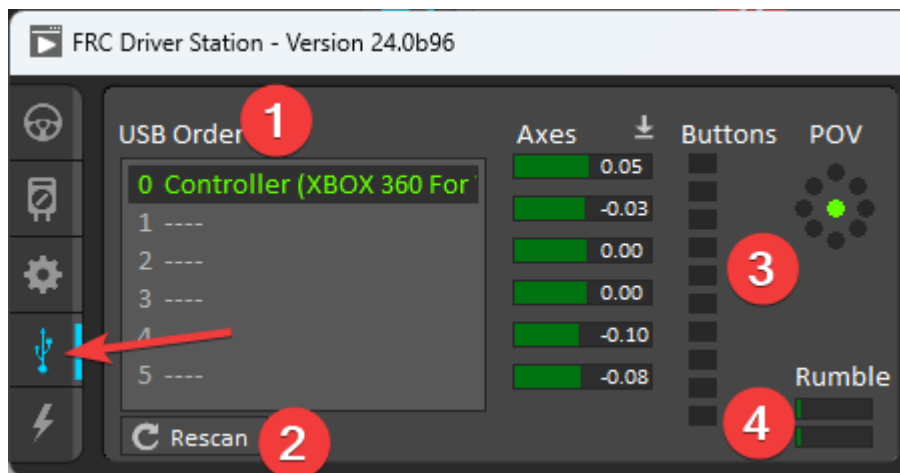
A joystick can be used with the Driver Station program to control the robot. Almost any "controller" that can be recognized by Windows can be used as a joystick. Joysticks are accessed using the `GenericHID` class. This class has three relevant subclasses for preconfigured joysticks. You may also implement your own for other controllers by extending `GenericHID`. The first is `Joystick` which is useful for standard flight joysticks. The second is `XboxController` which works for the Xbox 360, Xbox One, or Logitech F310 (in XInput mode). Finally, the `PS4Controller` class is ideal for using that controller. Each axis of the controller ranges from -1 to 1.

可以通过 Driver Station Program 使用 Joystick 控制机器人。几乎所有能够被 Windows 识别的“Controller”都可以用作 Joystick。通过 `GenericHID` 类访问 Joystick。该类为预先配置的 Joystick 提供三个相关 Subclass。对于其他 Controller，也可以扩展 `GenericHID` 来实现自己的类。第一个是 `Joystick`，适用于标准 Flight Joystick。第二个是 `XboxController`，适用于 Xbox 360、Xbox One 或处于 XInput Mode 的 Logitech F310。最后，`PS4Controller` 类适合使用 PlayStation 4 Controller。Controller 的每个 Axis 范围均为 -1 到 1。

The command based way to use these classes is detailed in the section: Binding Commands to Triggers.

在 Command-Based 框架中使用这些类的方法详见“Binding Commands to Triggers”章节。

Driver Station Joysticks / Driver Station Joystick



The 4th tab down on the left hand side is the USB devices tab. / 左侧向下第 4 个 Tab 是 USB Devices Tab。

The USB Devices Tab of the Driver Station is used to setup and configure the joystick for use with the robot. Pressing a button on a joystick will cause its entry in the table to light up green. Selecting the joystick will show the values of axes, buttons, and the POV that can be used to determine the mapping between physical joystick features and axis or button numbers.

Driver Station 的 USB Devices Tab 用于设置并配置机器人所使用的 Joystick。按下 Joystick 上的某个 Button，会使其表格条目亮起绿色。选择 Joystick 后，会显示 Axis、Button 和 POV 的数值，可用于确定 Joystick 上的实体控件与 Axis Number 或 Button Number 之间的映射关系。



On the USB tab the indicators light up to show what is currently being pressed. / USB Tab 上的指示器亮起，显示当前按下的控件。

The USB Devices Tab also assigns a joystick index to each joystick. To reorder the joysticks simply click and drag. The Driver Station software will try to preserve the ordering of devices between runs. It is a good idea to note what order your devices should be in and check each time you start the Driver Station software that they are correct.

USB Devices Tab 还会为每个 Joystick 分配一个 Joystick Index。只需单击并拖动即可重新排列 Joystick。Driver Station Software 会尝试在多次运行之间保持设备顺序。建议记录设备应有的顺序，并在每次启动 Driver Station Software 时检查顺序是否正确。

When the Driver Station is in disabled mode, it is routinely looking for status changes on the joystick devices. Unplugged devices are removed from the list and new devices are opened and added. When not connected to the FMS, unplugging a joystick will force the Driver Station into disabled mode. To start using the joystick again: plug the joystick in, check that it shows up in the right spot, then re-enable the robot. While the Driver Station is in enabled mode, it will not scan for new devices. This is a time consuming operation and timely update of signals from attached devices takes priority.

Driver Station 处于 Disabled Mode 时，会定期检查 Joystick Device 的状态变化。被拔出的设备会从列表中移除，新设备则会被打开并加入列表。未连接到 FMS 时，拔出 Joystick 会强制 Driver Station 进入 Disabled Mode。若要重新使用 Joystick，请插入 Joystick，确认其显示在正确位置，然后重新启用机器人。Driver Station 处于 Enabled Mode 时不会扫描新设备，因为扫描操作耗时较长，及时更新已连接设备的信号具有更高优先级。

NOTE: For some joysticks the startup routine will read whatever position the joysticks are in as the center position, therefore, when the computer is turned on (or when the joystick is plugged in) the joysticks should be at their center position.

****说明：** **对于某些 Joystick，启动流程会把 Joystick 当时所在的位置读取为中心位置。因此，打开计算机时或插入 Joystick 时，应让 Joystick 处于中心位置。

When the robot is connected to the Field Management System at competition, the Driver Station mode is dictated by the FMS. This means that you cannot disable your robot and the DS cannot disable itself in order to detect joystick changes. A manual complete refresh of the joysticks can be initiated by pressing the F1 key on the keyboard. Note that this will close and re-open all devices, so all devices should be in their center position as noted above.

比赛期间机器人连接到 Field Management System 时，Driver Station Mode 由 FMS 决定。这意味着无法为了检测 Joystick 变化而禁用机器人，DS 也无法自行进入 Disabled Mode。按下键盘上的 F1 Key 可以手动对所有 Joystick 执行完整刷新。请注意，此操作会关闭并重新打开所有设备，因此所有设备都应如上所述处于中心位置。

Joystick Class / Joystick 类



A Logitech flight stick with an explanation of the axis values and buttons. / Logitech Flight Stick, 其中说明了 Axis Value 和 Button。

```
Joystick exampleJoystick = new Joystick(0); // 0 is the USB Port to be used as indicated on the Driver Station
```

The `Joystick` class is designed to make using a flight joystick to operate the robot significantly easier. Depending on the flight joystick, the user may need to set the specific X, Y, Z, and Throttle channels that your flight joystick uses. This class offers special methods for accessing the angle and magnitude of the flight joystick.

`Joystick` 类旨在显著简化使用 Flight Joystick 操控机器人的过程。根据所用 Flight Joystick，用户可能需要设置该 Joystick 使用的具体 X、Y、Z 和 Throttle Channel。此类提供了访问 Flight Joystick 角度和幅值的专用方法。

IMPORTANT: Due to differences in coordinate systems, teams usually negate the values when reading joystick axes. See the Joystick and controller coordinate system section for more detail.

****重要:**** 由于坐标系不同，队伍读取 Joystick Axis 时通常会对数值取负。更多详细信息请参阅“Joystick and controller coordinate system”章节。

XboxController Class / XboxController 类



Original Xbox Controller. / 初代 Xbox Controller。

```
XboxController exampleXbox = new XboxController(0); // 0 is the USB Port to be used as indicated on the Driver Station
```

The `XboxController` class provides named methods (e.g. `getXButton`, `getXButtonPressed`, `getXButtonReleased`) for each of the buttons, and the indices can be accessed with `XboxController.Button.kX.value`. The rumble feature of the controller can be controlled by using `XboxController.setRumble(GenericHID.RumbleType.kRightRumble, value)`. Many users do a split stick arcade drive that uses the left stick for just forwards / backwards and the right stick for left / right turning.

XboxController 类为每个 Button 提供了具名方法，例如 `getXButton`、`getXButtonPressed` 和 `getXButtonReleased`；可以通过 `XboxController.Button.kX.value` 访问对应 Index。可以使用 `XboxController.setRumble(GenericHID.RumbleType.kRightRumble, value)` 控制 Controller 的振动功能。许多用户会使用 Split-Stick Arcade Drive：左摇杆仅控制前进/后退，右摇杆控制左转/右转。

IMPORTANT: Due to differences in coordinate systems, teams usually negate the values when reading joystick axes. See the Joystick and controller coordinate system section for more detail.

****重要：** **由于坐标系不同，队伍读取 Joystick Axis 时通常会对数值取负。更多详细信息请参阅“Joystick and controller coordinate system” 章节。

PS4Controller Class / PS4Controller 类



PlayStation 4 controller. / PlayStation 4 Controller.

```
PS4Controller examplePS4 = new PS4Controller(0); // 0 is the USB Port to be used as indicated on the Driver Station
```

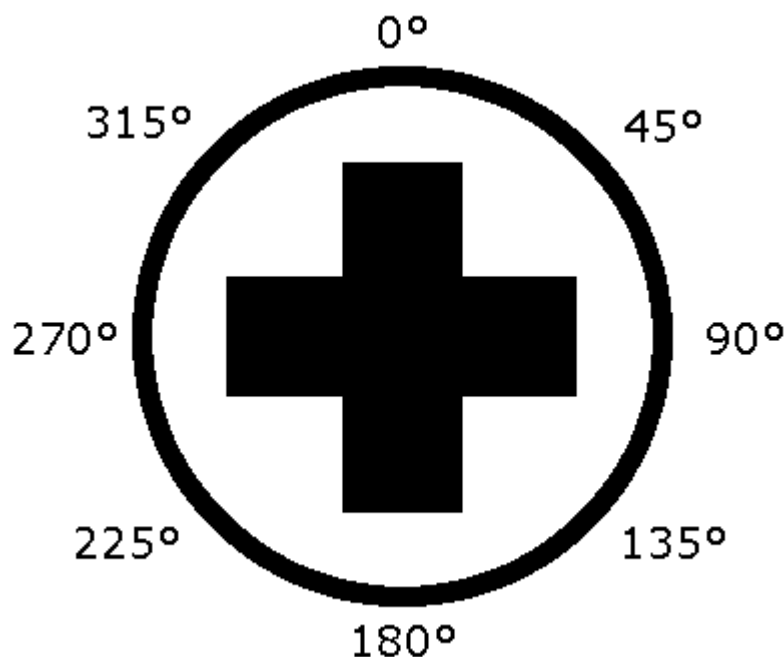
The PS4Controller class provides named methods (e.g. `getSquareButton`, `getSquareButtonPressed`, `getSquareButtonReleased`) for each of the buttons, and the indices can be accessed with `PS4Controller.Button.kSquare.value`. The rumble feature of the controller can be controlled by using `PS4Controller.setRumble(GenericHID.RumbleType.kRightRumble, value)`.

PS4Controller 类为每个 Button 提供了具名方法，例如 `getSquareButton`、`getSquareButtonPressed` 和 `getSquareButtonReleased`；可以通过 `PS4Controller.Button.kSquare.value` 访问对应 Index。可以使用 `PS4Controller.setRumble(GenericHID.RumbleType.kRightRumble, value)` 控制 Controller 的振动功能。

IMPORTANT: Due to differences in coordinate systems, teams usually negate the values when reading joystick axes. See the Joystick and controller coordinate system section for more detail.

****重要：****由于坐标系不同，队伍读取 Joystick Axis 时通常会对数值取负。更多详细信息请参阅“Joystick and controller coordinate system”章节。

POV / POV



The angles used by the code of the POV/D-pad with 0 at the top and continuing clockwise. / 代码中的 POV/D-Pad 角度：顶部为 0，并沿顺时针方向增加。

On joysticks, the POV is a directional hat that can select one of 8 different angles or read -1 for unpressed. The XboxController/PS4Controller D-pad works the same as a POV. Be careful when using a POV with exact angle requirements as it is hard for the user to ensure they select exactly the angle desired.

在 Joystick 上，POV 是一种方向帽，可以选择 8 个不同角度之一；未按下时读数为 -1。

XboxController/PS4Controller 的 D-Pad 与 POV 工作方式相同。使用有精确角度要求的 POV 时需要谨慎，因为用户很难保证准确选择所需角度。

GenericHID Usage / GenericHID 用法

An axis can be used with `.getRawAxis(int index)` (if not using any of the classes above) that returns the current value. Zero and one in this example are each the index of an axis as found in the Driver Station mentioned above.

如果未使用上述任何类，可以通过 `.getRawAxis(int index)` 使用 Axis，该方法返回当前值。此示例中的 0 和 1 分别是上文所述 Driver Station 中显示的 Axis Index。

```
private final PWMSparkMax m_leftMotor = new PWMSparkMax(Constants.kLeftMotorPort);
private final PWMSparkMax m_rightMotor = new PWMSparkMax(Constants.kRightMotorPort);
private final DifferentialDrive m_robotDrive = new DifferentialDrive(m_leftMotor::set, m_rightMotor::set);
private final GenericHID m_stick = new GenericHID(Constants.kJoystickPort);
m_robotDrive.arcadeDrive(-m_stick.getRawAxis(0), m_stick.getRawAxis(1));
```

Button Usage / Button 用法

NOTE: Usage such as the following is for code not using the command-based framework. For button usage in the command-based framework, see Binding Commands to Triggers.

****说明：****以下用法适用于未使用 Command-Based Framework 的代码。有关 Command-Based Framework 中的 Button 用法，请参阅“Binding Commands to Triggers”。

Unlike an axis, you will usually want to use the `pressed` and `released` methods to respond to button input. These will return true if the button has been activated since the last check. This is helpful for taking an action once when the event occurs but not having to continuously do it while the button is held down.

与 Axis 不同，响应 Button Input 时通常应使用 `pressed` 和 `released` 方法。如果自上次检查后 Button 被激活，这些方法会返回 true。这样可以在 Event 发生时只执行一次操作，而无需在按住 Button 期间持续执行。

```
if (joystick.getRawButtonPressed(0)) {
    turnIntakeOn(); // When pressed the intake turns on
}
if (joystick.getRawButtonReleased(0)) {
    turnIntakeOff(); // When released the intake turns off
}
// OR
if (joystick.getRawButton(0)) {
    turnIntakeOn();
} else {
    turnIntakeOff();
}
```

A common request is to toggle something on and off with the press of a button. Toggles should be used with caution, as they require the user to keep track of the robot state.

一种常见需求是按下 Button 时切换某项功能的开关状态。使用 Toggle 时应谨慎，因为它要求用户持续掌握机器人的当前状态。

```
boolean toggle = false;
if (joystick.getRawButtonPressed(0)) {
    if (toggle) {
        // Current state is true so turn off
        retractIntake();
        toggle = false;
    } else {
        // Current state is false so turn on
        deployIntake();
        toggle = true;
    }
}
```

Coordinate System / 坐标系

Coordinate systems are used in FRC programming in several places. A few of the common places are: robot movement, joystick input, pose estimation, AprilTags, and path planning.

FRC 编程在多个方面都会使用坐标系，常见用途包括机器人运动、Joystick Input、位姿（pose）估计、AprilTag 和路径规划。

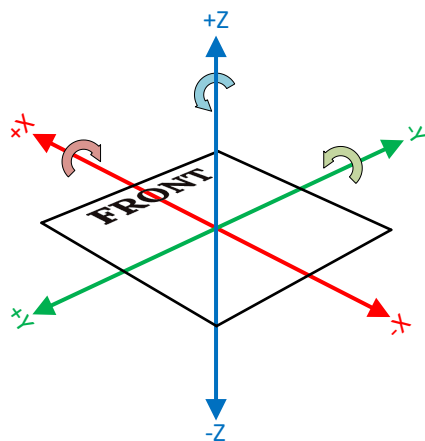
It is important to understand the basics of the coordinate system used throughout WPILib and other common tools for programming an FRC robot, such as PathPlanner. Many teams intuitively think of a coordinate system that is different from what is used in WPILib, and this leads to problems that need to be tracked down throughout the season. It is worthwhile to take a few minutes to understand the coordinate system, and come back here as a reference when programming. It's not very difficult to get robot movement with a joystick working without getting the coordinate system right, but it will be much more difficult to build on code using a different coordinate system to add pose estimation with AprilTags and path planning for autonomous.

理解 WPILib 以及 PathPlanner 等其他常用 FRC Robot 编程工具所采用坐标系的基础知识非常重要。许多队伍凭直觉想到的坐标系与 WPILib 所使用的坐标系不同，这会导致整个赛季都需要持续排查的问题。花几分钟理解该坐标系很有价值，编程时也可以回到本文作为参考。即使没有正确处理坐标系，让 Joystick 控制机器人运动也并不困难；但如果代码使用了不同的坐标系，之后要在其基础上添加通过 AprilTag 进行的 pose 估计和 Autonomous 路径规划，就会困难得多。

WPILib coordinate system / WPILib 坐标系

In most cases, WPILib uses the NWU axes convention (North-West-Up as external reference in the world frame.) In the NWU axes convention, where the positive X axis points ahead, the positive Y axis points left, and the positive Z axis points up referenced from the floor. When viewed with each positive axis pointing toward you, counter-clockwise (CCW) is a positive value and clockwise (CW) is a negative value.

在大多数情况下，WPILib 使用 NWU 轴约定（在 World Frame 中以 North-West-Up 作为外部参考）。在 NWU 轴约定中，X 轴正方向指向前方，Y 轴正方向指向左侧，Z 轴正方向以地面为参考指向上方。当从每个正轴朝向观察者的方向看时，逆时针（CCW）为正，顺时针（CW）为负。



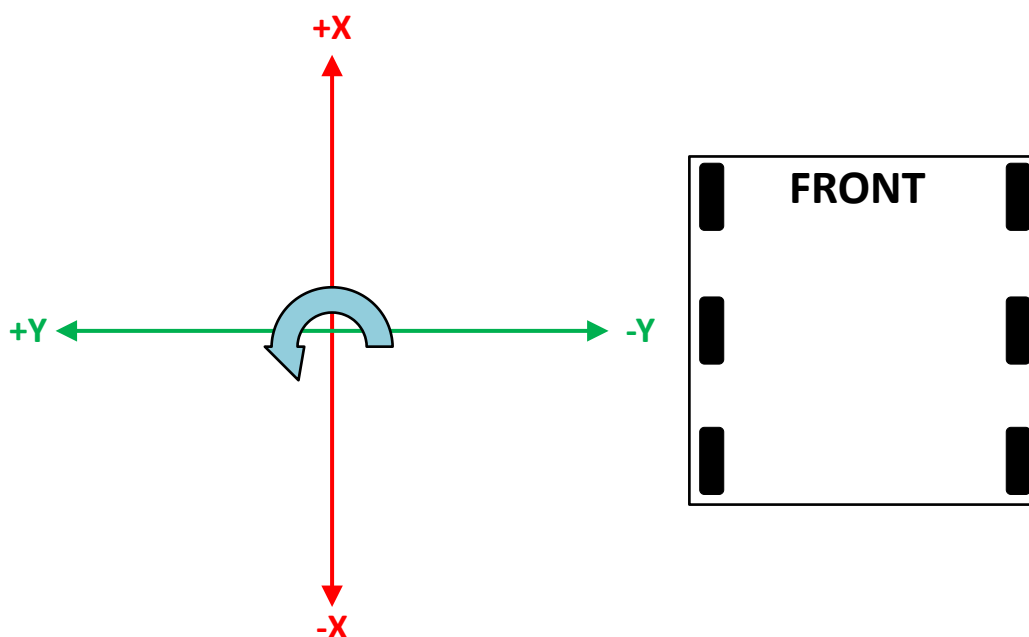
NWU axes convention in three dimensions / 三维 NWU 轴约定

Robot coordinate system in three dimensions

三维机器人坐标系。

The figure above shows the coordinate system in relation to an FRC robot. The figure below shows this same coordinate system when viewed from the top (with the Z axis pointing toward you.) This is how you can think of the robot's coordinates in 2D.

上图展示了该坐标系与 FRC Robot 的相对关系。下图展示从顶部观察同一坐标系时的情况，此时 Z 轴朝向观察者。可以用这种方式理解机器人的二维坐标。



NWU axes convention in two dimensions / 二维 NWU 轴约定

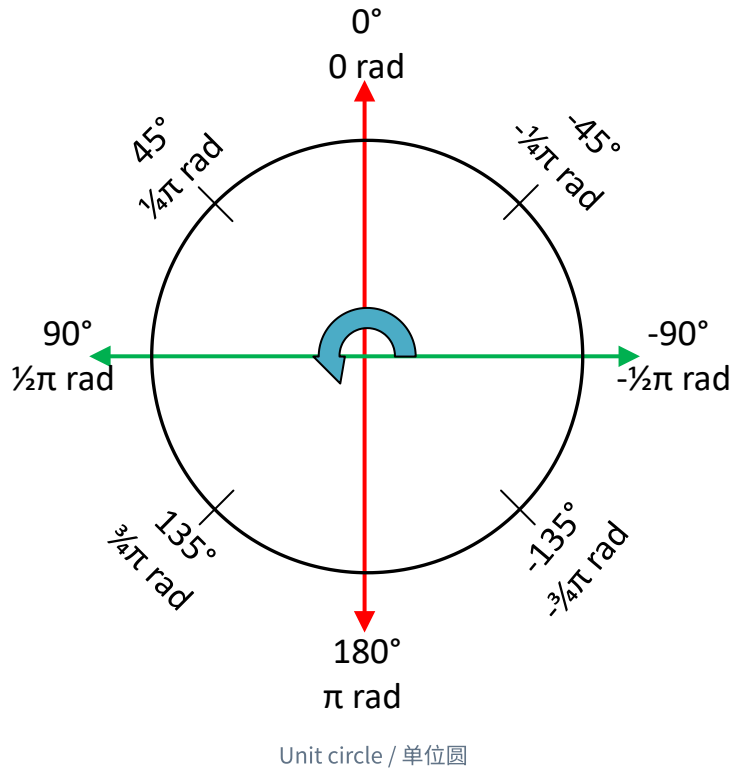
Robot coordinate system in two dimensions

二维机器人坐标系。

Rotation conventions / 旋转约定

In most cases in WPILib programming, 0° is aligned with the positive X axis, and 180° is aligned with the negative X axis. CCW rotation is positive, so 90° is aligned with the positive Y axis, and -90° is aligned with the negative Y axis.

在 WPILib 编程的大多数情况下， 0° 与 X 轴正方向对齐， 180° 与 X 轴负方向对齐。CCW 旋转为正，因此 90° 与 Y 轴正方向对齐， -90° 与 Y 轴负方向对齐。



Unit circle with common angles

标有常用角度的单位圆。

The figure above shows the unit circle with common angles labeled in degrees ($^\circ$) and radians (rad). Notice that rotation to the right is negative, and the range for the whole unit circle is -180° to 180° ($-\pi$ radians to π radians).

上图展示了单位圆，并以 Degree ($^\circ$) 和 Radian (rad) 标出了常用角度。请注意，向右旋转为负，整个单位圆的范围为 -180° 到 180° ($-\pi$ radian 到 π radian)。

NOTE: The range is $(-180, 180]$, meaning it is exclusive of -180° and inclusive of 180° .

**说明: **范围为 $(-180, 180]$ ，即不包含 -180° ，但包含 180° 。

There are some places you may choose to use a different range, such as 0° to 360° or 0 to 1 rotation, but be aware that many core WPILib classes and FRC tools are built with the unit circle above.

在某些情况下可以选择不同范围，例如 0° 到 360° ，或 0 到 1 rotation；但需要注意，许多 WPILib 核心类和 FRC 工具都以上述单位圆为基础。

WARNING: Some gyroscope and IMU models use CW positive rotation, such as the NavX IMU. Care must be taken to handle rotation properly, sensor values may need to be inverted. Read the documentation and verify that rotation is CCW positive.

****警告:**** 某些 Gyroscope 和 IMU 型号（例如 NavX IMU）使用 CW 正旋转。必须谨慎、正确地处理旋转，Sensor Value 可能需要取反。请阅读相应文档，并确认旋转以 CCW 为正。

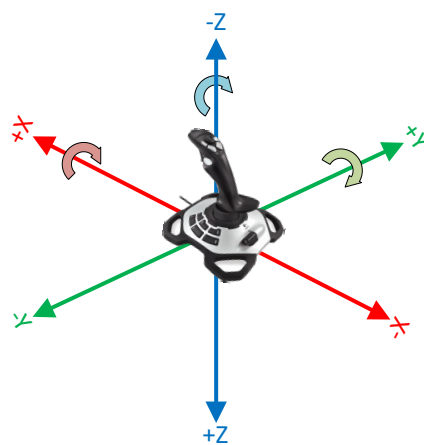
WARNING: Many sensors that read rotation around an axis, such as encoders and IMU's, read continuously. This means they read more than one rotation, so when rotating past 180° they read 181° , not -179° . Some sensors have configuration settings where you can choose their wrapping behavior and range, while others need to be handled in your code. Careful attention should be paid to make sure sensor readings are consistent and your control loop handles wrapping in the same way as your sensor.

****警告:**** 许多读取绕轴旋转的 Sensor（例如 Encoder 和 IMU）会连续读取。这意味着它们会读取超过一整圈的旋转，因此越过 180° 后会读出 181° ，而不是 -179° 。某些 Sensor 提供配置选项，可选择其 Wrap 行为和范围；其他 Sensor 则需要在代码中处理。必须仔细确认 Sensor Reading 保持一致，并且 Control Loop 处理 Wrap 的方式与 Sensor 相同。

Joystick and controller coordinate system / Joystick 和 Controller 坐标系

Joysticks, including the sticks on controllers, don't use the same NWU coordinate system. They use the NED (North-East-Down) convention, where the positive X axis points ahead, the positive Y axis points right, and the positive Z axis points down. When viewed with each positive axis pointing toward you, counter-clockwise (CCW) is a positive value and clockwise (CW) is a negative value.

Joystick（包括 Controller 上的摇杆）不使用相同的 NWU 坐标系，而是使用 NED（North-East-Down）约定：X 轴正方向指向前方，Y 轴正方向指向右侧，Z 轴正方向指向下方。当从每个正轴朝向观察者的方向看时，逆时针（CCW）为正，顺时针（CW）为负。



NED axes convention / NED 轴约定

Joystick coordinate system

Joystick 坐标系。

It's important to note that joystick input values are rotations around an axis, not translations. In practical terms, this means:

必须注意，Joystick Input Value 表示绕轴旋转，而不是平移。具体而言：

- pushing forward on the joystick (toward the positive X axis) is a CW rotation around the Y axis, so you get a negative Y value.

向前推动 Joystick（朝 X 轴正方向）是绕 Y 轴的 CW 旋转，因此会得到负 Y Value。

- pushing to the right (toward the positive Y axis) is a CCW rotation around the X axis, so you get a positive X value.

向右推动 Joystick（朝 Y 轴正方向）是绕 X 轴的 CCW 旋转，因此会得到正 X Value。

- twisting the joystick CW (toward the positive Y axis) is a CCW rotation around the Z axis, so you get a positive Z value.

将 Joystick 沿 CW 方向扭转（朝 Y 轴正方向）是绕 Z 轴的 CCW 旋转，因此会得到正 Z Value。

Using Joystick and controller input to drive a robot / 使用 Joystick 和 Controller Input 驱动机器人

You may have noticed, the coordinate system used by WPILib for the robot is not the same as the coordinate system used for joysticks and controllers. Care needs to be taken to understand the difference, and properly pass driver input to the drive subsystem.

可以注意到，WPILib 为机器人使用的坐标系与 Joystick 和 Controller 使用的坐标系不同。需要仔细理解两者的差异，并将 Driver Input 正确传递给 Drive Subsystem。

Differential drivetrain example / Differential Drivetrain 示例

Differential drivetrains are non-holonomic, which means the robot drivetrain cannot move side-to-side (strafe). This type of drivetrain can move forward and backward along the X axis, and rotate around the Z axis. Consider a common arcade drive scheme using a single joystick where the driver pushes the joystick forward/backward for forward/backward robot movement, and push the joystick left/right to rotate the robot left/right.

Differential Drivetrain 是非完整约束（non-holonomic）的，这意味着 Robot Drivetrain 无法横向移动（strafe）。这种 Drivetrain 可以沿 X 轴前后移动，并绕 Z 轴旋转。考虑一种使用单个 Joystick 的常见 Arcade Drive 方案：Driver 前后推动 Joystick 控制机器人前后移动，左右推动 Joystick 控制机器人左右旋转。

The code snippet below uses the `DifferentialDrive` and `Joystick` classes to drive the robot with the arcade scheme described above. `DifferentialDrive` uses the robot coordinate system defined above, and `Joystick` uses the joystick coordinate system.

下方代码片段使用 `DifferentialDrive` 和 `Joystick` 类，按照上述 Arcade Drive 方案驱动机器人。
`DifferentialDrive` 使用上文定义的机器人坐标系，`Joystick` 使用 `Joystick` 坐标系。

```
public void teleopPeriodic() {  
    // Arcade drive with a given forward and turn rate  
    myDrive.arcadeDrive(-driveStick.getY(), -driveStick.getX());  
}
```

The code calls the `DifferentialDrive.arcadeDrive(xSpeed, zRotation)` method, with values it gets from the `Joystick` class:

代码调用 `DifferentialDrive.arcadeDrive(xSpeed, zRotation)` 方法，并传入从 `Joystick` 类获得的数值：

- The first argument is `xSpeed`

第一个参数是 `xSpeed`：

- Robot: `xSpeed` is the speed along the robot's X axis, which is forward/backward.

Robot: `xSpeed` 是沿机器人 X 轴的速度，即前进/后退速度。

- Joystick: The driver sets forward/backward speed by rotating the joystick along its Y axis, which is pushing the joystick forward/backward.

Joystick: Driver 通过使 Joystick 绕 Y 轴旋转来设置前进/后退速度，也就是前后推动 Joystick。

- Code: Moving the joystick forward is negative Y rotation, whereas moving the robot forward is along the positive X axis. This means the joystick value needs to be inverted by placing a - (minus sign) in front of the value.

Code: 向前推动 Joystick 是负 Y 旋转，而机器人向前运动沿 X 轴正方向。因此需要在 Joystick Value 前添加 - (Minus Sign) 将其取反。

- The second argument is `zRotation`

第二个参数是 `zRotation`：

- Robot: `zRotation` is the speed of rotation along the robot's Z axis, which is rotating left/right.

Robot: `zRotation` 是绕机器人 Z 轴的旋转速度，即左右旋转速度。

- Joystick: The driver sets rotation speed by rotating the joystick along its X axis, which is pushing the joystick left/right.

Joystick: Driver 通过使 Joystick 绕 X 轴旋转来设置旋转速度，也就是左右推动 Joystick。

- Code: Moving the joystick to the right is positive X rotation, whereas robot rotation is CCW positive. This means the joystick value needs to be inverted by placing a - (minus sign) in front of the value.

Code: 向右推动 Joystick 是正 X 旋转，而机器人旋转以 CCW 为正。因此需要在 Joystick Value 前添加 - (Minus Sign) 将其取反。

Mecanum drivetrain example / Mecanum Drivetrain 示例

Mecanum drivetrains are holonomic, meaning they have the ability to move side-to-side. This type of drivetrain can move forward/backward and rotate around the Z axis like differential drivetrains, but it can also move side-to-side along the robot's Y axis. Consider a common arcade drive scheme using a single joystick where the driver pushes the joystick forward/backward for forward/backward robot movement, pushes the joystick left/right to move side-to-side, and twists the joystick to rotate the robot.

Mecanum Drivetrain 是完整约束（holonomic）的，因此能够横向移动。与 Differential Drivetrain 一样，这种 Drivetrain 可以前后移动并绕 Z 轴旋转；此外，它还能沿机器人的 Y 轴横向移动。考虑一种使用单个 Joystick 的常见 Arcade Drive 方案：Driver 前后推动 Joystick 控制机器人前后移动，左右推动 Joystick 控制横向移动，并扭转 Joystick 控制机器人旋转。

```
public void teleopPeriodic() {
    // Drive using the X, Y, and Z axes of the joystick.
    m_robotDrive.driveCartesian(-m_stick.getY(), -m_stick.getX(), -m_stick.getZ());
}
```

The code calls the `MecanumDrive.driveCartesian(xSpeed, ySpeed, zRotation)` method, with values it gets from the `Joystick` class:

代码调用 `MecanumDrive.driveCartesian(xSpeed, ySpeed, zRotation)` 方法，并传入从 `Joystick` 类获得的数值：

- The first argument is `xSpeed`

第一个参数是 `xSpeed`：

- Robot: `xSpeed` is the speed along the robot's X axis, which is forward/backward.

Robot: `xSpeed` 是沿机器人 X 轴的速度，即前进/后退速度。

- Joystick: The driver sets forward/backward speed by rotating the joystick along its Y axis, which is pushing the joystick forward/backward.

Joystick: Driver 通过使 Joystick 绕 Y 轴旋转来设置前进/后退速度，也就是前后推动 Joystick。

- Code: Moving the joystick forward is negative Y rotation, whereas robot forward is along the positive X axis. This means the joystick value needs to be inverted by placing a - (minus sign) in front of the value.

Code: 向前推动 Joystick 是负 Y 旋转，而机器人向前运动沿 X 轴正方向。因此需要在 Joystick Value 前添加 - (Minus Sign) 将其取反。

- The second argument is `ySpeed`

第二个参数是 `ySpeed`：

- Robot: `ySpeed` is the speed along the robot's Y axis, which is left/right.

Robot: `ySpeed` 是沿机器人 Y 轴的速度，即左/右方向速度。

- Joystick: The driver sets left/right speed by rotating the joystick along its X axis, which is pushing the joystick left/right.

Joystick: Driver 通过使 Joystick 绕 X 轴旋转来设置左/右方向速度，也就是左右推动 Joystick。

- Code: Moving the joystick to the right is positive X rotation, whereas robot right is along the negative Y axis. This means the joystick value needs to be inverted by placing a - (minus sign) in front of the value.

Code: 向右推动 Joystick 是正 X 旋转，而机器人向右移动沿 Y 轴负方向。因此需要在 Joystick Value 前添加 - (Minus Sign) 将其取反。

- The third argument is `zRotation`

第三个参数是 `zRotation`:

- Robot: `zRotation` is the speed of rotation along the robot's Z axis, which is rotating left/right.

Robot: `zRotation` 是绕机器人 Z 轴的旋转速度，即左右旋转速度。

- Joystick: The driver sets rotation speed by twisting the joystick along its Z axis, which is twisting the joystick left/right.

Joystick: Driver 通过沿 Z 轴扭转 Joystick 来设置旋转速度，也就是左右扭转 Joystick。

- Code: Twisting the joystick to the right is positive Z rotation, whereas robot rotation is CCW positive. This means the joystick value needs to be inverted by placing a - (minus sign) in front of the value.

Code: 向右扭转 Joystick 是正 Z 旋转，而机器人旋转以 CCW 为正。因此需要在 Joystick Value 前添加 - (Minus Sign) 将其取反。

Swerve drivetrain example / Swerve Drivetrain 示例

Like mecanum drivetrains, swerve drivetrains are holonomic and have the ability to move side-to-side. Joystick control can be handled the same way for all holonomic drivetrains, but WPILib doesn't have a built-in robot drive class for swerve. Swerve coding is described in other sections of this documentation, but an example of using joystick input to set `ChassisSpeeds` values is included below. Consider the same common arcade drive scheme described in the mecanum section above. The scheme uses a single joystick where the driver pushes the joystick forward/backward for forward/backward robot movement, pushes the joystick left/right to move side-to-side, and twists the joystick to rotate the robot.

与 Mecanum Drivetrain 一样，Swerve Drivetrain 也是完整约束的，能够横向移动。所有 Holonomic Drivetrain 都可以用相同方式处理 Joystick Control，但 WPILib 没有针对 swerve 的内置 Robot Drive 类。本文档的其他章节介绍了 swerve 编程；下方则给出一个使用 Joystick Input 设置 `ChassisSpeeds` Value 的示例。考虑与上方 Mecanum 章节相同的常见 Arcade Drive 方案：使用单个 Joystick，Driver 前后推动 Joystick 控制机器人前后移动，左右推动 Joystick 控制横向移动，并扭转 Joystick 控制机器人旋转。

```
// Drive using the X, Y, and Z axes of the joystick.
var speeds = new ChassisSpeeds(-m_stick.getY(), -m_stick.getX(), -m_stick.getZ());
```

The three arguments to the `ChassisSpeeds` constructor are the same as `driveCartesian` in the mecanum section above; `xSpeed`, `ySpeed`, and `zRotation`. See the description of the arguments, and their joystick input in the section above.

`ChassisSpeeds` Constructor 的三个参数与上方 Mecanum 章节中的 `driveCartesian` 相同，分别为 `xSpeed`、`ySpeed` 和 `zRotation`。有关这些参数及其 Joystick Input 的说明，请参阅上方章节。

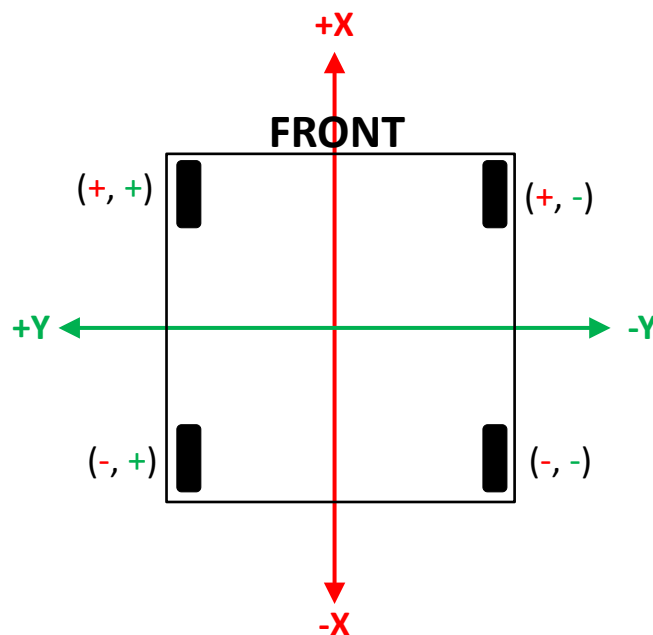
Robot drive kinematics / Robot Drive Kinematics

Kinematics is a topic that is covered in a different section, but it's worth discussing here in relation to the coordinate system. It is critically important that kinematics is configured using the coordinate system described above. Kinematics is a common starting point for coordinate system errors that then cascade to basic drivetrain control, field oriented driving, pose estimation, and path planning.

Kinematics 是其他章节介绍的主题，但值得在此结合坐标系进行讨论。必须使用上文所述坐标系配置 Kinematics，这一点至关重要。Kinematics 往往是坐标系错误的起点，而错误随后会连锁影响基础 Drivetrain Control、Field-Oriented Driving、pose 估计和路径规划。

When you construct a `SwerveDriveKinematics` Or `MecanumDriveKinematics` object, you specify a translation from the center of your robot to each wheel. These translations use the coordinate system above, with the origin in the center of your robot.

构造 `SwerveDriveKinematics` 或 `MecanumDriveKinematics` 对象时，需要指定从机器人中心到每个 Wheel 的平移。这些平移使用上文所述坐标系，原点位于机器人中心。



Kinematics with translation signs / 标有平移正负号的 Kinematics

Kinematics with translation signs

标有平移正负号的 Kinematics。

For the robot in the diagram above, let's assume the distance between the front and rear wheels (wheelbase) is 2'. Let's also assume the distance between the left and right wheels (trackwidth) is also 2'. Our translations (x, y) would be like this:

对于上图中的机器人，假设前后 Wheel 之间的距离（wheelbase）为 2'，左右 Wheel 之间的距离（trackwidth）也为 2'。平移 (x, y) 如下：

- Front left: (1', 1')
左前：(1', 1')
- Front right: (1', -1')
右前：(1', -1')
- Rear left: (-1', 1')
左后：(-1', 1')
- Rear right: (-1', -1')
右后：(-1', -1')

WARNING: A common error is to use an incorrect coordinate system where the positive Y axis points forward on the robot. The correct coordinate system has the positive X axis pointing forward.

****警告：****一种常见错误是使用 Y 轴正方向指向机器人前方的错误坐标系。正确坐标系应让 X 轴正方向指向前方。

Field coordinate systems / 场地坐标系

The field coordinate system (or global coordinate system) is an absolute coordinate system where a point on the field is designated as the origin. Two common uses of the field coordinate system will be explored in this document:

场地坐标系（或全局坐标系）是一种绝对坐标系，将场地上的某一点指定为原点。本文将介绍场地坐标系的两种常见用途：

- Field oriented driving is a drive scheme for holonomic drivetrains, where the driver moves the controls relative to their perspective of the field, and the robot moves in that direction regardless of where the front of the robot is facing. For example, a driver on the red alliance pushes the joystick forward, the robot will move downfield toward the blue alliance wall, even if the robot's front is facing the driver.

Field-Oriented Driving 是用于 Holonomic Drivetrain 的 Drive Scheme。在该方案中，Driver 根据自己观察场地的视角移动控件；无论机器人前方朝向哪里，机器人都会向相应方向移动。例如，红方 Alliance 的 Driver 向前推动 Joystick 时，即使机器人前方朝向 Driver，机器人仍会沿场地向蓝方 Alliance Wall 移动。

- Pose estimation with odometry and/or AprilTags are used to estimate the robot's pose on the field.

使用里程计（odometry）和/或 AprilTag 进行 pose 估计，以估算机器人在场地上的 pose。

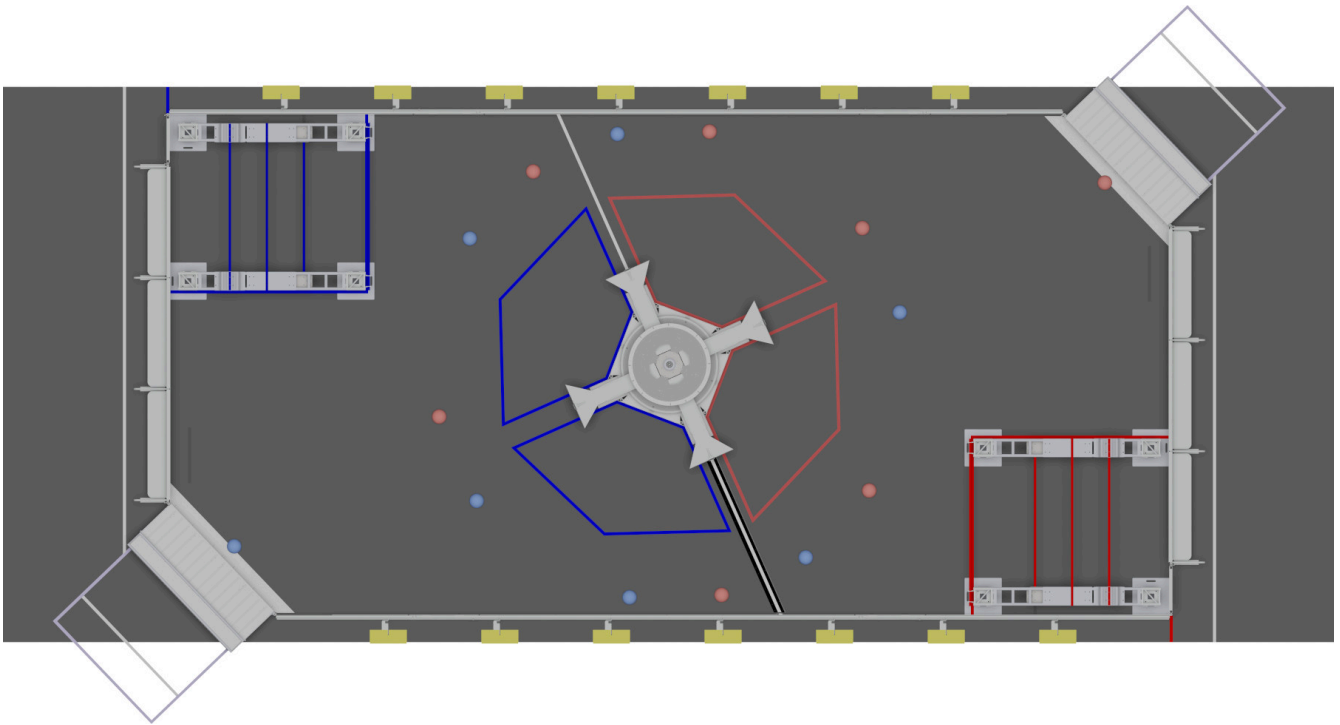
Mirrored field vs. rotated field / 镜像场地与旋转场地

Historically, FRC has used two types of field layouts in relation to the red and blue alliance.

在历史上，FRC 针对红方和蓝方 Alliance 使用过两种类型的场地布局。

Games such as Rapid React in 2022 used a rotated layout. A rotated layout means that, from your perspective from behind your alliance wall, your field elements and your opponent's elements are in the same location. Notice in the Rapid React field layout diagram below, whether you are on the red or blue alliance, your human player station is on your right and your hanger is on your left.

2022 年 Rapid React 等比赛采用旋转布局。旋转布局意味着，从己方 Alliance Wall 后方观察时，己方和对方的 Field Element 位于相同位置。请注意下方 Rapid React 场地布局图：无论处于红方还是蓝方 Alliance，己方 Human Player Station 都在右侧，Hanger 都在左侧。



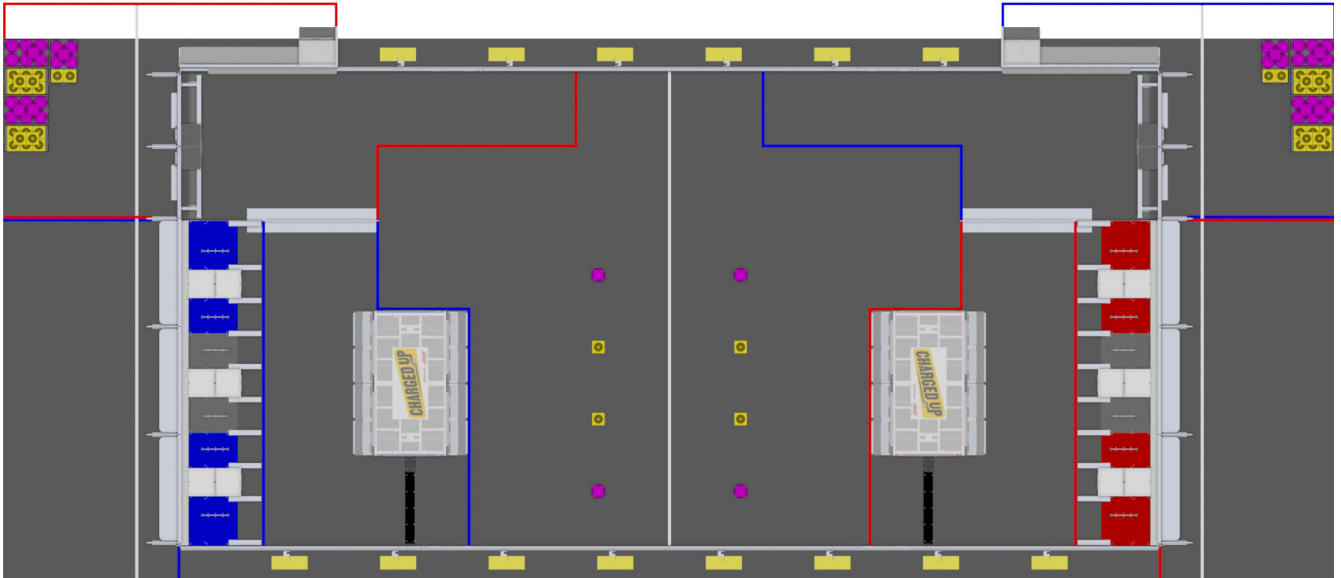
Rotated Rapid React field from 2022 / 2022 年旋转布局的 Rapid React 场地

Rotated field from RAPID REACT in 2022 (image source)

2022 年 RAPID REACT 的旋转场地（图片来源）。

Games such as CHARGED UP in 2023 and CRESCENDO in 2024 used a mirrored layout. A mirrored layout means that the red and blue alliance layout are mirrored across the centerpoint of the field. Refer to the CHARGED UP field diagram below. When you are standing behind the blue alliance wall, the charge station is on the right side of the field from your perspective. However, standing behind the red alliance wall, the charge station is on the left side of the field from your perspective.

2023 年 CHARGED UP 和 2024 年 CRESCENDO 等比赛采用镜像布局。镜像布局意味着红方和蓝方 Alliance 的布局沿场地中心点互为镜像。请参阅下方 CHARGED UP 场地图：站在蓝方 Alliance Wall 后方时，从观察者视角看，Charge Station 位于场地右侧；站在红方 Alliance Wall 后方时，Charge Station 则位于场地左侧。



Mirrored CHARGED UP field from 2023 / 2023 年镜像布局的 CHARGED UP 场地

Mirrored field from CHARGED UP in 2023 (image source)

2023 年 CHARGED UP 的镜像场地（图片来源）。

Dealing with red or blue alliance / 处理红方或蓝方 Alliance

There are two primary ways many teams choose to define the field coordinate system. In both methods, positive rotation (theta) is in the counter-clockwise (CCW) direction.

许多队伍主要采用两种方式定义场地坐标系。在这两种方式中，正旋转（theta）均为逆时针（CCW）方向。

WARNING: There are cases where your alliance may change (or appear to change) after the code is initialized. When you are not connected to the FMS at a competition, you can change your alliance station in the Driver Station application at any time. Even when you are at a competition, your robot will usually initialize before connecting to the FMS so you will not have alliance information.

****警告：****代码初始化后，Alliance 可能发生变化，或看起来发生了变化。比赛中未连接到 FMS 时，可以随时在 Driver Station Application 中更改 Alliance Station。即使位于比赛现场，机器人通常也会在连接到 FMS 前完成初始化，因此初始化时不会获得 Alliance 信息。

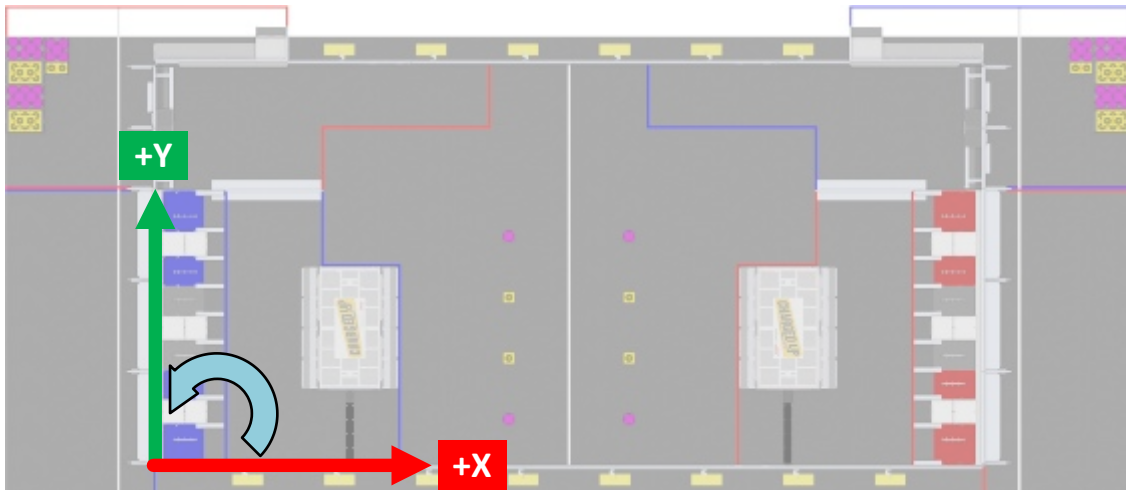
NOTE: At competition events, the FMS will automatically report your Team Station and alliance color. When you are not connected to an FMS, you can choose your Team Station and alliance color on the Driver Station operation tab.

****说明：****在比赛现场，FMS 会自动报告 Team Station 和 Alliance Color。未连接到 FMS 时，可以在 Driver Station 的 Operation Tab 中选择 Team Station 和 Alliance Color。

Always blue origin / 始终使用蓝方原点

You may choose to define the origin of the field on the blue side, and keep it there regardless of your alliance color. With this solution, positive x-axis points away from the blue alliance wall.

可以选择将场地原点定义在蓝方一侧，并且无论己方 Alliance Color 如何都保持不变。采用这种方案时，X 轴正方向背离蓝方 Alliance Wall。



CHARGED UP with blue origin / 使用蓝方原点的 CHARGED UP

CHARGED UP with blue origin

使用蓝方原点的 CHARGED UP。

Some advantages to this approach are:

此方法的一些优点包括：

- Pose estimation with AprilTags is simplified. AprilTags throughout the field are unique. If you keep the coordinate system the same regardless of alliance, there is no need for special logic to deal with the location of AprilTags on the field relative to your alliance.

简化使用 AprilTag 进行的 pose 估计。场地上的每个 AprilTag 都是唯一的。如果无论 Alliance 如何都保持坐标系不变，就无需使用特殊逻辑处理 AprilTag 在场上相对于己方 Alliance 的位置。

- Many of the tools and libraries used in FRC follow this convention. Some of the tools include: PathPlanner, Choreo, and the ShuffleBoard and Glass Field2d widget.

FRC 使用的许多工具和 Library 都遵循此约定，其中包括 PathPlanner、Choreo，以及 Shuffleboard 和 Glass 的 Field2d Widget。

In order to use this approach for field oriented driving, driver input needs to consider the alliance color. When your alliance is red and the driver is standing behind the red alliance wall, they will want the robot to move downfield toward the blue alliance wall. However, when your alliance is blue, the driver will want the robot to go downfield toward the red alliance wall.

若要将此方法用于 Field-Oriented Driving，Driver Input 必须考虑 Alliance Color。己方为红方且 Driver 站在红方 Alliance Wall 后方时，会希望机器人沿场地向蓝方 Alliance Wall 移动；己方为蓝方时，Driver 则会希望机器人沿场地向红方 Alliance Wall 移动。

A simple way to deal with field oriented driving is to check the alliance color reported by the DriverStation class, and invert the driver's controls based on the alliance. As noted above, your alliance color can change so it needs to be checked on every robot iteration.

处理 Field-Oriented Driving 的一种简单方法是检查 `DriverStation` 类报告的 Alliance Color，并根据 Alliance 对 Driver Control 取反。如上所述，Alliance Color 可能发生变化，因此必须在机器人的每次 Iteration 中进行检查。

```
// The origin is always blue. When our alliance is red, X and Y need to be inverted
var alliance = DriverStation.getAlliance();
var invert = 1;
if (alliance.isPresent() && alliance.get() == Alliance.Red) {
    invert = -1;
}
// Create field relative ChassisSpeeds for controlling Swerve
var chassisSpeeds = ChassisSpeeds
    .fromFieldRelativeSpeeds(xSpeed * invert, ySpeed * invert, zRotation, imu.getRotation2d());
// Control a mecanum drivetrain
m_robotDrive.driveCartesian(xSpeed * invert, ySpeed * invert, zRotation, imu.getRotation2d());
```

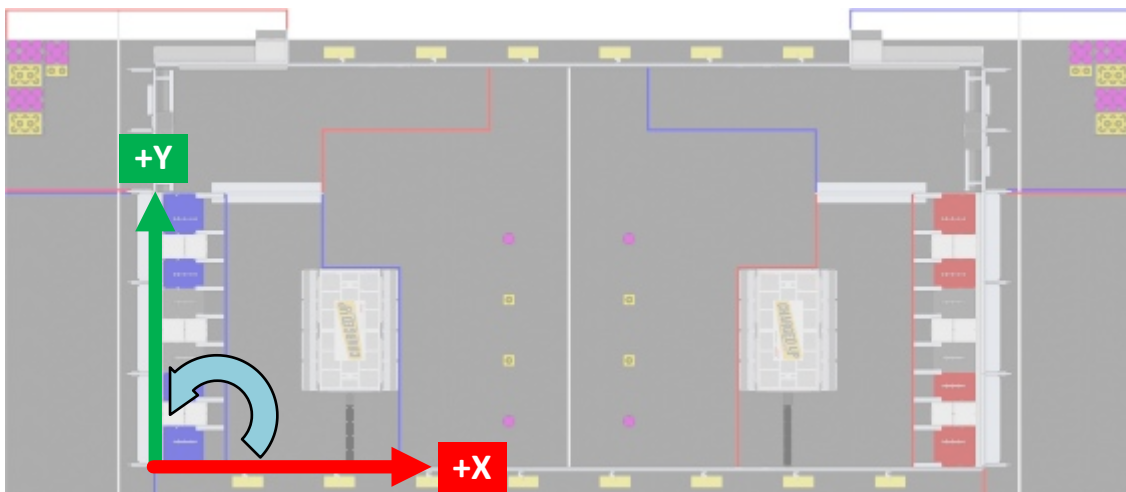
Origin follows your alliance / 原点跟随己方 Alliance

You may choose to define the origin of the field based on the alliance you are one. With this approach, the positive x-axis always points away from your alliance wall.

可以选择根据所在 Alliance 定义场地原点。采用这种方法时，X 轴正方向始终背离己方 Alliance Wall。

When you are on the blue alliance, your origin looks like this:

处于蓝方 Alliance 时，原点如下所示：



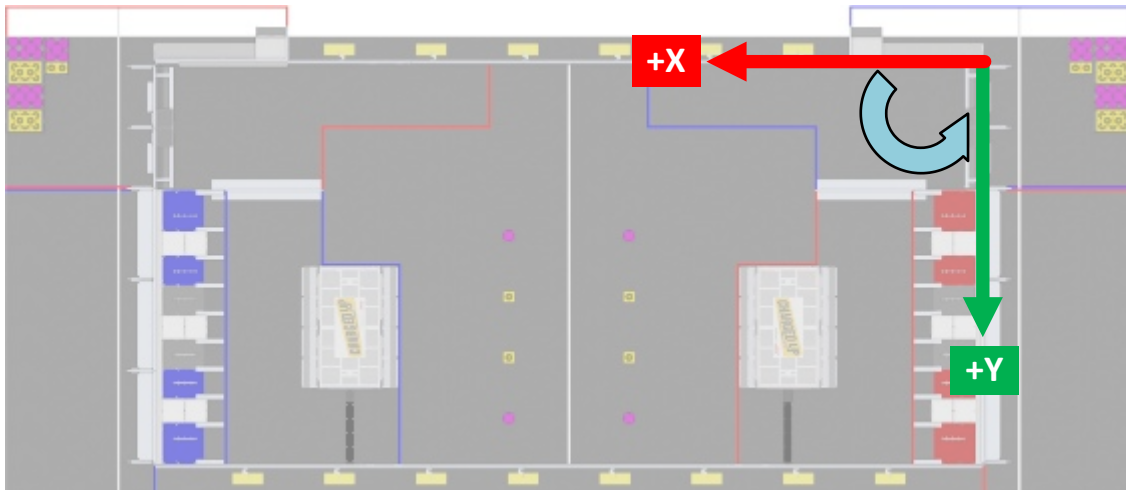
CHARGED UP with alliance as origin / 以己方 Alliance 为原点的 CHARGED UP

CHARGED UP field with blue alliance as origin

以蓝方 Alliance 为原点的 CHARGED UP 场地。

When you are on the red alliance, your origin looks like this:

处于红方 Alliance 时，原点如下所示：



CHARGED UP with alliance as origin / 以己方 Alliance 为原点的 CHARGED UP

CHARGED UP field with red alliance as origin

以红方 Alliance 为原点的 CHARGED UP 场地。

This approach has a few more complications than the previous approach, especially in years when the field layout is mirrored between alliances.

与前一种方法相比，此方法会带来更多复杂性，尤其是在两个 Alliance 之间采用镜像场地布局的赛季。

In years when the field layout is rotated, this is a simple approach if you are not using AprilTags for pose estimation or doing other advanced techniques. When the field layout is rotated, the field elements appear at the same coordinates regardless of your alliance.

在采用旋转场地布局的赛季，如果不使用 AprilTag 进行 pose 估计，也不采用其他高级技术，这是一种简单的方法。场地布局旋转时，无论己方 Alliance 如何，Field Element 都会出现在相同坐标处。

Some things you need to consider when using this approach are:

使用此方法时，需要考虑以下事项：

- As warned above, your alliance color can change after initialization. If you are not using AprilTags, you may not have anything to adjust when the alliance changes. However, if you are using AprilTags and your robot has seen a tag and used it for pose estimation, you will need to adjust your origin and reset your estimated pose.

如上方警告所述，Alliance Color 可能在初始化后发生变化。如果未使用 AprilTag，Alliance 变化时可能无需调整任何内容。但是，如果正在使用 AprilTag，且机器人已经观察到 Tag 并使用它进行 pose 估计，就需要调整原点并重置估计 pose。

- The field image in the ShuffleBoard and Glass Field2d widget follows the *Always blue origin* approach. Special handling is needed to display your robot pose correctly when your alliance is red. You will need to change the origin for your estimated pose to the blue alliance coordinate system before sending it to the dashboard.

Shuffleboard 和 Glass 的 Field2d Widget 中的场地图像遵循始终使用蓝方原点的方法。己方为红方时，需要进行特殊处理才能正确显示机器人 pose。在将估计 pose 发送到 Dashboard 前，需要把估计 pose 的原点转换到蓝方 Alliance 坐标系。

Setting Robot Preferences / 设置 Robot Preferences

The Robot Preferences (Java) class is used to store values in the flash memory on the roboRIO. The values might be for remembering preferences on the robot such as calibration settings for potentiometers, PID values, setpoints, etc. that you would like to change without having to rebuild the program. The values can be viewed on SmartDashboard or Shuffleboard and read and written by the robot program.

Robot Preferences 类（Java）用于在 roboRIO 的 Flash Memory 中存储数值。这些数值可以用于记录希望无需重新构建程序即可更改的机器人设置，例如 Potentiometer 校准设置、PID Value、setpoint 等。可以在 SmartDashboard 或 Shuffleboard 上查看这些数值，Robot Program 也可以读写它们。

This example shows how to utilize Preferences to change the setpoint of a PID controller and the P constant. The code examples are adapted from the Arm Simulation example (Java). You can run the Arm Simulation example in the Robot Simulator to see how to use the preference class and interact with it using the dashboards without needing a robot.

此示例展示如何使用 Preferences 更改 PID Controller 的 setpoint 和 P Constant。代码示例改编自 Arm Simulation 示例（Java）。可以在 Robot Simulator 中运行 Arm Simulation 示例，无需真实机器人即可了解如何使用 Preferences 类，并通过 Dashboard 与其交互。

Initializing Preferences / 初始化 Preferences

```
public static final String kArmPositionKey = "ArmPosition";
public static final String kArmPKey = "ArmP";

// The P gain for the PID controller that drives this arm.
public static final double kDefaultArmKp = 50.0;
public static final double kDefaultArmSetpointDegrees = 75.0;
```

```
// The P gain for the PID controller that drives this arm.
private double m_armKp = Constants.kDefaultArmKp;
private double m_armSetpointDegrees = Constants.kDefaultArmSetpointDegrees;
public Arm() {
    m_encoder.setDistancePerPulse(Constants.kArmEncoderDistPerPulse);
    // Set the Arm position setpoint and P constant to Preferences if the keys don't already exist
    Preferences.initDouble(Constants.kArmPositionKey, m_armSetpointDegrees);
    Preferences.initDouble(Constants.kArmPKey, m_armKp);
}
```

Preferences are stored using a name, the key. It's helpful to store the key in a constant, like `kArmPositionKey` and `kArmPKey` in the code above to avoid typing it multiple times and avoid typos. We also declare variables, `kArmKp` and `armPositionDeg` to hold the data retrieved from preferences.

Preference 使用称为 Key 的名称进行存储。将 Key 存入 Constant（例如上方代码中的 `kArmPositionKey` 和 `kArmPKey`）有助于避免重复输入和拼写错误。还声明了变量 `kArmKp` 和 `armPositionDeg`，用于保存从

Preferences 中获取的数据。

In `Arm` constructor, each key is checked to see if it already exists in the Preferences database. The `containsKey` method takes one parameter, the key to check if data for that key already exists in the preferences database. If it doesn't exist, a default value is written. The `setDouble` method takes two parameters, the key to write and the data to write. There are similar methods for other data types like booleans, ints, and strings.

在 `Arm` Constructor 中，会检查每个 Key 是否已存在于 Preferences Database。 `containsKey` 方法接收一个参数，即要检查的 Key，用于判断该 Key 的数据是否已存在于 Preferences Database。如果不存在，则写入 Default Value。 `setDouble` 方法接收两个参数：要写入的 Key 和要写入的数据。对于 Boolean、int 和 String 等其他数据类型，也有类似方法。

If using the Command Framework, this type of code could be placed in the constructor of a Subsystem or Command.

如果使用 Command Framework，可以将此类代码放在 Subsystem 或 Command 的 Constructor 中。

Reading Preferences / 读取 Preferences

```
public void loadPreferences() {
    // Read Preferences for Arm setpoint and kP on entering Teleop
    m_armSetpointDegrees = Preferences.getDouble(Constants.kArmPositionKey, m_armSetpointDegrees);
    if (m_armKp != Preferences.getDouble(Constants.kArmPKey, m_armKp)) {
        m_armKp = Preferences.getDouble(Constants.kArmPKey, m_armKp);
        m_controller.setP(m_armKp);
    }
}
```

Reading a preference is easy. The `getDouble` method takes two parameters, the key to read, and a default value to use in case the preference doesn't exist. There are similar methods for other data types like booleans, ints, and strings.

读取 Preference 很简单。 `getDouble` 方法接收两个参数：要读取的 Key，以及 Preference 不存在时使用的 Default Value。对于 Boolean、int 和 String 等其他数据类型，也有类似方法。

Depending on the data that is stored in preferences, you can use it when you read it, such as the proportional constant above. Or you can store it in a variable and use it later, such as the setpoint, which is used in `teleopPeriodic` below.

根据 Preferences 中存储的数据，可以在读取后立即使用，例如上方的 Proportional Constant；也可以将其存入变量供稍后使用，例如下方 `teleopPeriodic` 中使用的 setpoint。

```

@Override
public void teleopPeriodic() {
    if (m_joystick.getTrigger()) {
        // Here, we run PID control like normal.
        m_arm.reachSetpoint();
    } else {
        // Otherwise, we disable the motor.
        m_arm.stop();
    }
}

```

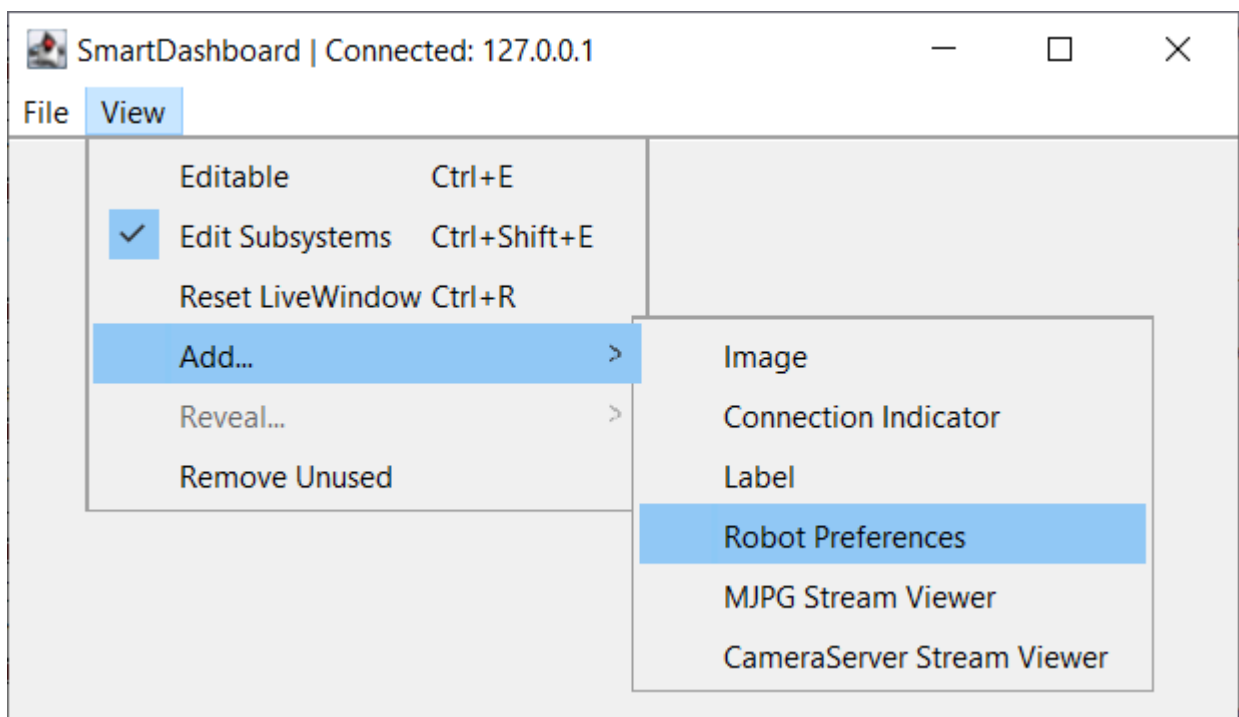
```

/** Run the control loop to reach and maintain the setpoint from the preferences. */
public void reachSetpoint() {
    var pidOutput =
        m_controller.calculate(
            m_encoder.getDistance(), Units.degreesToRadians(m_armSetpointDegrees));
    m_motor.setVoltage(pidOutput);
}

```

Using Preferences in SmartDashboard / 在 SmartDashboard 中使用 Preferences

Displaying Preferences in SmartDashboard / 在 SmartDashboard 中显示 Preferences

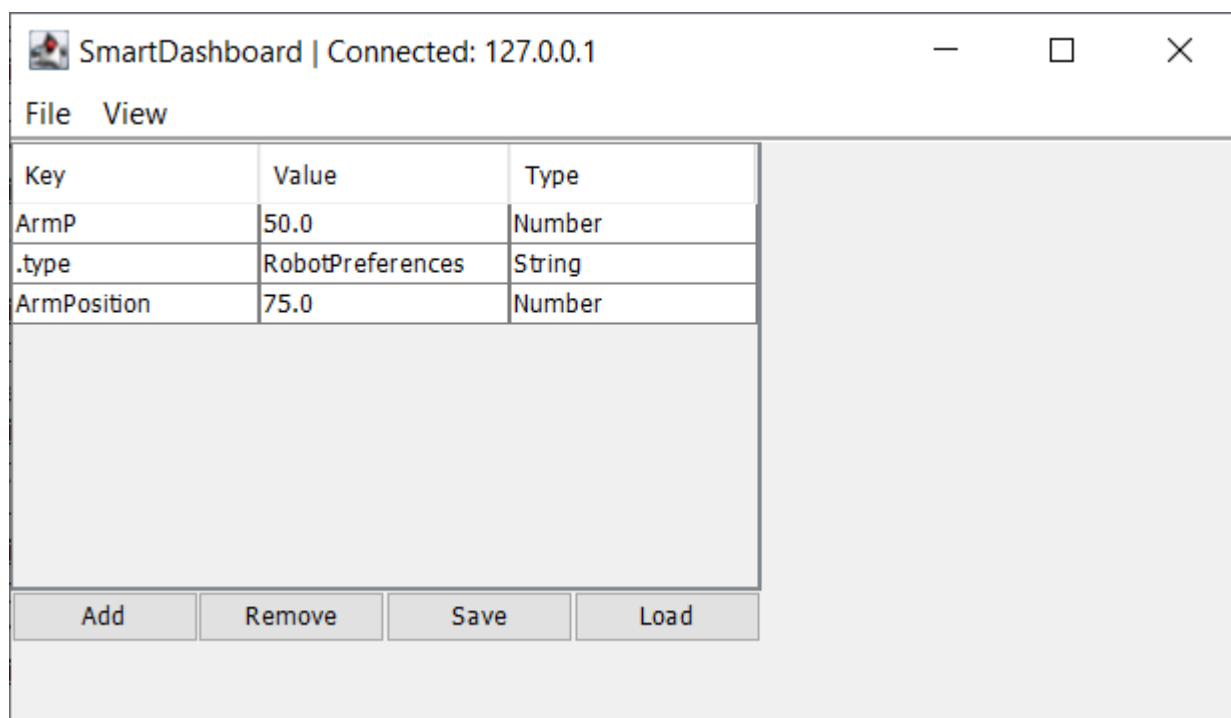


Adding preferences from the Smartdashboard menu / 从 SmartDashboard 菜单添加 Preferences

In the SmartDashboard, the Preferences display can be added to the display by selecting **View** then **Add...** then **Robot Preferences**. This reveals the contents of the preferences file stored in the roboRIO flash memory.

在 SmartDashboard 中，依次选择 View、Add... 和 Robot Preferences，即可添加 Preferences 显示。该显示会呈现存储在 roboRIO Flash Memory 中的 Preferences File 内容。

Editing Preferences in SmartDashboard / 在 SmartDashboard 中编辑 Preferences



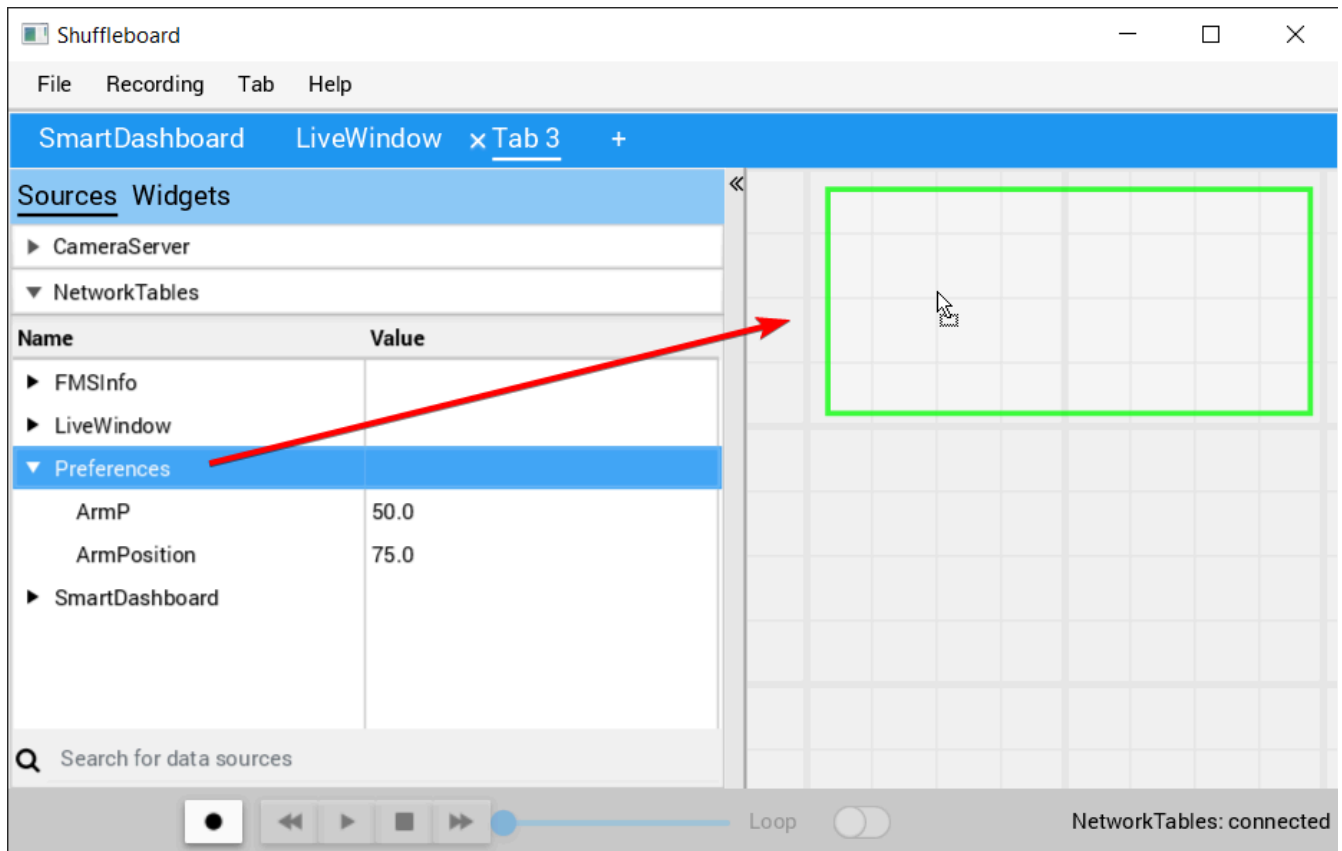
Editing the robot preferences via the SmartDashboard widget. / 通过 SmartDashboard Widget 编辑 Robot Preferences。

The values are shown here with the default values from the code. If the values need to be adjusted they can be edited here and saved.

此处显示代码中的 Default Value。如果需要调整这些数值，可以在此处编辑并保存。

Using Preferences in Shuffleboard / 在 Shuffleboard 中使用 Preferences

Displaying Preferences in Shuffleboard / 在 Shuffleboard 中显示 Preferences

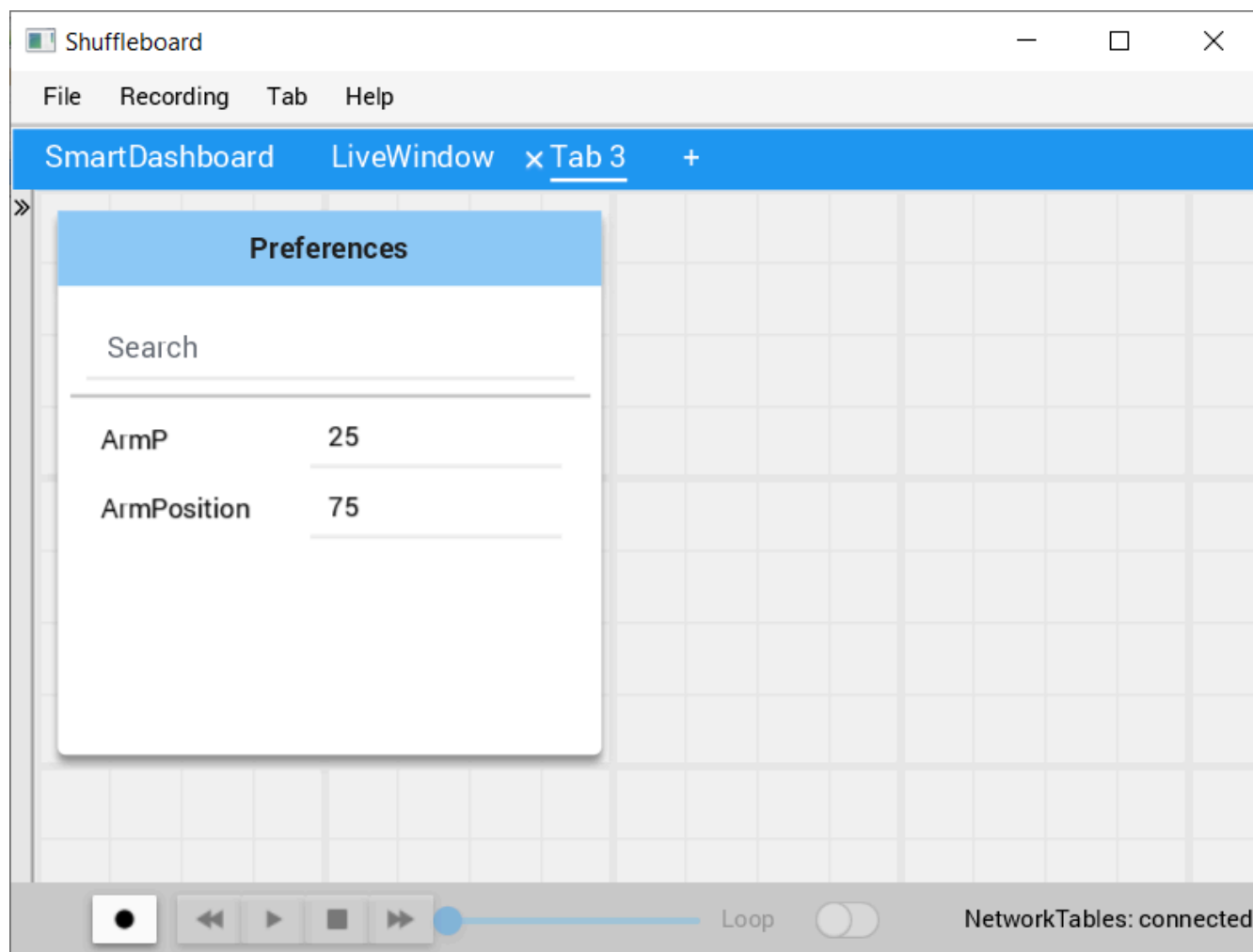


Adding preferences from the sources window in Shuffleboard / 从 Shuffleboard 的 Sources Window 添加 Preferences

In Shuffleboard, the Preferences display can be added to the display by dragging the preferences field from the sources window. This reveals the contents of the preferences file stored in the roboRIO flash memory.

在 Shuffleboard 中，可以从 Sources Window 拖动 Preferences Field，以添加 Preferences 显示。该显示会呈现存储在 roboRIO Flash Memory 中的 Preferences File 内容。

Editing Preferences in Shuffleboard / 在 Shuffleboard 中编辑 Preferences



Editing the robot preferences via the Shuffleboard widget. / 通过 Shuffleboard Widget 编辑 Robot Preferences。

The values are shown here with the default values from the code. If the values need to be adjusted they can be edited here.

此处显示代码中的 Default Value。如果需要调整这些数值，可以在此处编辑。

Robot Project Deploy Directory / Robot Project Deploy Directory

The deploy directory is a part of an FRC project, used to store files that are transferred to the roboRIO during the deployment process. This directory allows teams to include additional resources, such as configuration files, scripts, or other assets, that their robot code may need at runtime.

Deploy Directory 是 FRC Project 的一部分，用于存储部署过程中传输到 roboRIO 的文件。队伍可以在此目录中加入 Robot Code 运行时可能需要的其他资源，例如配置文件、脚本或其他资源文件。

Location of the Deploy Directory / Deploy Directory 的位置

In a standard GradleRIO project, the deploy directory is located under the `src/main` folder. Any files placed in this directory will automatically be copied to the roboRIO when the code is deployed.

在标准 GradleRIO Project 中，Deploy Directory 位于 `src/main` 文件夹下。部署代码时，放入该目录的所有文件都会自动复制到 roboRIO。

How the Deploy Directory Works / Deploy Directory 的工作方式

When you run the `deploy` task, GradleRIO packages the contents of the deploy directory and transfers them to the roboRIO. By default, these files are placed in the `/home/lvuser/deploy` directory on the roboRIO. This ensures that your robot code can access these files during runtime.

运行 `deploy` Task 时，GradleRIO 会打包 Deploy Directory 的内容并将其传输到 roboRIO。默认情况下，这些文件会放入 roboRIO 上的 `/home/lvuser/deploy` 目录，从而确保 Robot Code 能够在运行时访问这些文件。

Example Use Cases / 示例用途

Here are some common examples of how teams use the `deploy` directory:

以下是队伍使用 `deploy` Directory 的一些常见示例：

1. **Configuration Files** Teams may include configuration files (e.g., JSON, YAML, or XML) that define robot-specific settings, such as PID constants, swerve configuration, or autonomous routines. The Preferences class provides another method to store configuration on the roboRIO.

配置文件 队伍可以加入用于定义机器人专用设置的配置文件（例如 JSON、YAML 或 XML），其中可以包含 PID Constant、swerve 配置或 Autonomous Routine。Preferences Class 提供了另一种在 roboRIO 上存储配置的方法。

2. **Path Planning Files** For teams using path planning libraries, trajectory files such as those from Choreo and PathPlanner can be stored in the deploy directory and loaded by the robot code.

路径规划文件 对于使用路径规划库的队伍，可以将 Choreo 和 PathPlanner 等工具生成的轨迹文件存储在 Deploy Directory 中，并由 Robot Code 加载。

Accessing Deploy Files in Code / 在代码中访问 Deploy File

To access files from the deploy directory in your robot code, use the `Filesystem` class (Java). The `getDeployDirectory` method returns the path to the deploy directory. This ensures that your code can locate the files regardless of whether the code is running on the roboRIO or in simulation on your computer.

若要在 Robot Code 中访问 Deploy Directory 中的文件，请使用 `Filesystem` 类 (Java)。`getDeployDirectory` 方法返回 Deploy Directory 的路径。这样，无论代码是在 roboRIO 上运行，还是在计算机上进行 Simulation，都能找到这些文件。

Deleting Unused Deploy Files / 删除不再使用的 Deploy File

By default the deploy directory in your project is transferred to the roboRIO when code is deployed. It is initiated by this section of the `build.gradle` file.

默认情况下，部署代码时，Project 中的 Deploy Directory 会传输到 roboRIO。该过程由 `build.gradle` 文件中的以下部分启动。

```
// Static files artifact
frcStaticFileDeploy(getArtifactTypeClass('FileTreeArtifact')) {
    files = project.fileTree('src/main/deploy')
    directory = '/home/lvuser/deploy'
    deleteOldFiles = false // Change to true to delete files on roboRIO that no
                           // longer exist in deploy directory of this project
}
```

This will overwrite any duplicate files found in the `/home/lvuser/deploy` directory on the RIO and copy over any additional not present there. If `deleteOldFiles` is false it will not remove any files no longer present in the project deploy directory. Changing it to `true` helps prevent programs like Choreo and PathPlanner from getting confused by files that were deleted locally but still exist on the roboRIO.

此操作会覆盖 RIO 上 `/home/lvuser/deploy` 目录中发现的所有同名文件，并复制该目录中尚不存在的其他文件。如果 `deleteOldFiles` 为 false，则不会删除已经不在 Project Deploy Directory 中的文件。将其改为 `true`，有助于防止 Choreo 和 PathPlanner 等程序因为某些文件已在本地删除、但仍保留在 roboRIO 上而发生混淆。

If you want to manage the roboRIO files directly, the FTP documentation provides one method to do so.

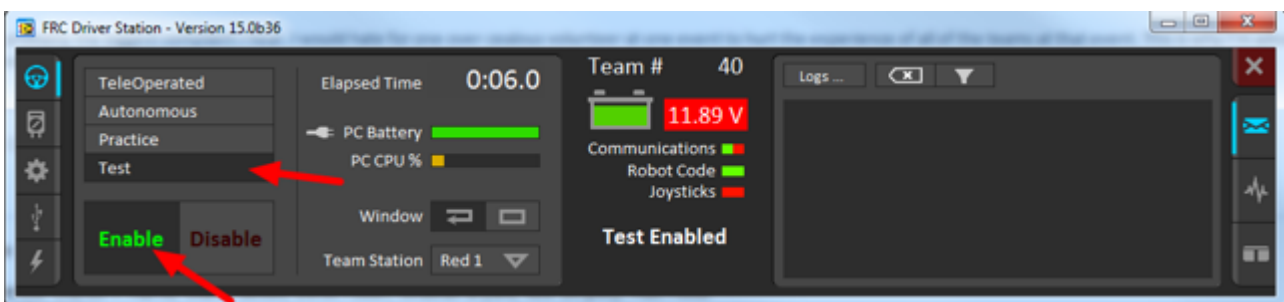
如果希望直接管理 roboRIO 上的文件，可以使用 WPILib 文档中介绍的 FTP 方法。

Using Test Mode / 使用 Test Mode

Test mode is designed to enable programmers to have a place to put code to verify that all systems on the robot are functioning. In each of the robot program templates there is a place to add test code to the robot.

Test Mode 旨在为程序员提供一个放置验证代码的位置，用于确认机器人上的所有系统均正常工作。每种 Robot Program Template 都提供了向机器人添加测试代码的位置。

Enabling Test Mode / 启用 Test Mode



Selecting the "Test" button on the Driver Station and then "Enable". / 在 Driver Station 中选择 “Test” 按钮，然后选择 “Enable”。

Test mode on the robot can be enabled from the Driver Station just like autonomous or teleop. To enable test mode in the Driver Station, select the "Test" button and enable the robot. The test mode code will then run.

与 Autonomous 或 Teleop 一样，可以通过 Driver Station 启用机器人的 Test Mode。若要在 Driver Station 中启用 Test Mode，请选择 “Test” 按钮并启用机器人，随后 Test Mode Code 将开始运行。

Adding Test mode code to your robot code / 向 Robot Code 添加 Test Mode Code

When in test mode, the `testInit` method is run once, and the `testPeriodic` method is run once per tick, in addition to `robotPeriodic`, similar to teleop and autonomous control modes.

进入 Test Mode 后，`testInit` 方法会运行一次；除 `robotPeriodic` 外，`testPeriodic` 方法还会在每个 Tick 运行一次，这与 Teleop 和 Autonomous Control Mode 类似。

Adding test mode can be as painless as calling your already written Teleop methods from Test. Or you can write special code to try out a new feature that is only run in Test mode, before integrating it into your teleop or autonomous code. You could even write code to move all motors and check all sensors to help the pit crew!

添加 Test Mode 可以非常简单，例如在 Test 中调用已经编写好的 Teleop 方法。也可以编写仅在 Test Mode 中运行的专用代码，在将新功能集成到 Teleop 或 Autonomous Code 前先进行试验。甚至可以编写代码驱动所有

电机并检查所有传感器，为 Pit Crew 提供帮助！

LiveWindow in Test Mode / Test Mode 中的 LiveWindow

IMPORTANT: Since 2024, LiveWindow in Test Mode is disabled by default! See [Enabling LiveWindow in Test Mode](#) in the WPILib documentation to enable it.

****重要：**自 2024 年起，Test Mode 中的 LiveWindow 默认处于禁用状态！请参阅 WPILib 文档中的“[Enabling LiveWindow in Test Mode](#)”以将其启用。

With LiveWindow, all actuator outputs can be controlled on the Dashboard and all sensor values can be seen. PID Controllers can also be tuned. The sensors and actuators are added automatically, no code is necessary. See the LiveWindow documentation for more details.

借助 LiveWindow，可以在 Dashboard 上控制所有 Actuator Output，并查看所有 Sensor Value；还可以调整 PID Controller。Sensor 和 Actuator 会自动添加，无需编写代码。有关更多详细信息，请参阅 WPILib 的 LiveWindow 文档。

Reading Stacktraces / 阅读 Stack Trace

An unexpected error has occurred.

发生了意外错误。

When your robot code hits an unexpected error, you will see this message show up in some console output (Driver Station or RioLog). You'll probably also notice your robot abruptly stop, or possibly never move. These unexpected errors are called *unhandled exceptions*.

当 Robot Code 遇到意外错误时，会在某些 Console Output（Driver Station 或 RioLog）中看到此消息。可能还会注意到 Robot 突然停止，或根本没有移动。这些意外错误称为 *Unhandled Exception*。

When an unhandled exception occurs, it means that your code has one or more bugs which need to be fixed.

发生 Unhandled Exception 意味着代码中存在一个或多个需要修复的 Bug。

This article will explore some of the tools and techniques involved in finding and fixing those bugs.

本文将介绍查找和修复这些 Bug 时会用到的一些工具和技巧。

What's a "Stack Trace"? / 什么是 “Stack Trace” ?

The unexpected error has occurred message is a signal that a *stack trace* has been printed out.

unexpected error has occurred 消息表示已经输出了 *Stack Trace*。

In Java, the call stack data structure is used to store information about which function or method is currently being executed.

在 Java 中，Call Stack Data Structure 用于存储当前正在执行哪个 Function 或 Method 的信息。

A *stack trace* prints information about what was on this stack when the unhandled exception occurred. This points you to the lines of code which were running just before the problem happened. While it doesn't always point you to the exact *root cause* of your issue, it's usually the best place to start looking.

Stack Trace 会输出发生 Unhandled Exception 时 Stack 中的内容。这会指向问题发生前正在运行的 Code Line。虽然它不一定总能指向问题的准确 *根本原因*，但通常是开始排查的最佳位置。

What's an "Unhandled Exception"? / 什么是 “Unhandled Exception” ?

An unrecoverable error is any condition which arises where the processor cannot continue executing code. It almost always implies that, even though the code compiled and started running, it no longer makes sense for execution to continue.

Unrecoverable Error 是指 Processor 无法继续执行代码时出现的任何状况。它几乎总是意味着，虽然代码已经 Compile 并开始运行，但继续执行已经没有意义。

In almost all cases, the root cause of an unhandled exception is code that isn't correctly implemented. It almost never implies that any hardware has malfunctioned.

在几乎所有情况下，Unhandled Exception 的根本原因都是代码实现不正确。它几乎从不表示 Hardware 发生故障。

So How Do I Fix My Issue? / 那么如何修复问题?

Read the Stack Trace / 阅读 Stack Trace

To start, search above the `unexpected error has occurred` for the stack trace.

首先，在 `unexpected error has occurred` 上方查找 Stack Trace。

In Java, it should look something like this:

在 Java 中，它应类似如下内容：

```
Error at frc.robot.Robot.robotInit(Robot.java:24): Unhandled exception: java.lang.NullPointerException
    at frc.robot.Robot.robotInit(Robot.java:24)
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:94)
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:335)
    at edu.wpi.first.wpilibj.RobotBase.lambda$startRobot$0(RobotBase.java:387)
    at java.base/java.lang.Thread.run(Thread.java:834)
```

There's a few important things to pick out of here:

这里有几项重要信息需要提取：

- There was an Error
- 发生了 Error
- The error was due to an Unhandled exception
- 该 Error 由 Unhandled exception 引起
- The exception was a `java.lang.NullPointerException`
- Exception 为 `java.lang.NullPointerException`
- The error happened while running line 24 inside of `Robot.java`
- Error 发生在运行 `Robot.java` 第 24 行时
 - `robotInit` was the name of the method executing when the error happened.
 - Error 发生时，正在执行的 Method 名称为 `robotInit`。
- `robotInit` is a function in the `frc.robot.Robot` package (AKA, your team's code)
- `robotInit` 是 `frc.robot.Robot` Package 中的 Function（即 Team Code）

- `robotInit` was called from a number of functions from the `edu.wpi.first.wpilibj` package (AKA, the WPILib libraries)
- `robotInit` 由 `edu.wpi.first.wpilibj` Package 中的多个 Function 调用（即 WPILib Library）

The list of indented lines starting with the word `at` represent the state of the *stack* at the time the error happened. Each line represents one method, which was *called by* the method right below it.

以 `at` 开头的缩进行列表表示 Error 发生时 *Stack* 的状态。每一行表示一个 Method，该 Method 是由其正下方的 Method 调用的。

For example, If the error happened deep inside your codebase, you might see more entries on the stack:

例如，如果 Error 发生在 Codebase 深处，Stack 中可能会看到更多 Entry：

```
Error at frc.robot.Robot.buggyMethod(TooManyBugs.java:1138): Unhandled exception:
java.lang.NullPointerException
    at frc.robot.Robot.buggyMethod(TooManyBugs.java:1138)
    at frc.robot.Robot.barInit(Bar.java:21)
    at frc.robot.Robot.fooInit(Foo.java:34)
    at frc.robot.Robot.robotInit(Robot.java:24)
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:94)
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:335)
    at edu.wpi.first.wpilibj.RobotBase.lambda$startRobot$0(RobotBase.java:387)
    at java.base/java.lang.Thread.run(Thread.java:834)
```

In this case: `robotInit` called `fooInit`, which in turn called `barInit`, which in turn called `buggyMethod`. Then, during the execution of `buggyMethod`, the `NullPointerException` occurred.

在此情况下，`robotInit` 调用了 `fooInit`，后者又调用了 `barInit`，`barInit` 又调用了 `buggyMethod`。随后，在执行 `buggyMethod` 期间发生了 `NullPointerException`。

Perform Code Analysis / 执行代码分析

Once you've found the stack trace, and found the lines of code which are triggering the unhandled exception, you can start the process of determining root cause.

找到 Stack Trace 以及触发 Unhandled Exception 的 Code Line 后，即可开始确定根本原因。

Often, just looking in (or near) the problematic location in code will be fruitful. You may notice things you forgot, or lines which don't match an example you're referencing.

通常，只需查看代码中的问题位置或其附近，就能有所收获。可能会发现遗漏的事项，或与参考示例不匹配的代码行。

NOTE: Developers who have lots of experience working with code will often have more luck looking at code than newer folks. That's ok, don't be discouraged! The experience will come with time.

说明： 与经验较少的开发者相比，拥有丰富代码工作经验的 Developer 通常更容易通过查看代码找到问题。这很正常，请不要气馁；经验会随时间积累。

A key strategy for analyzing code is to ask the following questions:

分析代码的一项关键策略是提出以下问题：

- When was the last time the code "worked" (i.e., didn't have this particular error)?
- 代码上一次“正常工作”（即没有这个特定 Error）是什么时候？
- What has changed in the code between the last working version, and now?
- 从上一个正常工作的 Version 到现在，代码发生了哪些变化？

Frequent testing and careful code changes help make this particular strategy more effective.

频繁进行测试并谨慎更改代码，有助于提高该策略的有效性。

Run the Single Step Debugger / 运行 Single Step Debugger

Sometimes, just looking at code isn't enough to spot the issue. The single-step debugger is a great option in this case - it allows you to inspect the series of events leading up to the unhandled exception.

有时，仅查看代码不足以发现问题。在这种情况下，Single Step Debugger 是很好的选择——它允许检查导致 Unhandled Exception 的一系列事件。

Search for More Information / 搜索更多信息

Google is a phenomenal resource for understanding the root cause of errors. Searches involving the programming language and the name of the exception will often yield good results on more explanations for what the error means, how it comes about, and potential fixes.

Google 是理解 Error 根本原因的出色资源。使用 Programming Language 和 Exception Name 进行搜索，通常能找到关于 Error 含义、发生原因和潜在修复方法的更多良好说明。

Seeking Outside Help / 寻求外部帮助

If all else fails, you can seek out advice and help from others (both in-person and online). When working with folks who aren't familiar with your codebase, it's very important to provide the following information:

如果其他方法均无效，可以向他人寻求建议和帮助，无论是线下还是线上。与不熟悉 Codebase 的人合作时，提供以下信息非常重要：

- Access to your source code, (EX: on github.com)
- Source Code 的访问方式（例如：放在 github.com 上）
- The **full text** of the error, including the full stack trace.
- Error 的**完整文本**，包括完整 Stack Trace。

Common Examples & Patterns / 常见示例与 Pattern

There are a number of common issues which result in runtime exceptions.

有许多常见问题会导致 Runtime Exception。

Null Pointers and References / Null Pointer 与 Reference

Java has the concept of "null" - it uses it to indicate something which has not yet been initialized, and does not refer to anything meaningful.

Java 具有 “null” 概念，用它表示某项内容尚未 Initialization，并且没有引用任何有意义的对象。

Manipulating a "null" reference will produce a runtime error.

操作 “null” Reference 会产生 Runtime Error。

For example, consider the following code:

例如，请看以下代码：

```
PWMSparkMax armMotorCtrl;  
@Override  
public void robotInit() {  
    armMotorCtrl.setInverted(true);  
}
```

When run, you'll see output that looks like this:

运行时，会看到类似以下内容的 Output：

```
***** Robot program starting *****  
Error at frc.robot.Robot.robotInit(Robot.java:23): Unhandled exception: java.lang.NullPointerException  
    at frc.robot.Robot.robotInit(Robot.java:23)  
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:107)  
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:373)  
    at edu.wpi.first.wpilibj.RobotBase.startRobot(RobotBase.java:463)  
    at frc.robot.Main.main(Main.java:23)  
Warning at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:388): The robot program quit  
unexpectedly. This is usually due to a code error.  
    The above stacktrace can help determine where the error occurred.  
    See https://wpilib.org/stacktrace for more information.  
Error at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:395): The startCompetition() method (or  
methods called by it) should have handled the exception above.
```

Reading the stack trace, you can see that the issue happened inside of the `robotInit()` function, on line 23, and the exception involved "Null Pointer".

阅读 Stack Trace 可以看到，问题发生在 `robotInit()` Function 内的第 23 行，并且 Exception 与 “Null Pointer” 有关。

By going to line 23, you can see there is only one thing which could be null - `armMotorCtrl`. Looking further up, you can see that the `armMotorCtrl` object is declared, but never instantiated.

转到第 23 行可以看到，只有一项内容可能为 null，即 `armMotorCtrl`。继续向上查看，可以发现 `armMotorCtrl` Object 已声明，但从未实例化。

Alternatively, you can step through lines of code with the single step debugger, and stop when you hit line 23. Inspecting the `armMotorCtrl` object at that point would show that it is null.

也可以使用 Single Step Debugger 逐行执行代码，并在到达第 23 行时停止。此时检查 `armMotorCtrl` Object，会看到它为 null。

Fixing Null Object Issues / 修复 Null Object 问题

Generally, you will want to ensure each reference has been initialized before using it. In this case, there is a missing line of code to instantiate the `armMotorCtrl` before calling the `setInverted()` method.

通常，应确保每个 Reference 在使用前都已经 Initialization。在此情况下，缺少一行用于在调用 `setInverted()` Method 前实例化 `armMotorCtrl` 的代码。

A functional implementation could look like this:

可以正常工作的实现如下所示：

```
PWMSparkMax armMotorCtrl;
@Override
public void robotInit() {
    armMotorCtrl = new PWMSparkMax(0);
    armMotorCtrl.setInverted(true);
}
```

Divide by Zero / 除以零

It is not generally possible to divide an integer by zero, and expect reasonable results. Most processors (including the roboRIO) will raise an Unhandled Exception.

通常无法将 Integer 除以零并期望得到合理结果。大多数 Processor（包括 roboRIO）都会引发 Unhandled Exception。

For example, consider the following code:

例如，请看以下代码：

```
int armLengthRatio;
int elbowToWrist_in = 39;
int shoulderToElbow_in = 0; //TODO
@Override
public void robotInit() {
    armLengthRatio = elbowToWrist_in / shoulderToElbow_in;
}
```

When run, you'll see output that looks like this:

运行时，会看到类似以下内容的 Output：

```

***** Robot program starting *****
Error at frc.robot.Robot.robotInit(Robot.java:24): Unhandled exception: java.lang.ArithmeticException: /
by zero
    at frc.robot.Robot.robotInit(Robot.java:24)
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:107)
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:373)
    at edu.wpi.first.wpilibj.RobotBase.startRobot(RobotBase.java:463)
    at frc.robot.Main.main(Main.java:23)
Warning at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:388): The robot program quit
unexpectedly. This is usually due to a code error.
    The above stacktrace can help determine where the error occurred.
    See https://wpilib.org/stacktrace for more information.
Error at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:395): The startCompetition() method (or
methods called by it) should have handled the exception above.

```

Looking at the stack trace, we can see a `java.lang.ArithmeticException: / by zero` exception has occurred on line 24. If you look at the two variables which are used on the right-hand side of the `=` operator, you might notice one of them has been initialized to zero. Looks like someone forgot to update it! Furthermore, the zero-value variable is used in the denominator of a division operation. Hence, the divide by zero error happens.

查看 Stack Trace 可以看到，第 24 行发生了 `java.lang.ArithmeticException: / by zero` Exception。查看 `=` Operator 右侧使用的两个 Variable，可能会注意到其中一个被 Initialization 为零。看起来有人忘记更新它了。此外，这个 Value 为零的 Variable 用作 Division Operation 的 Denominator，因此发生了 Divide by Zero Error。

Alternatively, by running the single-step debugger and stopping on line 24, you could inspect the value of all variables to discover `shoulderToElbow_in` has a value of 0.

也可以运行 Single Step Debugger 并在第 24 行停止，检查所有 Variable 的 Value，从而发现 `shoulderToElbow_in` 的 Value 为 0。

Fixing Divide By Zero Issues / 修复除以零问题

Divide By Zero issues can be fixed in a number of ways. It's important to start by thinking about what a zero in the denominator of your calculation *means*. Is it plausible? Why did it happen in the particular case you saw?

可以通过多种方式修复 Divide By Zero 问题。首先必须考虑计算的 Denominator 为零意味着什么。这是否合理？为什么会在所看到的具体情况下发生？

Sometimes, you just need to use a different number other than 0.

有时，只需使用 0 以外的其他数字。

A functional implementation could look like this:

可以正常工作的实现如下所示：

```
int armLengthRatio;
int elbowToWrist_in = 39;
int shoulderToElbow_in = 3;
@Override
public void robotInit() {
    armLengthRatio = elbowToWrist_in / shoulderToElbow_in;
}
```

Alternatively, if zero *is* a valid value, adding `if/else` statements around the calculation can help you define alternate behavior to avoid making the processor perform a division by zero.

如果零确实是 Valid Value，也可以在计算周围添加 `if/else` Statement，帮助定义替代行为，避免让 Processor 执行除以零操作。

Finally, changing variable types to be `float` or `double` can help you get around the issue - floating-point numbers have special values like `NaN` to represent the results of a divide-by-zero operation. However, you may still have to handle this in code which consumes that calculation's value.

最后，将 Variable Type 更改为 `float` 或 `double` 可以帮助绕过此问题——Floating-point Number 具有 `NaN` 等 Special Value，用于表示 Divide-by-zero Operation 的结果。不过，使用该计算结果的代码仍可能需要处理这种情况。

HAL Resource Already Allocated / HAL Resource 已分配

A very common FRC-specific error occurs when the code attempts to put two hardware-related entities on the same HAL resource (usually, roboRIO IO pin).

当代码尝试将两个 Hardware-related Entity 放在同一个 HAL Resource（通常是 roboRIO IO Pin）上时，会发生一种非常常见的 FRC-specific Error。

For example, consider the following code:

例如，请看以下代码：

```
PWMSparkMax leftFrontMotor;
PWMSparkMax leftRearMotor;
@Override
public void robotInit() {
    leftFrontMotor = new PWMSparkMax(0);
    leftRearMotor = new PWMSparkMax(0);
}
```

When run, you'll see output that looks like this:

运行时，会看到类似以下内容的 Output：

```

***** Robot program starting *****
Error at frc.robot.Robot.robotInit(Robot.java:25): Unhandled exception:
edu.wpi.first.hal.util.AllocationException: Code: -1029
PWM or DIO 0 previously allocated.
Location of the previous allocation:
    at frc.robot.Robot.robotInit(Robot.java:24)
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:107)
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:373)
    at edu.wpi.first.wpilibj.RobotBase.startRobot(RobotBase.java:463)
    at frc.robot.Main.main(Main.java:23)
Location of the current allocation:
    at edu.wpi.first.hal.PWMJNI.initializePWMPort(Native Method)
    at edu.wpi.first.wpilibj.PWM.<init>(PWM.java:66)
    at edu.wpi.first.wpilibj.motorcontrol.PWMMotorController.<init>(PWMMotorController.java:27)
    at edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax.<init>(PWMSparkMax.java:35)
    at frc.robot.Robot.robotInit(Robot.java:25)
    at edu.wpi.first.wpilibj.TimedRobot.startCompetition(TimedRobot.java:107)
    at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:373)
    at edu.wpi.first.wpilibj.RobotBase.startRobot(RobotBase.java:463)
    at frc.robot.Main.main(Main.java:23)
Warning at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:388): The robot program quit
unexpectedly. This is usually due to a code error.
    The above stacktrace can help determine where the error occurred.
    See https://wpilib.org/stacktrace for more information.
Error at edu.wpi.first.wpilibj.RobotBase.runRobot(RobotBase.java:395): The startCompetition() method (or
methods called by it) should have handled the exception above.

```

This stack trace shows that a `edu.wpi.first.hal.util.AllocationException` has occurred. It also gives the helpful message: PWM or DIO 0 previously allocated..

该 Stack Trace 表明发生了 `edu.wpi.first.hal.util.AllocationException`，并给出了有帮助的消息：PWM or DIO 0 previously allocated.。

Looking at our stack trace, we see two stack traces. The first stack trace shows that the first allocation occurred in `Robot.java:25`. The second stack trace shows that the error *actually* happened deep within WPILib. However, we should start by looking in our own code. Halfway through the stack trace, you can find a reference to the last line of the team's robot code that called into WPILib: `Robot.java:25`.

查看 Stack Trace，可以看到两个 Stack Trace。第一个 Stack Trace 表明第一次 Allocation 发生在 `Robot.java:25`。第二个 Stack Trace 表明 Error 实际发生在 WPILib 深处。不过，应从自己的代码开始查看。在 Stack Trace 中间，可以找到 Team Robot Code 调用 WPILib 的最后一行：`Robot.java:25`。

Taking a peek at the code, we see line 24 is where the first motor controller is declared and line 25 is where the second motor controller is declared. We can also note that *both* motor controllers are assigned to PWM output 0. This doesn't make logical sense, and isn't physically possible. Therefore, WPILib purposely generates a custom error message and exception to alert the software developers of a non-achievable hardware configuration.

查看代码可以发现，第 24 行声明了第一个 Motor Controller，第 25 行声明了第二个 Motor Controller。还可以注意到，两个 Motor Controller 都分配给 PWM Output 0。这在逻辑上不合理，在物理上也不可能。因此，WPILib 会有意生成 Custom Error Message 和 Exception，提醒 Software Developer 该 Hardware Configuration 无法实现。

Fixing HAL Resource Already Allocated Issues / 修复 HAL Resource Already Allocated 问题

HAL: Resource already allocated are some of the most straightforward errors to fix. Just spend a bit of time looking at the electrical wiring on the robot, and compare that to what's in code.

HAL: Resource already allocated 属于最容易修复的 Error。只需花些时间查看 Robot Electrical Wiring，并与代码中的内容进行比较。

In the example, the left motor controllers are plugged into PWM ports 0 and 1. Therefore, corrected code would look like this:

在该示例中，左侧 Motor Controller 分别插入 PWM Port 0 和 1。因此，修正后的代码如下所示：

```
PWMSparkMax leftFrontMotor;
PWMSparkMax leftRearMotor;
@Override
public void robotInit() {
    leftFrontMotor = new PWMSparkMax(0);
    leftRearMotor = new PWMSparkMax(1);
}
```

Another way this error can manifest for teams using the Command framework is if a Subsystem is instantiated twice, which causes the hardware in the subsystem to be instantiated twice.

对于使用 Command Framework 的 Team，如果某个 Subsystem 被实例化两次，导致该 Subsystem 中的 Hardware 也被实例化两次，也会出现此 Error。

gradlew is not recognized... / gradlew 无法识别……

gradlew is not recognized as an internal or external command is a common error that can occur when the project or directory that you are currently in does not contain a gradlew file. This usually occurs when you open the wrong directory.

当当前所在的 Project 或 Directory 中不包含 gradlew 文件时，可能会出现常见 Error: gradlew is not recognized as an internal or external command。这通常是因为打开了错误的 Directory。

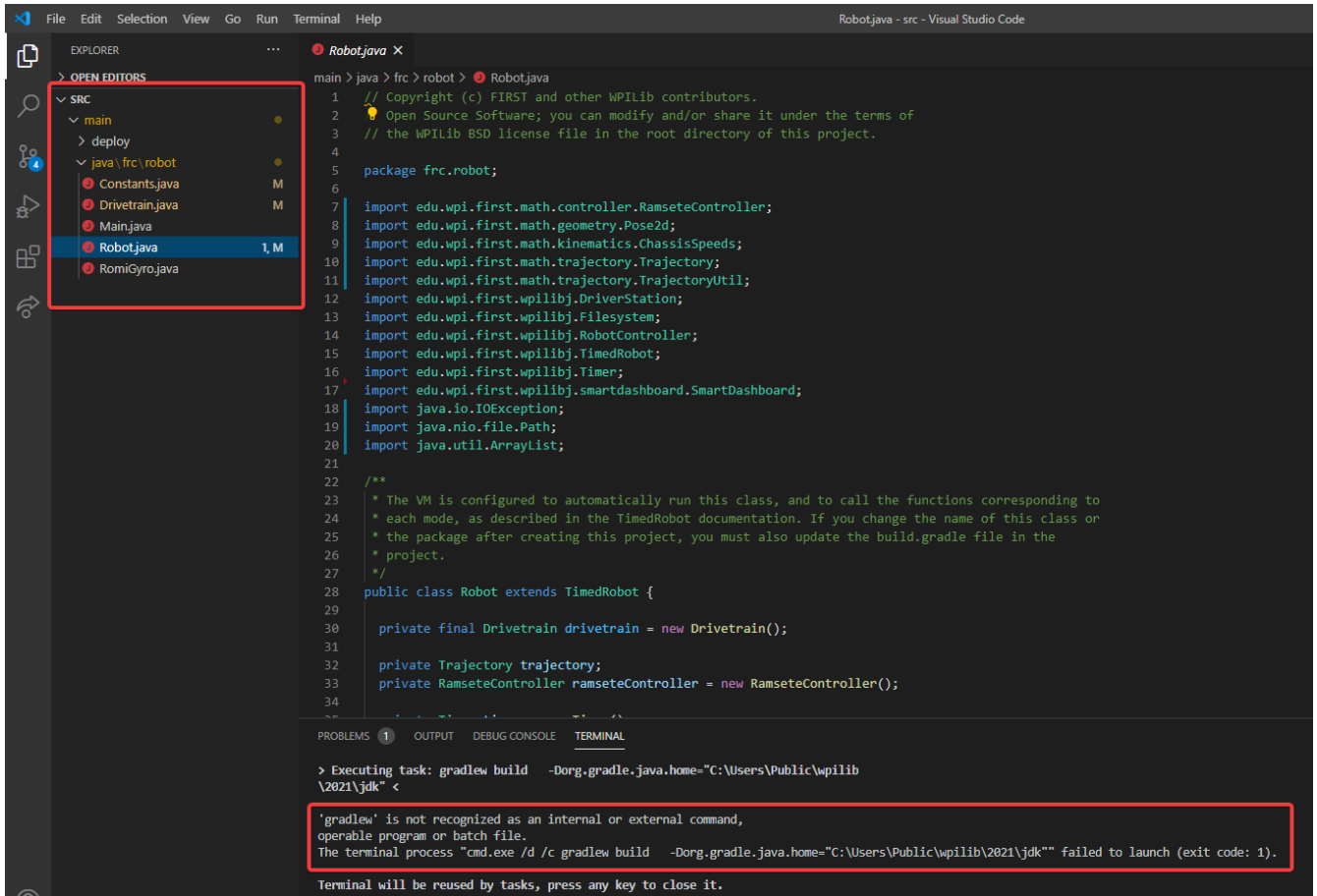


Image containing that the left-hand VS Code sidebar does not contain gradlew / 图中左侧 VS Code Sidebar 不包含 gradlew

In the above screenshot, you can see that the left-hand sidebar does not contain many files. At a minimum, VS Code needs a couple of files to properly build and deploy your project.

在上方 Screenshot 中，可以看到左侧 Sidebar 中没有很多文件。要正确 Build 和 Deploy Project，VS Code 至少需要以下几个文件。

- gradlew
- gradlew
- build.gradle
- build.gradle
- gradlew.bat
- gradlew.bat

If you do not see any one of the above files in your project directory, then you have two possible causes.

如果 Project Directory 中缺少上述任意一个文件，则可能有两个原因。

- A corrupt or bad project.
- Project 已损坏或存在问题。
- You are in the wrong directory.
- 当前位于错误的 Directory。

Fixing gradlew is not recognized... / 修复 gradlew 无法识别问题……

`gradlew is not recognized...` is a fairly easy problem to fix. First identify the problem source:

`gradlew is not recognized...` 是相对容易修复的问题。首先确定问题来源：

Are you in the wrong directory?

是否位于错误的 Directory?

- Verify that the project directory is the correct directory and open this.
- 确认 Project Directory 正确，并打开该 Directory。

Is your project missing essential files?

Project 是否缺少必要文件?

- This issue is more complex to solve. The recommended solution is to recreate your project and manually copy necessary code in.
- 此问题的解决方式更复杂。建议重新创建 Project，并手动复制所需代码。

Treating Functions as Data / 将函数作为数据处理

Regardless of programming language, one of the first things anyone learns to do when programming a computer is to write a function (also known as a "method" or a "subroutine"). Functions are a fundamental part of organized code - writing functions lets us avoid duplicating the same piece of code over and over again. Instead of writing duplicated sections of code, we call a single function that contains the code we want to execute from multiple places (provided we named the function well, the function name is also easier to read than the code itself!). If the section of code needs some additional information about its surrounding context to run, we pass those to the function as "parameters", and if it needs to yield something back to the rest of the code once it finishes, we call that a "return value" (together, the parameters and return value are called the function's "signature");

无论使用哪种 Programming Language，任何人在学习 Computer Programming 时，最先掌握的内容之一都是编写函数（也称为“Method”或“Subroutine”）。Function 是组织良好代码的基本组成部分——编写 Function 可以避免一遍又一遍地复制同一段代码。我们不必编写重复的 Code Section，而可以从多个位置调用包含所需执行代码的同一个 Function（如果 Function 命名良好，Function Name 也比代码本身更易读）。如果 Code Section 运行时需要其周围 Context 的额外信息，就将这些信息作为“Parameter”传递给 Function；如果它在完成后需要向其余代码返回某些内容，就将其称为“Return Value”（Parameter 与 Return Value 合称 Function 的“Signature”）。

Sometimes, we need to pass functions from one part of the code to another part of the code. This might seem like a strange concept, if we're used to thinking of functions as part of a class definition rather than objects in their own right. But at a basic level, functions are just data - in the same way we can store an `integer` or a `double` as a variable and pass it around our program, we can do the same thing with a function. A variable whose value is a function is called a "functional interface" in Java.

有时，需要将 Function 从代码的一部分传递到另一部分。如果习惯于将 Function 视为 Class Definition 的组成部分，而不是独立的 Object，这个概念可能显得奇怪。但从基本层面看，Function 只是 Data——就像可以将 `integer` 或 `double` 存入 Variable 并在 Program 中传递一样，也可以用相同方式传递 Function。在 Java 中，值为 Function 的 Variable 称为“Functional Interface”。

Why Would We Want to Treat Functions as Data? / 为什么要将 Function 作为 Data 处理？

Typically, code that calls a function is coupled to (depends on) the definition of the function. While this occurs all the time, it becomes problematic when the code *calling* the function (for example, WPILib) is developed independently and without direct knowledge of the code that *defines* the function (for example, code from an FRC team). Sometimes we solve this challenge through the use of class interfaces, which define collections of data and functions that are meant to be used together. However, often we really only have a dependency on a *single function*, rather than on an *entire class*.

通常，调用 Function 的代码与 Function Definition 耦合（依赖于它）。虽然这种情况非常常见，但当调用 Function 的代码（例如 WPILib）独立开发，并且并不直接了解定义 Function 的代码（例如 FRC Team 编写的

代码) 时, 就会产生问题。有时, 可以使用 Class Interface 解决这一难题; Class Interface 定义了应配合使用的一组 Data 和 Function。不过, 我们真正依赖的往往只是单个 *Function*, 而不是整个 *Class*。

For example, WPILib offers several ways for users to execute certain code whenever a joystick button is pressed - one of the easiest and cleanest ways to do this is to allow the user to *pass a function* to one of the WPILib joystick methods. This way, the user only has to write the code that deals with the interesting and team-specific things (e.g., "move my robot arm") and not the boring, error-prone, and universal thing ("properly read button inputs from a standard joystick").

例如, WPILib 提供了多种方式, 让 User 在每次按下 Joystick Button 时执行特定代码——其中最简单、最清晰的方式之一, 是允许 User 向某个 WPILib Joystick Method 传递 *Function*。这样, User 只需编写用于处理有趣且与队伍有关事项的代码 (例如 “移动 Robot Arm”), 而不必编写枯燥、容易出错且具有通用性的部分 (“正确读取标准 Joystick 的 Button Input”)。

For another example, the Command-based framework is built on *Command* objects that refer to methods defined on various *Subsystem* classes. Many of the included *Command* types (such as *InstantCommand* and *RunCommand*) work with *any* function - not just functions associated with a single *Subsystem*. To support building commands generically, we need to support passing functions from a *Subsystem* (which interacts with the hardware) to a *Command* (which interacts with the scheduler).

另一个示例是, Command-Based Framework 建立在 *Command Object* 之上, 这些 *Object* 引用各个 *Subsystem Class* 中定义的 *Method*。内置的许多 *Command Type* (例如 *InstantCommand* 和 *RunCommand*) 都可以使用任意 *Function*, 而不只是与单个 *Subsystem* 关联的 *Function*。为了支持以通用方式构建 *Command*, 需要支持将 *Function* 从与 *Hardware* 交互的 *Subsystem* 传递给与 *Scheduler* 交互的 *Command*。

In these cases, we want to be able to pass a single function as a piece of data, as if it were a variable - it doesn't make sense to ask the user to provide an entire class, when we really just want them to give us a single appropriately-shaped function.

在这些情况下, 希望能够像传递 *Variable* 一样, 将单个 *Function* 作为一项 *Data* 传递——如果真正需要的只是一个 *Signature* 合适的 *Function*, 要求 User 提供整个 *Class* 并不合理。

It's important that *passing* a function is not the same as *calling* a function. When we call a function, we execute the code inside of it and either receive a return value, cause some side-effects elsewhere in the code, or both. When we *pass* a function, nothing in particular happens *immediately*. Instead, by passing the function we are allowing some *other* code to call the function *in the future*. Seeing the name of a function in code does not always mean that the code in the function is being run!

必须注意, 传递 *Function* 与 调用 *Function* 并不相同。调用 *Function* 时, 会执行其中的代码, 并接收 *Return Value*、在代码其他位置产生 *Side Effect*, 或同时发生这两种情况。传递 *Function* 时, 不会立即发生任何特定操作。相反, 传递 *Function* 是在允许某些其他代码在将来调用该 *Function*。在代码中看到 *Function Name*, 并不一定意味着该 *Function* 中的代码正在运行。

Inside of code that passes a function, we will see some syntax that either refers to the name of an existing function in a special way, or else defines a new function to be passed inside of the call expression. The specific syntax needed (and the rules around it) depends on which programming language we are using.

在传递 *Function* 的代码中, 会看到某些 *Syntax*: 它们或者以特殊方式引用现有 *Function Name*, 或者在 *Call Expression* 内定义一个要传递的新 *Function*。所需的具体 *Syntax* 及其规则取决于所使用的 *Programming Language*。

Treating Functions as Data in Java / 在 Java 中将 Function 作为 Data 处理

Java represents functions-as-data as instances of functional interfaces. A "functional interface" is a special kind of class that has only a single method - since Java was originally designed strictly for object-oriented programming, it has no way of representing a single function detached from a class. Instead, it defines a particular group of classes that *only* represent single functions. Each type of function signature has its own functional interface, which is an interface with a single function definition of that signature.

Java 使用 Functional Interface 的 Instance 表示作为 Data 的 Function。“Functional Interface”是一种只有一个 Method 的特殊 Interface——由于 Java 最初严格按照 Object-Oriented Programming 进行设计，它无法表示脱离 Class 的单个 Function。Java 转而定义了一组专门用于表示单个 Function 的 Interface。每种 Function Signature 都有对应的 Functional Interface，也就是包含一个具有该 Signature 的 Function Definition 的 Interface。

This might sound complicated, but in the context of WPILib we don't really need to worry much about using the functional interfaces themselves - the code that does that is internal to WPILib. Instead, all we need to know is how to pass a function that we've written to a method that takes a functional interface as a parameter. For a simple example, consider the signature of `Commands.runOnce` (which creates an `InstantCommand` that, when scheduled, runs the given function once and then terminates):

这听起来可能很复杂，但在 WPILib Context 中，并不需要过多关注如何直接使用 Functional Interface——相关代码位于 WPILib 内部。只需了解如何将已编写的 Function 传递给以 Functional Interface 作为 Parameter 的 Method。作为简单示例，请看 `Commands.runOnce` 的 Signature（它会创建一个 `InstantCommand`；该 Command 在被 Schedule 时运行给定 Function 一次，然后终止）：

NOTE: The `requirements` parameter is explained in the Command-based documentation, and will not be discussed here.

说明： Command-Based Documentation 对 `requirements` Parameter 进行了说明，此处不再讨论。

```
public static Command runOnce(Runnable action, Subsystem... requirements)
```

`runOnce` expects us to give it a `Runnable` parameter (named `action`). A `Runnable` is the Java term for a function that takes no parameters and returns no value. When we call `runOnce`, we need to give it a function with no parameters and no return value. There are two ways to do this: we can refer to some existing function using a "method reference", or we can define the function we want inline using a "lambda expression".

`runOnce` 要求提供一个名为 `action` 的 `Runnable` Parameter。`Runnable` 是 Java 中对无 Parameter 且无 Return Value Function 的称呼。调用 `runOnce` 时，需要向它提供一个无 Parameter 且无 Return Value 的 Function。有两种方式可以做到这一点：可以使用“Method Reference”引用某个现有 Function，也可以使用“Lambda Expression”以内联方式定义所需 Function。

Method References / Method Reference

A method reference lets us pass an already-existing function as our `Runnable`:

Method Reference 可以将已经存在的 Function 作为 `Runnable` 传递：

```
// Create an InstantCommand that runs the `resetEncoders` method of the `drivetrain` object
Command disableCommand = runOnce(drivetrain::resetEncoders, drivetrain);
```

The expression `drivetrain::resetEncoders` is a reference to the `resetEncoders` method of the `drivetrain` object. It is not a method *call* - this line of code does not *itself* reset the encoders of the drivetrain. Instead, it returns a `Command` that will do so *when it is scheduled*.

Expression `drivetrain::resetEncoders` 是对 `drivetrain` Object 的 `resetEncoders` Method 的引用。它不是 Method *Call*——这行代码本身不会 Reset Drivetrain Encoder。相反，它返回一个 `Command`，该 `Command` 会在被 *Schedule* 时执行此操作。

Remember that in order for this to work, `resetEncoders` must be a `Runnable` - that is, it must take no parameters and return no value. So, its signature must look like this:

请记住，要使其正常工作，`resetEncoders` 必须符合 `Runnable`——也就是说，它必须无 Parameter 且无 Return Value。因此，其 Signature 必须如下所示：

```
// void because it returns no parameters, and has an empty parameter list
public void resetEncoders()
```

If the function signature does not match this, Java will not be able to interpret the method reference as a `Runnable` and the code will not compile. Note that all we need to do is make sure that the signature matches the signature of the single method in the `Runnable` functional interface - we don't need to *explicitly* name it as a `Runnable`.

如果 Function Signature 与此不匹配，Java 将无法把 Method Reference 解释为 `Runnable`，代码也将无法 Compile。请注意，只需确保该 Signature 与 `Runnable` Functional Interface 中唯一 Method 的 Signature 匹配，而无需显式将其声明为 `Runnable`。

Lambda Expressions in Java / Java 中的 Lambda Expression

If we do not already have a named function that does what we want, we can define a function "inline" - that means, right inside of the call to `runOnce`! We do this by writing our function with a special syntax that uses an "arrow" symbol to link the argument list to the function body:

如果尚不存在能够完成所需操作的 Named Function，可以“Inline”定义 Function——也就是直接在 `runOnce` Call 内定义。为此，需要使用一种特殊 Syntax 编写 Function，该 Syntax 使用“Arrow”Symbol 将 Argument List 与 Function Body 连接起来：

```
// Create an InstantCommand that runs the drive forward at half speed
Command driveHalfSpeed = runOnce(() -> { drivetrain.arcadeDrive(0.5, 0.0); }, drivetrain);
```

Java calls `() -> { drivetrain.arcadeDrive(0.5, 0.0); }` a "lambda expression"; it may be less-confusingly called an "arrow function", "inline function", or "anonymous function" (because it has no name). While this may look a bit funky, it is just another way of writing a function - the parentheses before the arrow are the function's argument list, and the code contained in the brackets is the function body. The "lambda expression" here represents a function that calls `drivetrain.arcadeDrive` with a specific set of parameters - note again that this does not *call* the function, but merely defines it and passes it to the `Command` to be run later when the `Command` is scheduled.

Java 将 `() -> { drivetrain.arcadeDrive(0.5, 0.0); }` 称为 “Lambda Expression”；也可以用较不容易混淆的名称称呼它，例如 “Arrow Function” “Inline Function” 或 “Anonymous Function”（因为它没有名称）。虽然这看起来有些特别，但它只是另一种编写 Function 的方式——Arrow 前的 Parentheses 是 Function Argument List, Braces 中的代码是 Function Body。此处的 Lambda Expression 表示一个使用特定 Parameter 调用 `drivetrain.arcadeDrive` 的 Function——再次注意，这并不会调用该 Function，而只是定义它并将其传递给 `Command`，以便稍后在 `Command` 被 Schedule 时运行。

As with method references, we do not need to *explicitly* name the lambda expression as a `Runnable` - Java can infer that our lambda expression is a `Runnable` so long as its signature matches that of the single method in the `Runnable` interface. Accordingly, our lambda takes no arguments and has no return statement - if it did not match the `Runnable` contract, our code would fail to compile.

与 Method Reference 一样，无需显式将 Lambda Expression 声明为 `Runnable`——只要它的 Signature 与 `Runnable` Interface 中唯一 Method 的 Signature 匹配，Java 就能推断该 Lambda Expression 是 `Runnable`。因此，这里的 Lambda 不接收 Argument，也没有 Return Statement；如果它不符合 `Runnable` Contract，代码将无法 Compile。

Capturing State in Java Lambda Expressions / 在 Java Lambda Expression 中捕获 State

In the above example, our function body references an object that lives outside of the function itself (namely, the `drivetrain` object). This is called a "capture" of a variable from the surrounding code (which is sometimes called the "outer scope" or "enclosing scope"). Usually the captured variables are either local variables from the enclosing method body in which the lambda expression is defined, or else fields of an enclosing class definition in which that method is defined.

在上方示例中，Function Body 引用了存在于 Function 本身之外的 Object（即 `drivetrain` Object）。这称为从周围代码（有时称为 “Outer Scope” 或 “Enclosing Scope”）“Capture” Variable。被 Capture 的 Variable 通常是定义 Lambda Expression 的 Enclosing Method Body 中的 Local Variable，或定义该 Method 的 Enclosing Class Definition 中的 Field。

In Java capturing state is a fairly safe thing to do in general, with one major caveat: we can only capture state that is "effectively final". That means it is only legal to capture a variable from the enclosing scope if that variable is never reassigned after initialization. Note that this does not mean that the captured state cannot change: Remember that Java objects are references, so the object that the reference *points to* may change after capture - but the reference itself cannot be made to point to another object.

在 Java 中，Capture State 通常相当安全，但有一个重要限制：只能 Capture “Effectively Final” 的 State。这意味着，只有 Enclosing Scope 中的 Variable 在 Initialization 后从未被重新赋值时，才可以合法地 Capture 它。请注意，这并不意味着被 Capture 的 State 不能改变：Java Object 使用 Reference，因此 Reference 指向的 Object 可以在 Capture 后发生变化，但 Reference 本身不能改为指向另一个 Object。

This means we can only capture primitive types (like `int`, `double`, and `boolean`) if they're constants. If we want to capture a state variable that can change, it *must be wrapped in a mutable object*.

这意味着，只有 Primitive Type（例如 `int`、`double` 和 `boolean`）为 Constant 时，才能 Capture 它。如果希望 Capture 可以变化的 State Variable，它必须封装在 *Mutable Object* 中。

Syntactic Sugar for Java Lambda Expressions / Java Lambda Expression 的 Syntactic Sugar

The full lambda expression syntax can be needlessly verbose in some cases. To help with this, Java lets us take some shortcuts (called "syntactic sugar") in cases where some of the notation is redundant.

在某些情况下，完整的 Lambda Expression Syntax 会显得不必要地冗长。为此，当部分记号属于冗余内容时，Java 允许使用一些简写方式（称为“Syntactic Sugar”）。

Omitting Function Body Brackets for One-Line Lambdas / 单行 Lambda 省略 Function Body Braces

If the function body of our lambda expression is only one line, Java lets us omit the brackets around the function body. When omitting function brackets, we also omit trailing semicolons And the `return` keyword.

如果 Lambda Expression 的 Function Body 只有一行，Java 允许省略 Function Body 周围的 Braces。省略 Function Braces 时，也要省略末尾的 Semicolon 和 `return` Keyword。

So, our `Runnable` lambda above could instead be written:

因此，上方的 `Runnable` Lambda 也可以写为：

```
// Create an InstantCommand that runs the drive forward at half speed
Command driveHalfSpeed = runOnce(() -> drivetrain.arcadeDrive(0.5, 0.0), drivetrain);
```

Omitting Parentheses around Single Lambda Parameters / 省略单个 Lambda Parameter 周围的 Parentheses

If the lambda expression is for a functional interface that takes only a single argument, we can omit the parenthesis around the parameter list:

如果 Lambda Expression 对应的 Functional Interface 只接收一个 Argument，就可以省略 Parameter List 周围的 Parentheses：

```
// We can write this lambda with no parenthesis around its single argument
IntConsumer exampleLambda = (a -> System.out.println(a));
```

Get Alliance Color / 获取 Alliance Color

The `DriverStation` class (Java) has many useful features for getting data from the Driver Station computer. One of the most important features is `getAlliance`.

`DriverStation` 类（Java）提供了许多从 Driver Station 计算机获取数据的实用功能。其中最重要的功能之一是 `getAlliance`。

Note that there are three cases: red, blue, and no color yet. It is important that code handles the third case correctly because the alliance color will not be available until the Driver Station connects. In particular, code should not assume that the alliance color will be available during constructor methods or `robotInit`, but it should be available by the time `autoInit` or `teleopInit` is called. FMS will set the alliance color automatically; when not connected to FMS, the alliance color can be set from the Driver Station (see "Team Station" on the Operation Tab).

请注意，共有三种情况：红方、蓝方和尚未获得颜色。代码必须正确处理第三种情况，因为在 Driver Station 建立连接前，Alliance Color 不可用。尤其需要注意，代码不应假定在构造方法或 `robotInit` 执行期间已经能够获得 Alliance Color，但在调用 `autoInit` 或 `teleopInit` 时通常应当已经可用。FMS 会自动设置 Alliance Color；未连接到 FMS 时，可以通过 Driver Station 设置 Alliance Color（参阅 Operation Tab 中的“Team Station”）。

Getting your Alliance Color and Doing an Action / 获取 Alliance Color 并执行操作

```
Optional<Alliance> ally = DriverStation.getAlliance();
if (ally.isPresent()) {
    if (ally.get() == Alliance.Red) {
        <RED ACTION>
    }
    if (ally.get() == Alliance.Blue) {
        <BLUE ACTION>
    }
}
else {
    <NO COLOR YET ACTION>
}
```

Java Garbage Collection / Java 垃圾回收

Java garbage collection is the process of automatically managing memory for Java objects. The Java Virtual Machine (JVM) is responsible for creating and destroying objects, and the garbage collector is responsible for identifying and reclaiming unused objects.

Java 垃圾回收是自动管理 Java Object Memory 的过程。Java Virtual Machine (JVM) 负责创建和销毁 Object，Garbage Collector 负责识别并回收不再使用的 Object。

Java garbage collection is an automatic process, which means that the programmer does not need to explicitly deallocate memory. The garbage collector keeps track of which objects are in use and which are not, and it periodically reclaims unused objects.

Java 垃圾回收是自动进行的，这意味着 Programmer 不需要显式释放 Memory。Garbage Collector 会跟踪哪些 Object 正在使用、哪些没有使用，并定期回收未使用的 Object。

Object Creation / 创建 Object

Creating a large number of objects in Java can lead to memory and performance issues. While the Java Garbage Collector (GC) is designed to handle memory management efficiently, creating too many objects can overwhelm the GC and cause performance degradation.

在 Java 中创建大量 Object 可能导致 Memory 和 Performance 问题。虽然 Java Garbage Collector (GC) 旨在高效处理 Memory Management，但创建过多 Object 可能使 GC 不堪重负，并导致 Performance 下降。

Memory Concerns / Memory 问题

When a large number of objects are created, it increases the overall memory footprint of the application. While the overhead for a single object may be insignificant, it can become substantial when multiplied by a large number of objects.

创建大量 Object 会增大 Application 的总体 Memory Footprint。单个 Object 的 Overhead 可能微不足道，但在乘以大量 Object 后可能变得非常可观。

NOTE: VisualVM can be used to see where memory is allocated.

说明： 可以使用 VisualVM 查看 Memory 分配位置。

Performance Concerns / Performance 问题

The GC's job is to periodically identify and reclaim unused objects in memory. While garbage collection is running on an FRC robot coded in Java, execution of the robot program is paused. When the GC has to

collect a large number of objects, it has to pause the application to run more frequently or for longer periods of time. This is because the GC has to perform more work to collect and process each object.

GC 的工作是定期识别并回收 Memory 中未使用的 Object。在使用 Java 编程的 FRC Robot 上运行 Garbage Collection 时，Robot Program 的执行会暂停。当 GC 必须回收大量 Object 时，它需要更频繁地暂停 Application，或使每次暂停持续更长时间。这是因为 GC 必须执行更多工作来回收和处理每个 Object。

GC-related performance degradation in robot programs can manifest as occasional pauses, freezes, or loop overruns as the GC works to reclaim memory.

当 GC 回收 Memory 时，与 GC 相关的 Robot Program Performance 下降可能表现为偶发暂停、卡死或 Loop Overrun。

Design Considerations / 设计注意事项

If you anticipate your application creating a large number of short-lived objects, it is important to consider design strategies to mitigate the potential memory and performance issues. Here are some strategies to consider:

如果预计 Application 会创建大量生命周期很短的 Object，就必须考虑能够减轻潜在 Memory 和 Performance 问题的设计策略。可以考虑以下策略：

- Minimize object creation: Carefully evaluate the need for each object creation. If possible, reuse existing objects or use alternative data structures, such as arrays or primitives, to avoid creating new objects.
- 尽量减少 Object 创建：仔细评估每次创建 Object 的必要性。如果可能，应复用现有 Object，或使用 Array、Primitive 等替代 Data Structure，以避免创建新 Object。
- Efficient data structures: Use data structures that are well-suited for the type of data you are working with. For example, if you are dealing with a large number of primitive values, consider using arrays or collections specifically designed for primitives.
- 高效的 Data Structure：使用适合所处理 Data Type 的 Data Structure。例如，如果要处理大量 Primitive Value，可以考虑使用 Array 或专门为 Primitive 设计的 Collection。

Diagnosing Out of Memory Errors with Heap Dumps / 使用 Heap Dump 诊断 Out of Memory Error

All objects in Java are retained in a section of memory called the *heap*. As objects typically consume the greatest amount of memory in a Java program, it is often useful to take a snapshot of the state of the heap---a heap dump---to analyze memory issues. Heap dumps only capture the state of a program's heap at a single point in time, so they are unlikely to be useful if not captured exactly at the time the program is experiencing memory issues.

Java 中的所有 Object 都保存在称为 *Heap* 的 Memory 区域中。由于 Object 通常占用 Java Program 中最多的 Memory，因此常常可以通过拍摄 Heap 状态快照——Heap Dump——来分析 Memory 问题。Heap Dump 只

捕获 Program Heap 在单一时间点的状态，因此如果未能恰好在 Program 出现 Memory 问题时捕获，它很可能无法发挥作用。

Since `OutOfMemoryErrors` both crash the program and are a common reason to want a heap dump, the JVM can be configured to automatically take a heap dump the moment an `OutOfMemoryError` is caught by the JVM. To configure these options, locate the `frcJava` code block in your project's `build.gradle`:

由于 `OutOfMemoryError` 会导致 Program 崩溃，并且也是需要 Heap Dump 的常见原因，因此可以将 JVM 配置为在捕获到 `OutOfMemoryError` 的瞬间自动生成 Heap Dump。要配置这些选项，请在 Project 的 `build.gradle` 中找到 `frcJava` Code Block:

```

deploy {
    targets {
        roborio(getTargetTypeClass('RoboRIO')) {
            // Team number is loaded either from the .wpilib/wpilib_preferences.json
            // or from command line. If not found an exception will be thrown.
            // You can use getTeamOrDefault(team) instead of getTeamNumber if you
            // want to store a team number in this file.
            team = project.frc.getTeamNumber()
            debug = project.frc.getDebugOrDefault(false)

            artifacts {
                // First part is artifact name, 2nd is artifact type
                // getTargetTypeClass is a shortcut to get the class type using a string

                frcJava(getArtifactTypeClass('FRCJavaArtifact')) {
                }

                // Static files artifact
                frcStaticFileDeploy(getArtifactTypeClass('FileTreeArtifact')) {
                    files = project.fileTree('src/main/deploy')
                    directory = '/home/lvuser/deploy'
                    deleteOldFiles = false // Change to true to delete files on roboRIO that no
                                           // longer exist in deploy directory of this project
                }
            }
        }
    }
}

```

Add to the code block so that it contains two `jvmArgs` commands, as shown below:

向该 Code Block 添加内容，使其包含两个 `jvmArgs` 命令，如下所示：

```

frcJava(getArtifactTypeClass('FRCJavaArtifact')) {
    // If you have other configuration here, you do not need to remove it.
    // Enable automatic heap dumps on OutOfMemoryError
    // Note: the heap dump path here is a path on a USB flash drive, see below
    jvmArgs.add("-XX:+HeapDumpOnOutOfMemoryError")
    jvmArgs.add("-XX:HeapDumpPath=/u/frc-usercode.hprof")
}

```

This will cause the JVM to write heap dumps to a file named `frc-usercode.hprof` at the root of a USB flash drive attached to the roboRIO when the code runs out of memory. It is recommended to save these heap dumps to a USB flash drive because heap dumps intrinsically consume the same amount of space on disk as the program heap did in memory when the program crashed, and are likely to be larger than the roboRIO's internal storage has capacity for. Once you have reproduced the `OutOfMemoryError`, redeploy

your code without these options enabled, and use the USB flash drive to transfer the heap dump to a computer for analysis in a memory profiler such as VisualVM.

当 Code 耗尽 Memory 时, 这会使 JVM 将 Heap Dump 写入连接到 roboRIO 的 USB Flash Drive Root 中名为 `frc-usercode.hprof` 的文件。建议将这些 Heap Dump 保存到 USB Flash Drive, 因为 Heap Dump 在 Disk 上占用的空间与 Program 崩溃时 Program Heap 在 Memory 中占用的空间基本相同, 并且很可能大于 roboRIO Internal Storage 的可用容量。复现 `OutOfMemoryError` 后, 请在不启用这些选项的情况下重新 Deploy Code, 再使用 USB Flash Drive 将 Heap Dump 传输到 Computer, 并通过 VisualVM 等 Memory Profiler 进行分析。

WARNING: Configuring the JVM this way requires that the flash drive remain connected to the roboRIO while your code is running.

警告: 以这种方式配置 JVM 时, 必须在 Code 运行期间始终保持 Flash Drive 与 roboRIO 连接。

Larger SD cards may provide enough onboard storage to allow the use of these options on the roboRIO 2 without a USB flash drive. To do this, set the `-XX:HeapDumpPath` option to reference a path on the SD card, and use FTP/SFTP to transfer the heap dump to a computer before deleting it from the SD card.

容量较大的 SD Card 可能提供足够的 Onboard Storage, 使 roboRIO 2 无需 USB Flash Drive 即可使用这些选项。为此, 请将 `-XX:HeapDumpPath` 选项设置为指向 SD Card 上的路径, 并使用 FTP/SFTP 将 Heap Dump 传输到 Computer, 然后再从 SD Card 中删除它。

Note that the JVM will **not** overwrite heap dumps with the exact path and filename specified by `-XX:HeapDumpPath` if they already exist, nor will it dump the process heap to a file with a different name. If a path to a directory is supplied instead of a path to a file, the JVM will instead write out heap dumps with unique filenames within the specified directory, with the name `java_pidNNNN.hprof`, where `NNNN` is the process ID of the JVM that ran out of memory. Note that this can cause large files to build up on disk if they are not cleaned out, so if you configure the JVM this way, be sure to frequently copy heap dumps to a computer and delete them from the flash drive/SD card afterward.

请注意, 如果 `-XX:HeapDumpPath` 指定的准确路径和文件名已经存在, JVM **不会**覆盖该 Heap Dump, 也不会将 Process Heap Dump 到具有其他名称的文件中。如果提供的是 Directory Path 而不是 File Path, JVM 会在指定 Directory 内以唯一文件名写出 Heap Dump, 名称为 `java_pidNNNN.hprof`, 其中 `NNNN` 是发生 Out of Memory 的 JVM Process ID。请注意, 如果不清理, 这可能导致 Disk 上积累大量大文件。因此, 如果以这种方式配置 JVM, 请务必频繁将 Heap Dump 复制到 Computer, 随后从 Flash Drive 或 SD Card 中删除它们。

CAUTION: Always be vigilant about the amount of available space on the underlying storage medium while you use this feature.

Use of this feature is not recommended during competitive play.

注意: 使用此功能时, 应始终密切关注底层 Storage Medium 的可用空间。

不建议在正式比赛期间使用此功能。

System Memory Tuning / System Memory 调优

If the JVM cannot allocate memory, the program will be terminated. As an embedded system with only a small amount of memory available (256 MB on the roboRIO 1, 512 MB on the roboRIO 2), the roboRIO is particularly susceptible to running out of memory.

如果 JVM 无法分配 Memory，Program 将被终止。roboRIO 是可用 Memory 很少的 Embedded System (roboRIO 1 为 256 MB，roboRIO 2 为 512 MB)，因此尤其容易耗尽 Memory。

No amount of system tuning can fix out of memory errors caused by out-of-control allocations.

If you are running out of memory, always investigate allocations with heap dumps and/or VisualVM first.

无论进行多少 System Tuning，都无法修复由失控分配导致的 Out of Memory Error。

如果 Memory 耗尽，应始终先使用 Heap Dump 和/或 VisualVM 调查 Allocation。

If you continue to run out of memory even after investigating with VisualVM and taking steps to minimize the number of allocated objects, a few different options are available to make additional memory available to the robot program.

如果在使用 VisualVM 调查并采取措施尽量减少已分配 Object 的数量后仍然耗尽 Memory，可以使用几种不同选项，为 Robot Program 提供更多 Memory。

- Disabling the system web server
- 禁用 System Web Server
- Setting sysctls (Linux kernel options)
- 设置 sysctl (Linux Kernel Option)
- Periodically calling the garbage collector
- 定期调用 Garbage Collector
- Setting up swap on a USB flash drive
- 在 USB Flash Drive 上设置 Swap

Implementing most of these options require connecting with SSH to the roboRIO and running commands. If run incorrectly, it may require a reimage to recover, so be careful when following the instructions.

实现这些选项中的大多数都需要通过 SSH 连接到 roboRIO 并运行命令。如果命令运行不正确，可能需要重新刷写 Image 才能恢复，因此请谨慎遵循说明。

Disabling the System Web Server / 禁用 System Web Server

The built-in NI system web server provides the webpage (the roboRIO Web Dashboard) seen when using a web browser to connect to the roboRIO, e.g. to change IP address settings. It also is used by the Driver

Station's data log download functionality. However, it consumes several MB of RAM, so disabling it will free up that memory for the robot program to use. There are several ways to disable the web server:

内置 NI System Web Server 提供使用 Web Browser 连接 roboRIO 时看到的网页 (roboRIO Web Dashboard)，例如用于更改 IP Address Setting。Driver Station 的 Data Log Download 功能也会使用它。不过，它会占用数 MB RAM，因此禁用它可以释放这些 Memory 供 Robot Program 使用。可以通过多种方式禁用 Web Server：

The first and easiest is to use the RoboRIO Team Number Setter tool. Versions 2024.2.1 and later of the tool have a button to disable or enable the web server. However, a few teams have reported that this does not work or does not persist between reboots. There are two alternate ways to disable the web server; both require connecting to the roboRIO with SSH and logging in as the `admin` user.

第一种也是最简单的方式是使用 RoboRIO Team Number Setter 工具。该工具的 2024.2.1 及更高版本提供了禁用或启用 Web Server 的按钮。不过，少数队伍报告该功能无效，或无法在 Reboot 后保持设置。还有两种禁用 Web Server 的替代方式；两者都需要通过 SSH 连接到 roboRIO，并以 `admin` User 登录。

1. Run `/etc/init.d/systemWebServer stop; update-rc.d -f systemWebServer remove; sync`
2. Run `chmod a-x /usr/local/natinst/etc/init.d/systemWebServer; sync`
3. 运行 `/etc/init.d/systemWebServer stop; update-rc.d -f systemWebServer remove; sync`
4. 运行 `chmod a-x /usr/local/natinst/etc/init.d/systemWebServer; sync`

To revert the alternate ways and re-enable the web server, take the corresponding step:

要撤销替代方式并重新启用 Web Server，请执行相应步骤：

1. Run `update-rc.d -f systemWebServer defaults; /etc/init.d/systemWebServer start; sync`
2. Run `chmod a+x /usr/local/natinst/etc/init.d/systemWebServer; sync`
3. 运行 `update-rc.d -f systemWebServer defaults; /etc/init.d/systemWebServer start; sync`
4. 运行 `chmod a+x /usr/local/natinst/etc/init.d/systemWebServer; sync`

Setting sysctls / 设置 sysctl

Several Linux kernel options (called sysctls) can be set to tweak how the kernel allocates memory. Several options have been found to reduce out-of-memory errors:

可以设置若干 Linux Kernel Option（称为 sysctl），以调整 Kernel 分配 Memory 的方式。已有若干 Option 被发现能够减少 Out of Memory Error：

- Setting `vm.overcommit_memory` to 1 (the default value is 2). This causes the kernel to always pretend there is enough memory for a requested memory allocation at the time of allocation; the default setting always checks to see if there's actually enough memory to back an allocation at the time of allocation, not when the memory is actually used.
- 将 `vm.overcommit_memory` 设置为 1（Default Value 为 2）。这会使 Kernel 在请求 Memory Allocation 时始终假定有足够的 Memory；Default Setting 会在执行 Allocation 时检查是否确实有足够 Memory 支持该

Allocation，而不是等到实际使用 Memory 时再检查。

- Setting `vm.vfs_cache_pressure` to 1000 (the default value is 100). Increasing this causes the kernel to much more aggressively reclaim file system object caches; it may slightly degrade performance.
- 将 `vm.vfs_cache_pressure` 设置为 1000（Default Value 为 100）。提高该值会使 Kernel 更积极地回收 File System Object Cache；这可能会略微降低 Performance。
- Setting `vm.swappiness` to 100 (the default value is 60). This causes the kernel to more aggressively swap process memory to the swap file. Changing this option has no effect unless you add a swap file.
- 将 `vm.swappiness` 设置为 100（Default Value 为 60）。这会使 Kernel 更积极地将 Process Memory 换出到 Swap File。除非添加 Swap File，否则更改此 Option 不会产生作用。

You can set some or all of these options; the most important one is `vm.overcommit_memory`. Setting these options requires connecting to the roboRIO with SSH and logging in as the `admin` user, then running the following commands:

可以设置其中部分或全部 Option；最重要的是 `vm.overcommit_memory`。设置这些 Option 需要通过 SSH 连接到 roboRIO，以 `admin` User 登录，然后运行以下命令：

```
echo "vm.overcommit_memory=1" >> /etc/sysctl.conf
echo "vm.vfs_cache_pressure=1000" >> /etc/sysctl.conf
echo "vm.swappiness=100" >> /etc/sysctl.conf
sync
```

The `/etc/sysctl.conf` file should contain the following lines at the end when done (to check, you can run the command `cat /etc/sysctl.conf`):

完成后，`/etc/sysctl.conf` 文件末尾应包含以下各行（可以运行命令 `cat /etc/sysctl.conf` 检查）：

```
vm.overcommit_memory=1
vm.vfs_cache_pressure=1000
vm.swappiness=100
```

To revert the change, edit `/etc/sysctl.conf` (this will require the use of the vi editor) and remove these 3 lines.

要撤销更改，请编辑 `/etc/sysctl.conf`（这需要使用 vi Editor），并删除这 3 行。

Periodically Calling the Garbage Collector / 定期调用 Garbage Collector

Sometimes the garbage collector won't run frequently enough to keep up with the quantity of allocations. As Java provides a way to trigger a garbage collection to occur, running it on a periodic basis may reduce peak memory usage. This can be done by adding a `Timer` and a periodic check:

有时 Garbage Collector 的运行频率不足以跟上 Allocation 数量。由于 Java 提供了触发 Garbage Collection 的方式，定期运行它可能会降低 Peak Memory Usage。可以通过添加 `Timer` 和 Periodic Check 来实现：

```

Timer m_gcTimer = new Timer();
public Robot() {
    m_gcTimer.start();
}
public void periodic() {
    // run the garbage collector every 5 seconds
    if (m_gcTimer.advanceIfElapsed(5)) {
        System.gc();
    }
}
}

```

Setting Up Swap on a USB Flash Drive / 在 USB Flash Drive 上设置 Swap

A swap file on a Linux system provides disk-backed space that can be used by the system as additional virtual memory to put infrequently used data and programs when they aren't being used, freeing up physical RAM for active use such as the robot program. It is strongly recommended to not use the built-in non-replaceable flash storage on the roboRIO 1 for a swap file, as it has very limited write cycles and may wear out quickly. Instead, however, a FAT32-formatted USB flash drive may be used for this purpose. This does require the USB flash drive to always be plugged into the roboRIO before boot.

Linux System 上的 Swap File 提供由 Disk 支持的空间，System 可以将其用作额外的 Virtual Memory，在不使用某些不常用的 Data 和 Program 时将其放入其中，从而释放 Physical RAM 供 Robot Program 等活动任务使用。强烈建议不要使用 roboRIO 1 内置且不可更换的 Flash Storage 存放 Swap File，因为其 Write Cycle 非常有限，可能很快磨损。可以改用 FAT32 格式的 USB Flash Drive。但这要求每次 Boot 前都将 USB Flash Drive 插入 roboRIO。

CAUTION: Having a swap file on a USB stick means it's critical the USB stick stay connected to the roboRIO at all times it is powered.

This should be used as a last resort if none of the other steps above help. Generally needing swap is indicative of some other allocation issue, so use VisualVM first to optimize allocations.

注意： 在 USB Stick 上设置 Swap File 意味着只要 roboRIO 处于 Power On 状态，USB Stick 就必须始终保持连接。

只有在上述其他步骤均无帮助时，才应将此方法作为最后手段。通常，需要 Swap 表明存在其他 Allocation 问题，因此应先使用 VisualVM 优化 Allocation。

A swap file can be set up by plugging the USB flash drive into the roboRIO USB port, connecting to the roboRIO with SSH and logging in as the `admin` user, and running the following commands. Note the `vi` step requires knowledge of how to edit and save a file in `vi`.

可以通过以下方式设置 Swap File：将 USB Flash Drive 插入 roboRIO USB Port，通过 SSH 连接 roboRIO 并以 `admin` User 登录，然后运行以下命令。请注意，`vi` 步骤要求了解如何在 `vi` 中编辑和保存文件。

```
fallocate -l 100M /u/swapfile
mkswap /u/swapfile
swapon /u/swapfile
vi /etc/init.d/addswap.sh
chmod a+x /etc/init.d/addswap.sh
update-rc.d -v addswap.sh defaults
sync
```

The `/etc/init.d/addswap.sh` file contents should look like this:

`/etc/init.d/addswap.sh` 文件内容应如下所示：

```
#!/bin/sh
[ -x /sbin/swapon ] && swapon -e /u/swapfile
: exit 0
```

To revert the change, run `update-rc.d -f addswap.sh remove; rm /etc/init.d/addswap.sh; sync; reboot`.

要撤销更改，请运行 `update-rc.d -f addswap.sh remove; rm /etc/init.d/addswap.sh; sync; reboot`。